1. Basic Syntax and Fundamentals - វាក្យសម្ព័ន្ធមូលដ្ឋាន និង មូលដ្ឋានគ្រឹះ

វាក្យសម្ព័ន្ធមូលដ្ឋាន និង មូលដ្ឋានគ្រឹះនៃភាសា JavaScript គឺជាចំណុចសំខាន់សម្រាប់អ្នកចាប់ផ្តើមរៀនសរសេរកម្មវិធី។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីរបៀបប្រើ variables, constants, data types, type conversion, operators, និង comments ដើម្បីបង្កើតកម្មវិធីដែលមានប្រសិទ្ធភាព។ រាល់គោលគំនិតនឹងត្រូវបានបង្ហាញជាមួយឧទាហរណ៍កូដ និង ការពន្យល់ច្បាស់លាស់ដើម្បីជួយអ្នកយល់ពីមូលដ្ឋានគ្រឹះនៃ JavaScript យ៉ាងជាក់លាក់។

1.1 Variables and Constants (let, const, var) - អថេរ និង ថេរ

អថេរ (variables) និង ថេរ (constants) ត្រូវបានប្រើដើម្បីរក្សាទុកទិន្នន័យនៅក្នុង JavaScript។ អថេរអនុញ្ញាតឱ្យអ្នកផ្លាស់ប្តូរតម្លៃបាន ចំណែកថេរមិនអនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃបន្ទាប់ពីកំណត់រួច។ JavaScript មានបីវិធីសម្រាប់ប្រកាសអថេរ និង ថេរ៖ let, const, និង var។

ឧទាហរណ៍៖

```
// ប្រើ let សម្រាប់អថេរដែលអាចផ្លាស់ប្តូរតម្លៃ
let age = 25;
age = 26; // អាចផ្លាស់ប្តូរបាន

// ប្រើ const សម្រាប់ថេរដែលមិនអាចផ្លាស់ប្តូរតម្លៃ
const name = "Sokha";
name = "Vannak"; // នឹងបណ្តាលឱ្យមានកំហុស (Error)

// ប្រើ var (វិធីចាស់ មិនសូវប្រើទំនើប)
var score = 100;
score = 150; // អាចផ្លាស់ប្តូរបាន

console.log(age);    // 26
console.log(name);   // Sokha
console.log(score);  // 150
```

ការពន្យល់៖ let អនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃអថេរ ប៉ុន្តែមាន block scope។ const មិនអនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃបន្ទាប់ពីកំណត់ និង មាន block scope។ var ជាវិធីចាស់ មាន function scope និង អាចបណ្តាលឱ្យមានបញ្ហាក្នុងកម្មវិធីធំ ដូច្នេះគួរជៀសវាងប្រើ។

លទ្ធផល៖

```
26
Sokha
150
```

1.2 Data Types (String, Number, Boolean, Object, Array, Null, Undefined, Symbol) - ប្រភេទទិន្នន័យ

JavaScript មានប្រភេទទិន្នន័យផ្សេងៗសម្រាប់រក្សាទុកទិន្នន័យ។ ប្រភេទទិន្នន័យទាំងនេះរួមមាន `String`, `Number`, `Boolean`, `Object`, `Array`, `Null`, `Undefined`, និង `Symbol`។ ការយល់ដឹងអំពីប្រភេទទិន្នន័យគឺសំខាន់សម្រាប់ការគ្រប់គ្រងទិន្នន័យឱ្យបានត្រឹមត្រូវ។

ឧទាហរណ៍៖

```
// String: អក្សរឬឃ្លា
let greeting = "សួស្តី";

// Number: លេខ (ទាំងគត់ និង ទសភាគ)
let price = 99.99;

// Boolean: តម្លៃ True ឬ False
let isStudent = true;

// Object: ទិន្នន័យដែលមានលក្ខណៈសម្បត្តិ
let person = { name: "Sokha", age: 25 };

// Array: បញ្ជីទិន្នន័យ
let fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];

// Null: គ្មានតម្លៃ
let empty = null;

// Undefined: អថេរដែលមិនបានកំណត់តម្លៃ
let notAssigned;

// Symbol: តម្លៃតែមួយគត់
let id = Symbol("id");

console.log(typeof greeting); // string
```

```
console.log(typeof price);    // number
console.log(typeof isStudent); // boolean
console.log(typeof person);   // object
console.log(typeof fruits);   // object (Array ជា object)
console.log(typeof empty);    // object (បច្ចុប្បន្ន null ត្រូវបានចាត់ទុកជា obj
console.log(typeof notAssigned); // undefined
console.log(typeof id);       // symbol
```

ការពន្យល់៖ typeof ត្រូវបានប្រើដើម្បីពិនិត្យប្រភេទទិន្នន័យ។ String សម្រាប់អក្សរ, Number សម្រាប់លេខ, Boolean សម្រាប់តម្លៃពិត/មិនពិត, Object សម្រាប់ទិន្នន័យស្មុគស្មាញ, Array សម្រាប់បញ្ជី, Null សម្រាប់គ្មានតម្លៃ, Undefined សម្រាប់អថេរមិនទាន់កំណត់, និង Symbol សម្រាប់តម្លៃតែមួយគត់។

លទ្ធផល៖

```
string
number
boolean
object
object
object
undefined
symbol
```

1.3 Type Conversion (Implicit and Explicit) - ការបម្លែងប្រភេទទិន្នន័យ

ការបម្លែងប្រភេទទិន្នន័យ (type conversion) គឺជាដំណើរការផ្លាស់ប្តូរទិន្នន័យពីប្រភេទមួយទៅប្រភេទមួយទៀត។ មានពីរប្រភេទ៖ implicit (ការបម្លែងដោយស្វ័យប្រវត្តិ) និង explicit (ការបម្លែងដោយចេតនា)។ ការយល់ដឹងអំពីការបម្លែងទាំងនេះជួយការពារកំហុសនៅក្នុងកម្មវិធី។

ឧទាហរណ៍៖

```
// Implicit Conversion
let result = "5" + 3; // String + Number = String
console.log(result); // "53"

let sum = "10" - 2; // String - Number = Number
console.log(sum); // 8

// Explicit Conversion
```

```
let str = "123";
let num = Number(str); // បម្លែង String ទៅ Number
console.log(num); // 123

let value = 456;
let text = String(value); // បម្លែង Number ទៅ String
console.log(text); // "456"

let truth = "true";
let bool = Boolean(truth); // បម្លែង String ទៅ Boolean
console.log(bool); // true
```

ការពន្យល់៖ ក្នុង `implicit conversion`, JavaScript បម្លែងទិន្នន័យដោយស្វ័យប្រវត្តិ (ឧ. `"5" + 3` បណ្តាលឱ្យ `"53"` ព្រោះ + បម្លែង 3 ទៅជា String)។ ក្នុង `explicit conversion`, យើងប្រើ `Number()`, `String()`, ឬ `Boolean()` ដើម្បីបម្លែងទិន្នន័យដោយចេតនា។

លទ្ធផល៖

```
53
8
123
456
true
```

1.4 Operators (Arithmetic, Comparison, Logical, Assignment, Ternary, Bitwise, String, Type, Special) - ប្រតិបត្តិករ

ប្រតិបត្តិករ (operators) គឺជាសញ្ញា ឬ ពាក្យគន្លឹះដែលប្រើសម្រាប់ធ្វើប្រតិបត្តិការលើទិន្នន័យនៅក្នុង JavaScript។ ប្រតិបត្តិករត្រូវបានបែងចែកជាប្រភេទផ្សេងៗ ដូចជា `arithmetic` (គណនា), `comparison` (ប្រៀបធៀប), `logical` (តក្កវិជ្ជា), `assignment` (កំណត់តម្លៃ), `ternary` (លក្ខខណ្ឌសង្ខេប), `bitwise` (ប្រតិបត្តិការប៊ីត), `string` (អក្សរ), `type` (ប្រភេទទិន្នន័យ), និង `special` (ប្រតិបត្តិករពិសេស)។ ផ្នែកនេះនឹងផ្តល់នូវការពន្យល់លម្អិត និង ឧទាហរណ៍សម្រាប់ប្រតិបត្តិករទាំងអស់។

Arithmetic Operators - ប្រតិបត្តិករគណនា

ប្រតិបត្តិករគណនា (`arithmetic operators`) ត្រូវបានប្រើសម្រាប់ការគណនាគណិតវិទ្យា ដូចជា ការបូក ដក គុណ ចែក និង សំណល់។

ឧទាហរណ៍៖

```
let a = 10;
let b = 3;

console.log(a + b);  // 13 (បូក)
console.log(a - b);  // 7 (ដក)
console.log(a * b);  // 30 (គុណ)
console.log(a / b);  // 3.3333333333333335 (ចែក)
console.log(a % b);  // 1 (សំណល់)
console.log(a ** b); // 1000 (ស្វ័យគុណ)
console.log(++a);    // 11 (បន្ថែម 1)
console.log(--b);    // 2 (ដក 1)
```

ការពន្យល់៖ + បូក, - ដក, * គុណ, / ចែក, % សំណល់, ** ស្វ័យគុណ, ++ បន្ថែម 1, និង -- ដក 1។ ប្រតិបត្តិករ ++ និង -- អាចប្រើជា prefix (ឧ. ++a) ឬ postfix (ឧ. a++)។

លទ្ធផល៖

```
13
7
30
3.3333333333333335
1
1000
11
2
```

Comparison Operators - ប្រតិបត្តិករប្រៀបធៀប

ប្រតិបត្តិករប្រៀបធៀប (comparison operators) ត្រូវបានប្រើដើម្បីប្រៀបធៀបតម្លៃពីរ ហើយប្រគល់តម្លៃ true ឬ false។

ឧទាហរណ៍៖

```
let x = 5;
let y = "5";

console.log(x == y);   // true (ស្មើ មិនខ្វល់ប្រភេទ)
console.log(x === y);  // false (ស្មើ និង ប្រភេទដូចគ្នា)
console.log(x != y);   // false (មិនស្មើ)
console.log(x !== y);  // true (មិនស្មើ ឬ ប្រភេទខុសគ្នា)
console.log(x > y);    // false (ធំជាង)
```

```
console.log(x >= y);   // true (ធំជាងឬស្មើ)
console.log(x < y);    // false (តូចជាង)
console.log(x <= y);   // true (តូចជាងឬស្មើ)
```

ការពន្យល់៖ == ប្រៀបធៀបតម្លៃ, === ប្រៀបធៀបទាំងតម្លៃនិងប្រភេទ, != ពិនិត្យមិនស្មើ, !== ពិនិត្យមិនស្មើឬប្រភេទខុសគ្នា, >, >=, <, <= ប្រៀបធៀបទំហំ។

លទ្ធផល៖

```
true
false
false
true
false
true
false
true
```

Logical Operators - ប្រតិបត្តិករតក្កវិជ្ជា

ប្រតិបត្តិករតក្កវិជ្ជា (`logical operators`) ត្រូវបានប្រើដើម្បីផ្សំលក្ខខណ្ឌ ឬ ធ្វើប្រតិបត្តិការលើតម្លៃ `true` ឬ `false`។

ឧទាហរណ៍៖

```
let isAdult = true;
let hasID = false;

console.log(isAdult && hasID); // false (AND: ទាំងពីរត្រូវតែពិត)
console.log(isAdult || hasID); // true (OR: យ៉ាងហោចមួយពិត)
console.log(!isAdult);         // false (NOT: ប្តូរតម្លៃ)
```

ការពន្យល់៖ && ត្រូវការលក្ខខណ្ឌទាំងពីរពិត, || ត្រូវការយ៉ាងហោចមួយពិត, ! ប្តូរតម្លៃពី `true` ទៅ `false` ឬផ្ទុយមកវិញ។

លទ្ធផល៖

```
false
true
false
```

Assignment Operators - ប្រតិបត្តិករកំណត់តម្លៃ

ប្រតិបត្តិករកំណត់តម្លៃ (`assignment operators`) ត្រូវបានប្រើដើម្បីកំណត់តម្លៃទៅអថេរ ឬ ធ្វើប្រតិបត្តិការហើយកំណត់តម្លៃក្នុងពេលតែមួយ។

ឧទាហរណ៍៖

```
let x = 5;

x += 2;  // x = x + 2
console.log(x); // 7

x -= 3;  // x = x - 3
console.log(x); // 4

x *= 2;  // x = x * 2
console.log(x); // 8

x /= 4;  // x = x / 4
console.log(x); // 2

x %= 3;  // x = x % 3
console.log(x); // 2

x **= 2; // x = x ** 2
console.log(x); // 4
```

ការពន្យល់៖ = កំណត់តម្លៃ, +=, -=, *=, /=, %=, **= ធ្វើប្រតិបត្តិការហើយកំណត់តម្លៃថ្មីទៅអថេរ។

លទ្ធផល៖

```
7
4
8
2
2
4
```

Ternary Operator - ប្រតិបត្តិករលក្ខខណ្ឌសង្ខេប

ប្រតិបត្តិករលក្ខខណ្ឌសង្ខេប (`ternary operator`) គឺជាវិធីសង្ខេបសម្រាប់សរសេរ `if-else` ដោយប្រើសញ្ញា `?:`។

ឧទាហរណ៍៖

```
let age = 20;
let status = age >= 18 ? "មនុស្សពេញវ័យ" : "អនីតិជន";
console.log(status); // មនុស្សពេញវ័យ

let score = 85;
let grade = score >= 90 ? "A" : score >= 80 ? "B" : "C";
console.log(grade); // B
```

ការពន្យល់៖ ទម្រង់ជា `condition ? valueIfTrue : valueIfFalse`។ ប្រសិនបើលក្ខខណ្ឌពិត ប្រគល់ `valueIfTrue` បើមិនដូច្នេះទេ ប្រគល់ `valueIfFalse`។ អាចប្រើជាលក្ខខណ្ឌច្រើនដោយបន្តប្រើ `?:`។

លទ្ធផល៖

```
មនុស្សពេញវ័យ
B
```

Bitwise Operators - ប្រតិបត្តិករប៊ីត

ប្រតិបត្តិករប៊ីត (`bitwise operators`) ធ្វើប្រតិបត្តិការលើតម្លៃជាលេខប្រព័ន្ធគោលពីរ (binary)។ ប្រើសម្រាប់ការងារទាបកម្រិត ដូចជាការគ្រប់គ្រងប៊ីត។

ឧទាហរណ៍៖

```
let a = 5;  // 0101 ក្នុង binary
let b = 3;  // 0011 ក្នុង binary

console.log(a & b);  // 1 (AND: 0001)
console.log(a | b);  // 7 (OR: 0111)
console.log(a ^ b);  // 6 (XOR: 0110)
console.log(~a);     // -6 (NOT)
console.log(a << 1); // 10 (Left shift: 1010)
console.log(a >> 1); // 2 (Right shift: 0010)
console.log(a >>> 1); // 2 (Unsigned right shift)
```

ការពន្យល់៖ & (AND), | (OR), ^ (XOR), ~ (NOT), << (Left shift), >> (Right shift), >>> (Unsigned right shift) ធ្វើការលើប៊ីតនៃលេខ។

លទ្ធផល៖

```
1
7
```

```
6
-6
10
2
2
```

String Operators - ប្រតិបត្តិករអក្សរ

ប្រតិបត្តិករអក្សរ (string operators) ត្រូវបានប្រើសម្រាប់ភ្ជាប់អក្សរ ដែលជាទូទៅប្រើ + និង +=។

ឧទាហរណ៍៖

```
let greeting = "សួស្តី";
let name = "សុខា";

console.log(greeting + " " + name); // សួស្តី សុខា

let message = "ជម្រាបសួរ";
message += " ពិភពលោក";
console.log(message); // ជម្រាបសួរ ពិភពលោក
```

ការពន្យល់៖ + ភ្ជាប់អក្សរពីរឬច្រើន, += បន្ថែមអក្សរទៅអថេរដែលមានស្រាប់។

លទ្ធផល៖

```
សួស្តី សុខា
ជម្រាបសួរ ពិភពលោក
```

Type Operators - ប្រតិបត្តិករប្រភេទទិន្នន័យ

ប្រតិបត្តិករប្រភេទទិន្នន័យ (type operators) ត្រូវបានប្រើដើម្បីពិនិត្យប្រភេទទិន្នន័យ ឬ ពិនិត្យថាតើវត្ថុជាកម្មសិទ្ធិរបស់ថ្នាក់ណាមួយ។

ឧទាហរណ៍៖

```
let num = 42;
let str = "Hello";
let obj = {};

console.log(typeof num);      // number
console.log(typeof str);      // string
```

```
console.log(obj instanceof Object); // true
console.log(str instanceof String); // false (str ជា primitive)
```

ការពន្យល់៖ typeof ប្រគល់ប្រភេទទិន្នន័យ, instanceof ពិនិត្យថាតើវត្ថុជាកម្មសិទ្ធិរបស់ថ្នាក់ឬអត់។

លទ្ធផល៖

```
number
string
true
false
```

Special Operators - ប្រតិបត្តិករពិសេស

ប្រតិបត្តិករពិសេស (special operators) រួមមានប្រតិបត្តិករដែលមានមុខងារពិសេស ដូចជា ,, delete, in, new, និង void។

ឧទាហរណ៍៖

```
// Comma Operator
let a = (1, 2, 3);
console.log(a); // 3

// Delete Operator
let obj = { name: "Sokha" };
delete obj.name;
console.log(obj); // {}

// In Operator
let person = { age: 25 };
console.log("age" in person); // true

// New Operator
let date = new Date();
console.log(date); // Current date and time

// Void Operator
let result = void 0;
console.log(result); // undefined
```

ការពន្យល់៖ , ប្រគល់តម្លៃចុងក្រោយ, delete លុបលក្ខណៈសម្បត្តិ, in ពិនិត្យវត្ថុមានលក្ខណៈសម្បត្តិ, new បង្កើតវត្ថុថ្មី, void ប្រគល់ undefined។

លទ្ធផល៖

```
3
{}
true
[Current date and time]
undefined
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើប្រតិបត្តិករទាំងអស់នៅក្នុង JavaScript៖

- `Arithmetic Operators`: +, -, *, /, %, **, ++, -- សម្រាប់ការគណនា។
- `Comparison Operators`: ==, ===, !=, !==, >, <, >=, <= សម្រាប់ប្រៀបធៀប។
- `Logical Operators`: &&, ||, ! សម្រាប់លក្ខខណ្ឌតក្កវិជ្ជា។
- `Assignment Operators`: =, +=, -=, *=, /=, %=, **= សម្រាប់កំណត់តម្លៃ។
- `Ternary Operator`: `?:` សម្រាប់លក្ខខណ្ឌសង្ខេប។
- `Bitwise Operators`: &, |, ^, ~, <<, >>, >>> សម្រាប់ប៊ីត។
- `String Operators`: +, += សម្រាប់ភ្ជាប់អក្សរ។
- `Type Operators`: `typeof`, `instanceof` សម្រាប់ពិនិត្យប្រភេទទិន្នន័យ។
- `Special Operators`: `,`, `delete`, `in`, `new`, `void` សម្រាប់មុខងារពិសេស។

ការយល់ដឹងអំពីប្រតិបត្តិករទាំងនេះគឺសំខាន់សម្រាប់ការសរសេរកម្មវិធី JavaScript ដែលមានប្រសិទ្ធភាព និង ត្រឹមត្រូវ។

1.5 Comments (Single-line, Multi-line) - កំណត់សម្គាល់

កំណត់សម្គាល់ (`comments`) ត្រូវបានប្រើដើម្បីបន្ថែមការពន្យល់ ឬ កំណត់ចំណាំនៅក្នុងកូដ ដែលមិនត្រូវបានប្រតិបត្តិដោយ JavaScript។ មានពីរប្រភេទ៖ `single-line` (មួយជួរ) និង `multi-line` (ច្រើនជួរ)។

ឧទាហរណ៍៖

```
// នេះជាកំណត់សម្គាល់មួយជួរ
let x = 10; // កំណត់អថេរ x

/*
   នេះជាកំណត់សម្គាល់ច្រើនជួរ
   ប្រើសម្រាប់ពន្យល់កូដលម្អិត
*/
let y = 20;
```

```
console.log(x + y); // បោះពុម្ពលទ្ធផលនៃ x + y
```

ការពន្យល់៖ // ប្រើសម្រាប់កំណត់សម្គាល់មួយជួរ។ /* */ ប្រើសម្រាប់កំណត់សម្គាល់ច្រើនជួរ។ កំណត់សម្គាល់ជួយធ្វើឱ្យកូដងាយយល់សម្រាប់អ្នកអភិវឌ្ឍន៍ផ្សេងទៀត។

លទ្ធផល៖

```
30
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើមូលដ្ឋានគ្រឹះនៃ JavaScript ដែលរួមមាន៖

- `Variables and Constants`: ប្រើ `let`, `const`, និង `var` ដើម្បីរក្សាទុកទិន្នន័យ។ `const` សម្រាប់តម្លៃថេរ និង `let` សម្រាប់តម្លៃអថេរ។
- `Data Types`: យល់ពីប្រភេទទិន្នន័យដូចជា String, Number, Boolean, Object, Array, Null, Undefined, និង Symbol។
- `Type Conversion`: ការបម្លែងទិន្នន័យដោយ `implicit` (ស្វ័យប្រវត្តិ) និង `explicit` (ចេតនា)។
- `Operators`: ប្រើ arithmetic, comparison, logical, assignment, និង ternary សម្រាប់ប្រតិបត្តិការលើទិន្នន័យ។
- `Comments`: បន្ថែមកំណត់សម្គាល់ដើម្បីពន្យល់កូដ ដោយប្រើ `//` និង `/* */`។

ការយល់ដឹងអំពីគោលគំនិតទាំងនេះគឺជាមូលដ្ឋានគ្រឹះសម្រាប់ការសរសេរកម្មវិធី JavaScript ដែលមានប្រសិទ្ធភាព។

2. Control Flow - លំហូរគ្រប់គ្រង

លំហូរគ្រប់គ្រង (`control flow`) នៅក្នុង JavaScript អនុញ្ញាតឱ្យអ្នកគ្រប់គ្រងលំដាប់នៃការប្រតិបត្តិកូដដោយប្រើលក្ខខណ្ឌ (`conditional statements`) និងរង្វិលជុំ (`loops`)។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីរបៀបប្រើ `if`, `else`, `else if`, `switch`, `for`, `while`, `do...while`, `for...in`, `for...of`, និងការគ្រប់គ្រងរង្វិលជុំជាមួយ `break` និង `continue` ដើម្បីធ្វើឱ្យកម្មវិធីមានភាពបត់បែន និង មានប្រសិទ្ធភាព។

2.1 Conditional Statements (if, if else, if else if else) - សេចក្តីបញ្ជាលក្ខខណ្ឌ

សេចក្តីបញ្ជាលក្ខខណ្ឌ (conditional statements) នៅក្នុង JavaScript ត្រូវបានប្រើដើម្បីសម្រេចចិត្តផ្អែកលើលក្ខខណ្ឌ។ ប្រសិនបើលក្ខខណ្ឌមួយពិត (true) កូដមួយផ្នែកនឹងត្រូវប្រតិបត្តិ ហើយបើមិនពិត (false) កូដផ្នែកផ្សេងទៀតអាចត្រូវបានប្រតិបត្តិ។ ផ្នែកនេះនឹងផ្តោតលើ if, if else, និង if else if else ដើម្បីជួយអ្នកយល់ពីការគ្រប់គ្រងលក្ខខណ្ឌនៅក្នុងកម្មវិធី។ រាល់ឧទាហរណ៍នឹងបង្ហាញជាមួយកូដ ការពន្យល់លម្អិត និងលទ្ធផល។

2.1.1 Simple if Statement - សេចក្តីបញ្ជា if សាមញ្ញ

if សាមញ្ញត្រូវបានប្រើនៅពេលអ្នកចង់ប្រតិបត្តិកូដតែនៅពេលលក្ខខណ្ឌមួយពិត (true)។ ប្រសិនបើលក្ខខណ្ឌមិនពិត កូដនៅក្នុង if នឹងត្រូវរំលង។

ឧទាហរណ៍៖

```
let temperature = 30;

if (temperature > 25) {
    console.log("អាកាសធាតុក្តៅណាស់");
}
```

ការពន្យល់៖ if (temperature > 25) ពិនិត្យថាតើសីតុណ្ហភាពធំជាង 25។ បើលក្ខខណ្ឌពិត (true) កូដនៅក្នុងប្លុក នឹងប្រតិបត្តិ ហើយបោះពុម្ព "អាកាសធាតុក្តៅណាស់"។ បើលក្ខខណ្ឌមិនពិត គ្មានអ្វីត្រូវបានបោះពុម្ពទេ។ ក្នុងឧទាហរណ៍នេះ temperature = 30 ដែលធំជាង 25 ដូច្នេះលក្ខខណ្ឌពិត។

លទ្ធផល៖

```
អាកាសធាតុក្តៅណាស់
```

2.1.2 if else Statement - សេចក្តីបញ្ជា if else

if else បន្ថែមផ្នែក else ដើម្បីប្រតិបត្តិកូដនៅពេលលក្ខខណ្ឌរបស់ if មិនពិត (false)។ វាផ្តល់ជម្រើសពីរផ្លូវ៖ មួយសម្រាប់លក្ខខណ្ឌពិត និងមួយទៀតសម្រាប់លក្ខខណ្ឌមិនពិត។

ឧទាហរណ៍៖

```
let age = 16;

if (age >= 18) {
    console.log("អ្នកជាមនុស្សពេញវ័យ");
} else {
    console.log("អ្នកនៅតែជាអនីតិជន");
}
```

ការពន្យល់៖ `if (age >= 18)` ពិនិត្យថាតើអាយុធំជាងឬស្មើ 18។ បើលក្ខខណ្ឌពិត បោះពុម្ព "អ្នកជាមនុស្សពេញវ័យ"។ បើមិនពិត កូដក្នុង `else` នឹងប្រតិបត្តិ ហើយបោះពុម្ព "អ្នកនៅតែជាអនីតិជន"។ ក្នុងឧទាហរណ៍នេះ `age = 16` ដែលតូចជាង 18 ដូច្នេះ `else` ត្រូវបានប្រតិបត្តិ។

លទ្ធផល៖

```
អ្នកនៅតែជាអនីតិជន
```

2.1.3 if else if else Statement - សេចក្តីបញ្ជា if else if else

`if else if else` អនុញ្ញាតឱ្យពិនិត្យលក្ខខណ្ឌច្រើនជាងមួយ។ វាប្រើ `else if` ដើម្បីបន្ថែមលក្ខខណ្ឌបន្ទាប់ និង `else` សម្រាប់ករណីដែលគ្មានលក្ខខណ្ឌណាពិត។ នេះជួយគ្រប់គ្រងស្ថានភាពស្មុគស្មាញដែលមានជម្រើសច្រើន។

ឧទាហរណ៍៖

```
let score = 75;

if (score >= 90) {
    console.log("អ្នកបាននិទ្ទេស A");
} else if (score >= 80) {
    console.log("អ្នកបាននិទ្ទេស B");
} else if (score >= 70) {
    console.log("អ្នកបាននិទ្ទេស C");
} else if (score >= 60) {
    console.log("អ្នកបាននិទ្ទេស D");
} else {
    console.log("អ្នកបាននិទ្ទេស F");
}
```

ការពន្យល់៖ `if (score >= 90)` ពិនិត្យថាតើពិន្ទុធំជាងឬស្មើ 90។ បើមិនពិត ពិនិត្យ `else if (score >= 80)` ហើយបន្តទៅ `else if` ផ្សេងទៀតតាមលំដាប់។ បើគ្មានលក្ខខណ្ឌណាពិត `else` នឹងប្រតិបត្តិ។ ក្នុងឧទាហរណ៍នេះ `score = 75` ដែលស្ថិតនៅចន្លោះ 70 និង 79 ដូច្នេះ `else if (score >= 70)` ពិត ហើយបោះពុម្ព "អ្នកបាននិទ្ទេស C"។

លទ្ធផល៖

```
អ្នកបាននិទ្ទេស C
```

2.1.4 Switch Statement - សេចក្តីបញ្ជា Switch

សេចក្តីបញ្ជា `switch` អនុញ្ញាតឱ្យជ្រើសរើសប្លុកកូដមួយក្នុងចំណោមប្លុកជាច្រើន ដោយផ្អែកលើតម្លៃនៃកន្សោម។ វាប្រៀបធៀបតម្លៃមួយជាមួយ `case` ផ្សេងៗ ហើយប្រតិបត្តិកូដនៅក្នុង `case` ដែលផ្គូផ្គង។ `break` ត្រូវបានប្រើ

ដើម្បីបញ្ឈប់ការប្រតិបត្តិ និងចាកចេញពី switch។ បើគ្មាន case ណាផ្គូផ្គងទេ default នឹងត្រូវប្រតិបត្តិ (បើមាន)។ នៅទីនេះ យើងប្រើ switch ដើម្បីកំណត់និទ្ទេសសិស្ស ស្រដៀងនឹង if else if else។

ឧទាហរណ៍៖

```
let score = 75;

switch (true) {
    case score >= 90:
        console.log("អ្នកបាននិទ្ទេស A");
        break;
    case score >= 80:
        console.log("អ្នកបាននិទ្ទេស B");
        break;
    case score >= 70:
        console.log("អ្នកបាននិទ្ទេស C");
        break;
    case score >= 60:
        console.log("អ្នកបាននិទ្ទេស D");
        break;
    default:
        console.log("អ្នកបាននិទ្ទេស F");
}
```

ការពន្យល់ / Explanation: `switch (true)` ពិនិត្យលក្ខខណ្ឌនៅក្នុង case នីមួយៗ។ `score = 75` ត្រូវបានប្រៀបធៀបជាមួយនឹង case នីមួយៗតាមលំដាប់។ `case score >= 70` ពិត ដូច្នេះបោះពុម្ព "អ្នកបាននិទ្ទេស C" ហើយ break បញ្ឈប់ការប្រតិបត្តិ។ បើគ្មាន case ណាពិត (ឧ. `score < 60`) default នឹងបោះពុម្ព "អ្នកបាននិទ្ទេស F"។ វិធីនេះបង្ហាញពីការប្រើ switch សម្រាប់លក្ខខណ្ឌជួរ ស្រដៀងនឹង `if else if else`។

លទ្ធផល / Output:

```
អ្នកបាននិទ្ទេស C
```

2.2 Loops (for, while, do...while, for...in, for...of) - រង្វិលជុំ

រង្វិលជុំ (loops) ត្រូវបានប្រើដើម្បីប្រតិបត្តិកូដម្តងហើយម្តងទៀតរហូតដល់លក្ខខណ្ឌមួយត្រូវបានបំពេញ។ JavaScript មាន `for`, `while`, `do...while`, `for...in`, និង `for...of` សម្រាប់គ្រប់គ្រងការងារដដែលៗ។

2.2.1 for Loop - រង្វិលជុំ for

for ប្រើសម្រាប់រង្វិលជុំដែលមានចំនួនជាក់លាក់ ដោយប្រើអថេររាប់។

ឧទាហរណ៍៖

```
for (let i = 0; i < 5; i++) {
    console.log(`លេខ: ${i}`);
}
```

ការពន្យល់៖ let i = 0 កំណត់អថេរដំបូង, i < 5 ជាលក្ខខណ្ឌប៊ូលីន, i++ បន្ថែម 1 ក្នុងនីមួយវដ្ត។ កូដក្នុងរង្វិលជុំប្រតិបត្តិ 5 ដង។

លទ្ធផល៖

```
លេខ: 0
លេខ: 1
លេខ: 2
លេខ: 3
លេខ: 4
```

2.2.2 while Loop - រង្វិលជុំ while

while ប្រតិបត្តិកូដដរាបណាលក្ខខណ្ឌនៅតែពិត។

ឧទាហរណ៍៖

```
let count = 0;
while (count < 5) {
    console.log(`ចំនួន: ${count}`);
    count++;
}
```

ការពន្យល់៖ while (count < 5) ពិនិត្យលក្ខខណ្ឌមុននឹងប្រតិបត្តិ។ count++ បន្ថែម 1 ក្នុងនីមួយវដ្ដ ដើម្បីជៀសវាងរង្វិលជុំគ្មានទីបញ្ចប់។

លទ្ធផល៖

```
ចំនួន: 0 tripping
ចំនួន: 1
ចំនួន: 2
ចំនួន: 3
ចំនួន: 4
```

2.2.3 do...while Loop - រង្វិលជុំ do...while

`do...while` ប្រតិបត្តិកូដយ៉ាងហោចម្តង មុននឹងពិនិត្យលក្ខខណ្ឌ។

ឧទាហរណ៍៖

```
let num = 0;
do {
    console.log(`តម្លៃ: ${num}`);
    num++;
} while (num < 3);
```

ការពន្យល់៖ កូដក្នុង `do` ប្រតិបត្តិម្តង មុននឹងពិនិត្យ `while (num < 3)`។ បន្ទាប់មក បន្តរង្វិលជុំបើលក្ខខណ្ឌពិត។

លទ្ធផល៖

```
តម្លៃ: 0
តម្លៃ: 1
តម្លៃ: 2
```

2.2.4 for...in Loop - រង្វិលជុំ for...in

`for...in` ប្រើសម្រាប់រង្វិលជុំលើលក្ខណៈសម្បត្តិរបស់វត្ថុ។

ឧទាហរណ៍៖

```
let person = { name: "សុខា", age: 25, city: "ភ្នំពេញ" };

for (let key in person) {
    console.log(`${key}: ${person[key]}`);
}
```

ការពន្យល់៖ `for...in` ឆ្លងកាត់ឈ្មោះលក្ខណៈសម្បត្តិ (key) នៃវត្ថុ។ យើងប្រើ `person[key]` ដើម្បីចូលប្រើតម្លៃ។

លទ្ធផល៖

```
name: សុខា
age: 25
city: ភ្នំពេញ
```

2.2.5 for...of Loop - រង្វិលជុំ for...of

`for...of` ប្រើសម្រាប់រង្វិលជុំលើតម្លៃនៃវត្ថុដែលអាចរាប់បាន ដូចជា Array ឬ String។

ឧទាហរណ៍៖

```
let fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];

for (let fruit of fruits) {
    console.log(`ផ្លែឈើ: ${fruit}`);
}
```

ការពន្យល់៖ for...of ឆ្លងកាត់តម្លៃនីមួយៗនៃ fruits ដោយផ្ទាល់ ដោយមិនចាំបាច់ប្រើលេខសន្ទស្សន៍។

លទ្ធផល៖

```
ផ្លែឈើ: ផ្លែប៉ោម
ផ្លែឈើ: ចេក
ផ្លែឈើ: ស្វាយ
```

2.3 Break and Continue (Flow Control in Loops) - ការគ្រប់គ្រងរង្វិលជុំ (break, continue)

break និង continue ត្រូវបានប្រើដើម្បីគ្រប់គ្រងតរិយាបថនៃរង្វិលជុំ។ break បញ្ចប់រង្វិលជុំទាំងមូល ចំណែក continue រំលងទៅវគ្គបន្ទាប់។

ឧទាហរណ៍៖

```
for (let i = 0; i < 10; i++) {
    if (i === 5) {
        break; // បញ្ចប់រង្វិលជុំនៅ i = 5
    }
    if (i === 3) {
        continue; // រំលង i = 3
    }
    console.log(`លេខ: ${i}`);
}
```

ការពន្យល់៖ break បញ្ចប់រង្វិលជុំនៅពេល i === 5 ដូច្នេះមិនបន្តទៅ i = 6 ឬបន្ទាប់។ continue រំលងការបោះពុម្ពនៅ i === 3 ប៉ុន្តែបន្តរង្វិលជុំ។

លទ្ធផល៖

```
លេខ: 0
លេខ: 1
```

```
លេខ: 2
លេខ: 4
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃលំហូរគ្រប់គ្រងនៅក្នុង JavaScript៖

- `Conditional Statements`
  - `if`: ប្រើសម្រាប់ប្រតិបត្តិកូដនៅពេលលក្ខខណ្ឌមួយពិត (`true`)។ វាសាមញ្ញ និងអនុញ្ញាតឱ្យអ្នកជ្រើសរើសប្រតិបត្តិកូដតែនៅពេលចាំបាច់ ដោយរំលងបើលក្ខខណ្ឌមិនពិត។ ឧទាហរណ៍ ការពិនិត្យសីតុណ្ហភាពថាតើក្តៅឬអត់។
  - `if else`: ពង្រីក `if` ដោយបន្ថែមផ្នែក `else` ដើម្បីប្រតិបត្តិកូដនៅពេលលក្ខខណ្ឌមិនពិត។ វាផ្តល់ជម្រើសពីរផ្លូវ ដែលស័ក្តិសមសម្រាប់ស្ថានភាពដែលត្រូវជ្រើសរើសរវាងលទ្ធផលពីរ ដូចជាការកំណត់ថាតើនរណាម្នាក់ជាមនុស្សពេញវ័យឬអនីតិជន។
  - `if else if else`: អនុញ្ញាតឱ្យពិនិត្យលក្ខខណ្ឌច្រើនជាងមួយជាមួយ `else if` និងបញ្ចប់ដោយ `else` សម្រាប់ករណីដែលគ្មានលក្ខខណ្ឌណាពិត។ វាល្អសម្រាប់ស្ថានភាពស្មុគស្មាញ ដូចជាការកំណត់និទ្ទេសសិស្សផ្អែកលើពិន្ទុ ដែលមានជម្រើសច្រើន។
  - `switch` អនុញ្ញាតឱ្យជ្រើសរើសប្លុកកូដមួយក្នុងចំណោមប្លុកជាច្រើន ដោយផ្អែកលើតម្លៃនៃកន្សោម។ វាប្រៀបធៀបតម្លៃមួយជាមួយ `case` ផ្សេងៗ ហើយប្រតិបត្តិកូដនៅក្នុង `case` ដែលផ្គូផ្គង។ `break` ត្រូវបានប្រើដើម្បីបញ្ឈប់ការប្រតិបត្តិ និងចាកចេញពី `switch`។ បើគ្មាន `case` ណាផ្គូផ្គងទេ `default` នឹងត្រូវប្រតិបត្តិ (បើមាន)។ នៅទីនេះ យើងប្រើ `switch` ដើម្បីកំណត់និទ្ទេសសិស្ស ស្រដៀងនឹង `if else if else`។
- `Loops`: `for` សម្រាប់រង្វិលជុំជាក់លាក់, `while` និង `do...while` សម្រាប់រង្វិលជុំផ្អែកលើលក្ខខណ្ឌ, `for...in` សម្រាប់លក្ខណៈសម្បត្តិវត្ថុ, និង `for...of` សម្រាប់តម្លៃនៃវត្ថុរាប់បាន។
- `Break and Continue`: `break` បញ្ចប់រង្វិលជុំ និង `continue` រំលងទៅវដ្តបន្ទាប់។

ការយល់ដឹងអំពីលំហូរគ្រប់គ្រងទាំងនេះគឺជាមូលដ្ឋានគ្រឹះសម្រាប់ការសរសេរកម្មវិធី JavaScript ដែលមានភាពបត់បែន និង សមត្ថភាពខ្ពស់។

3. Functions - អនុគមន៍

អនុគមន៍ (`functions`) គឺជាប្លុកកូដដែលត្រូវបានរចនាឡើងដើម្បីប្រតិបត្តិការងារជាក់លាក់មួយ និងអាចប្រើឡើងវិញបាននៅក្នុងកម្មវិធី។ អនុគមន៍ជួយធ្វើឱ្យកូដមានរបៀប ងាយស្រួលថែទាំ និងកាត់បន្ថយភាពស្មុគស្មាញ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីរបៀបបង្កើត និងប្រើ `function declarations`, `function expressions`, `arrow functions`, `parameters`, `return values`, `closures`, `callback functions`, និង `IIFE` នៅក្នុង JavaScript។

3.1 Function Declarations and Expressions - ការប្រកាស និង កន្សោមអនុគមន៍

អនុគមន៍នៅក្នុង JavaScript អាចត្រូវបានបង្កើតដោយប្រើ function declarations ឬ function expressions។ Function declarations ត្រូវបានកំណត់ជាមួយឈ្មោះ និងអាចហៅបានមុនការប្រកាស (hoisting)។ Function expressions ត្រូវបានកំណត់ជាអថេរ និងមិនអាចហៅបានមុនការកំណត់។

ឧទាហរណ៍៖

```
// Function Declaration
function greet(name) {
    console.log("សួស្តី " + name);
}
greet("សុខា"); // អាចហៅបានមុនការប្រកាស

// Function Expression
const add = function(a, b) {
    console.log(a + b);
};
add(5, 3); // ត្រូវហៅបន្ទាប់ពីកំណត់
```

ការពន្យល់៖ function greet ជា function declaration ដែលអាចហៅបានមុនការប្រកាស ដោយសារតែ hoisting។ const add ជា function expression ដែលត្រូវកំណត់ជាអថេរមុននឹងហៅ។ ទាំងពីរអាចប្រើបាន ប៉ុន្តែ function expressions ផ្តល់ភាពបត់បែនក្នុងការគ្រប់គ្រងអថេរ។

លទ្ធផល៖

```
សួស្តី សុខា
8
```

3.2 Arrow Functions (ES6) - អនុគមន៍សញ្ញាព្រួញ

Arrow functions គឺជាវិធីសរសេរអនុគមន៍ខ្លីជាង និងទំនើបជាងនៅក្នុង ES6។ ពួកវាប្រើសញ្ញា => និងមានឥរិយាបថខុសគ្នាជាមួយ this (lexical binding)។

ឧទាហរណ៍៖

```
// Arrow Function
const multiply = (a, b) => a * b;
console.log(multiply(4, 5));

// Arrow Function ជាមួយប្លុកកូដ
```

```
const sayHello = name => {
    console.log(`ជម្រាបសួរ ${name}`);
};
sayHello("វណ្ណៈ");
```

ការពន្យល់៖ multiply ជា arrow function ដែលសរសេរខ្លីដោយប្រើ => និងប្រគល់តម្លៃដោយផ្ទាល់។ sayHello បង្ហាញការប្រើប្លុកកូដជាមួយ ។ Arrow functions មិនមាន this ផ្ទាល់ខ្លួន ហើយស័ក្តិសមសម្រាប់ការងារសាមញ្ញ។

លទ្ធផល៖

```
20
ជម្រាបសួរ វណ្ណៈ
```

3.3 Function Parameters (Default, Rest Parameters) - ប៉ារ៉ាម៉ែត្រអនុគមន៍

ប៉ារ៉ាម៉ែត្រអនុគមន៍ (parameters) អនុញ្ញាតឱ្យអ្នកបញ្ជូនទិន្នន័យទៅក្នុងអនុគមន៍។ JavaScript ផ្តល់ default parameters សម្រាប់កំណត់តម្លៃលំនាំដើម និង rest parameters សម្រាប់ទទួលប៉ារ៉ាម៉ែត្រច្រើន។

ឧទាហរណ៍៖

```
// Default Parameters
function welcome(name = "ភ្ញៀវ") {
    console.log(`ស្វាគមន៍ ${name}`);
}
welcome(); // ប្រើតម្លៃលំនាំដើម
welcome("សុខា");

// Rest Parameters
function sum(...numbers) {
    let total = 0;
    for (let num of numbers) {
        total += num;
    }
    console.log(total);
}
sum(1, 2, 3, 4);
```

ការពន្យល់៖ default parameters ដូចជា name = "ភ្ញៀវ" ផ្តល់តម្លៃលំនាំដើមបើមិនបញ្ជូនប៉ារ៉ាម៉ែត្រ។ rest parameters ដូចជា ...numbers ប្រមូលប៉ារ៉ាម៉ែត្រទាំងអស់ទៅជា array ដើម្បីគ្រប់គ្រងចំនួនប៉ារ៉ាម៉ែត្រមិនកំណត់។

លទ្ធផល៖

```
ស្វាគមន៍ ភ្ញៀវ
ស្វាគមន៍ សុខា
10
```

3.4 Return Values - តម្លៃប្រគល់

return ត្រូវបានប្រើដើម្បីប្រគល់តម្លៃពីអនុគមន៍ទៅកន្លែងដែលវាត្រូវបានហៅ។ បន្ទាប់ពី return កូដផ្សេងទៀតនៅក្នុងអនុគមន៍នឹងមិនត្រូវបានប្រតិបត្តិទេ។

ឧទាហរណ៍៖

```
function square(num) {
    return num * num;
    console.log("នឹងមិនប្រតិបត្តិទេ"); // កូដបន្ទាប់ពី return
}
const result = square(4);
console.log(result);
```

ការពន្យល់៖ return num * num ប្រគល់តម្លៃ 16 ពីអនុគមន៍ square។ កូដបន្ទាប់ពី return ត្រូវបានរំលង។ តម្លៃប្រគល់ត្រូវបានរក្សាទុកនៅក្នុង result សម្រាប់ប្រើបន្ត។

លទ្ធផល៖

```
16
```

3.5 Closures (Functions within Functions) - ការបិទអនុគមន៍

Closures គឺជាអនុគមន៍ដែលស្ថិតនៅក្នុងអនុគមន៍មួយទៀត និងអាចចូលប្រើអថេរនៅក្នុង scope ខាងក្រៅរបស់វា ទោះបីជាអនុគមន៍ខាងក្រៅបានបញ្ចប់ការប្រតិបត្តិក៏ដោយ។

ឧទាហរណ៍៖

```
function counter() {
    let count = 0;
    return function() {
        count++;
```

```
        console.log(count);
    };
}
const increment = counter();
increment();
increment();
```

ការពន្យល់៖ counter ប្រគល់អនុគមន៍ខាងក្នុងដែលចូលប្រើអថេរ count។ ទោះបី counter បានបញ្ចប់ អនុគមន៍ខាងក្នុងនៅតែចងចាំ count ដែលបង្កើនតម្លៃនៅពេលហៅ increment។

លទ្ធផល៖

```
1
2
```

3.6 Callback Functions - អនុគមន៍ហៅត្រឡប់

Callback functions គឺជាអនុគមន៍ដែលត្រូវបានបញ្ជូនជាប៉ារ៉ាម៉ែត្រទៅអនុគមន៍ផ្សេង និងត្រូវបានហៅនៅពេលក្រោយ។ វាត្រូវបានប្រើជាទូទៅសម្រាប់ការងារអសមកាល។

ឧទាហរណ៍៖

```
function processData(data, callback) {
    console.log("កំពុងដំណើរការទិន្នន័យ: " + data);
    callback(data);
}
function display(data) {
    console.log("បង្ហាញទិន្នន័យ: " + data);
}
processData("ព័ត៌មាន", display);
```

ការពន្យល់៖ display ជា callback function ដែលត្រូវបញ្ជូនទៅ processData។ processData ដំណើរការទិន្នន័យ ហើយហៅ callback ដើម្បីបន្តការងារ។

លទ្ធផល៖

```
កំពុងដំណើរការទិន្នន័យ: ព័ត៌មាន
បង្ហាញទិន្នន័យ: ព័ត៌មាន
```

3.7 Immediately Invoked Function Expressions (IIFE) - កន្សោមអនុគមន៍ហៅភ្លាមៗ

IIFE (Immediately Invoked Function Expression) គឺជា `function expression` ដែលត្រូវបានប្រតិបត្តិភ្លាមៗបន្ទាប់ពីការកំណត់។ វាត្រូវបានប្រើដើម្បីបង្កើត scope ឯកជន និងការពារការប៉ះទង្គិចអថេរ។

ឧទាហរណ៍៖

```
(function() {
    let message = "សារឯកជន";
    console.log(message);
})();
console.log(typeof message); // មិនអាចចូលប្រើ message បាន
```

ការពន្យល់៖ IIFE ត្រូវបានបង្កើតដោយរុំ `function expression` ក្នុង `()` និងហៅភ្លាមៗជាមួយ `()`។ អថេរ `message` ស្ថិតនៅក្នុង scope ឯកជន ហើយមិនអាចចូលប្រើពីខាងក្រៅបាន។

លទ្ធផល៖

```
សារឯកជន
undefined
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃអនុគមន៍នៅក្នុង JavaScript៖

- `Function Declarations and Expressions`: `function declarations` អនុញ្ញាតឱ្យហៅមុនការប្រកាស ខណៈ `function expressions` ផ្តល់ភាពបត់បែនក្នុងការគ្រប់គ្រងអថេរ។
- `Arrow Functions`: វិធីសរសេរអនុគមន៍ខ្លីជាង និងមានឥរិយាបថ `this` ខុសគ្នា ស័ក្តិសមសម្រាប់ការងារសាមញ្ញ។
- `Function Parameters`: `default parameters` ផ្តល់តម្លៃលំនាំដើម និង `rest parameters` គ្រប់គ្រងប៉ារ៉ាម៉ែត្រច្រើន។
- `Return Values`: `return` ប្រគល់តម្លៃពីអនុគមន៍ និងបញ្ចប់ការប្រតិបត្តិ។
- `Closures`: អនុគមន៍ខាងក្នុងចូលប្រើអថេរខាងក្រៅ ទោះបីអនុគមន៍ខាងក្រៅបញ្ចប់។
- `Callback Functions`: អនុគមន៍បញ្ជូនជាប៉ារ៉ាម៉ែត្រ ប្រើសម្រាប់ការងារអសមកាល។
- `IIFE`: អនុគមន៍ហៅភ្លាមៗបន្ទាប់ពីកំណត់ បង្កើត scope ឯកជន។

ការយល់ដឹងអំពីអនុគមន៍ទាំងនេះជួយអ្នកសរសេរកូដដែលមានរបៀប បត់បែន និងអាចប្រើឡើងវិញបាននៅក្នុង JavaScript។

4. Objects - វត្ថុ

វត្ថុ (objects) នៅក្នុង JavaScript គឺជាទិន្នន័យដែលផ្ទុកលក្ខណៈសម្បត្តិ (properties) និង វិធីសាស្ត្រ (methods) ដែលតំណាងឱ្យអង្គភាពមួយ។ វត្ថុជួយរៀបចំទិន្នន័យឱ្យមានរបៀប និង អនុញ្ញាតឱ្យគ្រប់គ្រងទំនាក់ទំនងរវាងទិន្នន័យ និងអាកប្បកិរិយា។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការបង្កើត វត្ថុ ការចូលប្រើលក្ខណៈសម្បត្តិ វិធីសាស្ត្រ ការបំបែកវត្ថុ (destructuring) ការប្រើ object literals និង prototype សម្រាប់មរតក។

4.1 Creating Objects - ការបង្កើតវត្ថុ

វត្ថុនៅក្នុង JavaScript អាចត្រូវបានបង្កើតដោយប្រើ object literals ឬ constructor functions និង new ពាក្យគន្លឹះ។ វិធីទាំងនេះអនុញ្ញាតឱ្យអ្នកកំណត់លក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រសម្រាប់វត្ថុ។

ឧទាហរណ៍៖

```
// Object Literal
const person = {
    name: "សុខា",
    age: 25
};
console.log(person);

// Constructor Function
function Car(model, year) {
    this.model = model;
    this.year = year;
}
const myCar = new Car("Toyota", 2020);
console.log(myCar);
```

ការពន្យល់៖ object literal ប្រើ ដើម្បីកំណត់វត្ថុដោយផ្ទាល់។ constructor function ប្រើ this និង new ដើម្បីបង្កើតវត្ថុជាមួយលក្ខណៈសម្បត្តិដែលបានកំណត់។ ទាំងពីរវិធីអនុញ្ញាតឱ្យអ្នកបង្កើតវត្ថុដែលផ្ទុក ទិន្នន័យ។

លទ្ធផល៖

```
{ name: 'សុខា', age: 25 }
Car { model: 'Toyota', year: 2020 }
```

4.2 Accessing Object Properties - ការចូលប្រើលក្ខណៈសម្បត្តិវត្ថុ

លក្ខណៈសម្បត្តិវត្ថុអាចត្រូវបានចូលប្រើដោយប្រើ `dot notation` (.) ឬ `bracket notation` ([])។ វិធីទាំងនេះអនុញ្ញាតឱ្យអ្នកទាញ ឬកែប្រែតម្លៃនៅក្នុងវត្ថុ។

ឧទាហរណ៍៖

```
const student = {
    name: "វណ្ណៈ",
    grade: "A"
};

console.log(student.name); // Dot notation
console.log(student["grade"]); // Bracket notation

student.grade = "B"; // កែប្រែលក្ខណៈសម្បត្តិ
console.log(student.grade);
```

ការពន្យល់៖ `dot notation` ប្រើសម្រាប់ឈ្មោះលក្ខណៈសម្បត្តិដែលច្បាស់លាស់។ `bracket notation` អនុញ្ញាតឱ្យប្រើអថេរ ឬឈ្មោះដែលមានអក្សរពិសេស។ ទាំងពីរអាចប្រើដើម្បីទាញ ឬកែប្រែតម្លៃ។

លទ្ធផល៖

```
វណ្ណៈ
A
B
```

4.3 Methods and this Keyword - វិធីសាស្ត្រ និងពាក្យគន្លឹះ this

វិធីសាស្ត្រ (`methods`) គឺជាអនុគមន៍ដែលត្រូវបានកំណត់ជាលក្ខណៈសម្បត្តិនៅក្នុងវត្ថុ។ ពាក្យគន្លឹះ `this` សំដៅទៅលើវត្ថុដែលកំពុងប្រើវិធីសាស្ត្រ ដែលអនុញ្ញាតឱ្យចូលប្រើលក្ខណៈសម្បត្តិរបស់វត្ថុ។

ឧទាហរណ៍៖

```
const person = {
    name: "សុខា",
    age: 25,
    introduce: function() {
        console.log(`ជម្រាបសួរ ខ្ញុំឈ្មោះ ${this.name} អាយុ ${this.age} ឆ្នាំ`);
```

```
    }
};

person.introduce();
```

ការពន្យល់៖ introduce ជាវិធីសាស្ត្រដែលប្រើ this ដើម្បីចូលប្រើ name និង age នៃវត្ថុ person។ this ធានាថាកូដសំដៅទៅលើវត្ថុដែលហៅវិធីសាស្ត្រ។

លទ្ធផល៖

```
ជម្រាបសួរ ខ្ញុំឈ្មោះ សុខា អាយុ 25 ឆ្នាំ
```

4.4 Object Destructuring - ការបំបែកវត្ថុ

Object destructuring គឺជាវិធីសាស្ត្រនៅក្នុង ES6 ដែលអនុញ្ញាតឱ្យអ្នកទាញលក្ខណៈសម្បត្តិចេញពីវត្ថុទៅជាអថេរដោយផ្ទាល់ ធ្វើឱ្យកូដកាន់តែសង្ខេប។

ឧទាហរណ៍៖

```
const user = {
    username: "វណ្ណៈ",
    email: "vanna@example.com"
};

const { username, email } = user;
console.log(username);
console.log(email);

// ប្តូរឈ្មោះអថេរ
const { username: userName } = user;
console.log(userName);
```

ការពន្យល់៖ ទាញតម្លៃនៃ username និង email ទៅជាអថេរដែលមានឈ្មោះដូចគ្នា។ អ្នកអាចប្តូរឈ្មោះអថេរដោយប្រើ username: userName។ Destructuring ធ្វើឱ្យការចូលប្រើលក្ខណៈសម្បត្តិកាន់តែងាយស្រួល។

លទ្ធផល៖

```
វណ្ណៈ
vanna@example.com
វណ្ណៈ
```

4.5 Object Literals - វត្ថុលីតេរ៉ាល់

Object literals គឺជាវិធីសាមញ្ញបំផុតដើម្បីបង្កើតវត្ថុដោយប្រើ ។ ES6 បានធ្វើឱ្យ object literals កាន់តែសង្ខេបជាមួយនឹងការគាំទ្រសម្រាប់ឈ្មោះអថេរដោយផ្ទាល់ និងវិធីសាស្ត្រសង្ខេប។

ឧទាហរណ៍៖

```
const name = "សុខា";
const age = 25;

// Object Literal នៅក្នុង ES6
const person = {
    name, // សង្ខេបពី name: name
    age,
    greet() { // វិធីសាស្ត្រសង្ខេប
        console.log(`សួស្តី ខ្ញុំឈ្មោះ ${this.name}`);
    }
};

console.log(person);
person.greet();
```

ការពន្យល់៖ `name, age` ជាការសង្ខេបនៃ `name: name, age: age`។ `greet()` ជាវិធីសាស្ត្រសង្ខេប ដែលលុប `function` ពាក្យគន្លឹះ។ `Object literals` ធ្វើឱ្យការបង្កើតវត្ថុកាន់តែសាមញ្ញ។

លទ្ធផល៖

```
{ name: 'សុខា', age: 25, greet: [Function: greet] }
សួស្តី ខ្ញុំឈ្មោះ សុខា
```

4.6 Prototype and Inheritance - ប្រូតូថាយ និងមរតក

វត្ថុនៅក្នុង JavaScript មាន `prototype` ដែលអនុញ្ញាតឱ្យចែករំលែកលក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រទៅវត្ថុផ្សេងទៀតតាមរយៈ `inheritance`។ `prototype` ជួយបន្ថែមមុខងារទៅវត្ថុដែលមានស្រាប់។

ឧទាហរណ៍៖

```
function Animal(name) {
    this.name = name;
}

Animal.prototype.sayHello = function() {
```

```
    console.log(`សួស្តី ខ្ញុំឈ្មោះ ${this.name}`);
};

const dog = new Animal("កុក");
dog.sayHello();

function Dog(name, breed) {
    Animal.call(this, name);
    this.breed = breed;
}

Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

const myDog = new Dog("ប៊ុន", "Husky");
myDog.sayHello();
console.log(myDog.breed);
```

ការពន្យល់៖ `Animal.prototype` បន្ថែមវិធីសាស្ត្រ sayHello ទៅគ្រប់វត្ថុ `Animal`។ Dog ទទួលមរតកពី `Animal` ដោយប្រើ `Object.create` ដើម្បីភ្ជាប់ prototype។ `Animal.call` ធានាថា name ត្រូវបានកំណត់។ prototype អនុញ្ញាតឱ្យ myDog ចូលប្រើ sayHello និងមាន breed ផ្ទាល់ខ្លួន។

លទ្ធផល៖

```
សួស្តី ខ្ញុំឈ្មោះ កុក
សួស្តី ខ្ញុំឈ្មោះ ប៊ុន
Husky
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃវត្ថុនៅក្នុង JavaScript៖

- `Creating Objects`: បង្កើតវត្ថុដោយប្រើ `object literals` ឬ `constructor functions` ជាមួយ new។
- `Accessing Object Properties`: ប្រើ `dot notation` ឬ `bracket notation` ដើម្បីទាញ ឬកែប្រែលក្ខណៈសម្បត្តិ។
- `Methods and this`: វិធីសាស្ត្រជាអនុគមន៍ក្នុងវត្ថុ ដោយ this សំដៅទៅលើវត្ថុម្ចាស់។
- `Object Destructuring`: ទាញលក្ខណៈសម្បត្តិទៅជាអថេរដោយផ្ទាល់ ធ្វើឱ្យកូដសង្ខេប។
- `Object Literals`: បង្កើតវត្ថុសាមញ្ញជាមួយការសរសេរសង្ខេបនៅក្នុង ES6។
- `Prototype and Inheritance`: ប្រើ prototype ដើម្បីចែករំលែកលក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រតាមរយៈមរតក។

ការយល់ដឹងអំពីវត្ថុទាំងនេះគឺសំខាន់សម្រាប់ការសរសេរកម្មវិធី JavaScript ដែលមានរចនាសម្ព័ន្ធ និងអាចពង្រីកបាន។

5. Arrays - អារេ

អារេ (arrays) នៅក្នុង JavaScript គឺជាទិន្នន័យប្រភេទបញ្ជីដែលផ្ទុកធាតុជាច្រើន ដែលអាចជាតម្លៃណាមួយ (លេខ អក្សរ វត្ថុ។ល។)។ អារេអនុញ្ញាតឱ្យអ្នកគ្រប់គ្រងទិន្នន័យជាក្រុម និងមានវិធីសាស្ត្រជាច្រើនសម្រាប់រៀបចំ ឬស្វែងរកទិន្នន័យ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការបង្កើត ការចូលប្រើ និងការគ្រប់គ្រងអារេដោយប្រើវិធីសាស្ត្រផ្សេងៗ រួមទាំង `destructuring`, `spread operator`, និងវិធីសាស្ត្រសម្រាប់ស្វែងរក តម្រៀប និងបំប្លែងអារេ។

5.1 Creating and Accessing Arrays - ការបង្កើត និង ការចូលប្រើអារេ

អារេអាចត្រូវបានបង្កើតដោយប្រើ `[]` ឬ `new Array()`។ អ្នកអាចចូលប្រើធាតុអារេដោយប្រើលេខសន្ទស្សន៍ (index) ដែលចាប់ផ្តើមពី 0។

ឧទាហរណ៍៖

```
// បង្កើតអារេ
const fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];
const numbers = new Array(1, 2, 3);

// ចូលប្រើធាតុ
console.log(fruits[0]); // ធាតុទីមួយ
console.log(numbers[2]); // ធាតុទីបី

// កែប្រែធាតុ
fruits[1] = "ទំពាំង";
console.log(fruits);
```

ការពន្យល់៖ `[]` ជាវិធីសាមញ្ញបំផុតដើម្បីបង្កើតអារេ។ `fruits[0]` ទាញធាតុនៅលេខសន្ទស្សន៍ 0។ អ្នកអាចកែប្រែធាតុដោយកំណត់តម្លៃថ្មីនៅលេខសន្ទស្សន៍ជាក់លាក់។

លទ្ធផល៖

```
ផ្លែប៉ោម
3
[ 'ផ្លែប៉ោម', 'ទំពាំង', 'ស្វាយ' ]
```

5.2 Array Methods - វិធីសាស្ត្រអារេ

JavaScript ផ្តល់វិធីសាស្ត្រជាច្រើនសម្រាប់គ្រប់គ្រងអារេ ដូចជាការបន្ថែម លុប កែប្រែ ឬធ្វើប្រតិបត្តិការលើធាតុ។ ផ្នែកនេះនឹងគ្របដណ្តប់លើវិធីសាស្ត្រសំខាន់ៗ។

5.2.1 push() and pop() - បន្ថែម និង លុបធាតុចុងក្រោយ

`push()` បន្ថែមធាតុទៅចុងអារេ ខណៈ `pop()` លុបធាតុចុងក្រោយ និងប្រគល់តម្លៃនោះ។

ឧទាហរណ៍៖

```
const fruits = ["ផ្លែប៉ោម", "ចេក"];
fruits.push("ស្វាយ");
console.log(fruits);

const lastFruit = fruits.pop();
console.log(lastFruit);
console.log(fruits);
```

ការពន្យល់៖ `push("ស្វាយ")` បន្ថែម "ស្វាយ" ទៅចុងអារេ។ `pop()` លុប "ស្វាយ" និងប្រគល់តម្លៃនោះ។ `fruits` ត្រូវបានកែប្រែផ្ទាល់។

លទ្ធផល៖

```
[ 'ផ្លែប៉ោម', 'ចេក', 'ស្វាយ' ]
ស្វាយ
[ 'ផ្លែប៉ោម', 'ចេក' ]
```

5.2.2 shift() and unshift() - លុប និង បន្ថែមធាតុដើម

`shift()` លុបធាតុដំបូងនៃអារេ និងប្រគល់តម្លៃនោះ។ `unshift()` បន្ថែមធាតុទៅដើមអារេ។

ឧទាហរណ៍៖

```
const numbers = [10, 20, 30];
const firstNumber = numbers.shift();
console.log(firstNumber);
console.log(numbers);
```

```
numbers.unshift(5);
console.log(numbers);
```

ការពន្យល់៖ shift() លុប 10 និងប្រគល់តម្លៃនោះ។ unshift(5) បន្ថែម 5 ទៅដើមអារេ។ វិធីសាស្ត្រទាំងនេះកែប្រែអារេផ្ទាល់។

លទ្ធផល៖

```
10
[ 20, 30 ]
[ 5, 20, 30 ]
```

5.2.3 slice() and splice() - កាត់ និង កែប្រែអារេ

slice() ប្រគល់អារេថ្មីដែលជាផ្នែកមួយនៃអារេដើម ដោយមិនកែប្រែវា។ splice() លុប ឬបន្ថែមធាតុនៅទីតាំងជាក់លាក់ និងកែប្រែអារេដើម។

ឧទាហរណ៍៖

```
const fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ", "ទំពាំង"];
const sliced = fruits.slice(1, 3);
console.log(sliced);
console.log(fruits);

fruits.splice(1, 1, "ក្រូច");
console.log(fruits);
```

ការពន្យល់៖ slice(1, 3) ប្រគល់អារេថ្មីពីលេខសន្ទស្សន៍ 1 ដល់ 2។ splice(1, 1, "ក្រូច") លុប 1 ធាតុនៅលេខសន្ទស្សន៍ 1 និងបន្ថែម "ក្រូច"។ splice កែប្រែអារេដើម។

លទ្ធផល៖

```
[ 'ចេក', 'ស្វាយ' ]
[ 'ផ្លែប៉ោម', 'ចេក', 'ស្វាយ', 'ទំពាំង' ]
[ 'ផ្លែប៉ោម', 'ក្រូច', 'ស្វាយ', 'ទំពាំង' ]
```

5.2.4 forEach() - ឆ្លងកាត់ធាតុអារេ

forEach() ហៅអនុគមន៍មួយសម្រាប់ធាតុនីមួយៗនៅក្នុងអារេ ដោយមិនបង្កើតអារេថ្មី។

ឧទាហរណ៍៖

```
const numbers = [1, 2, 3];
numbers.forEach(num => console.log(num * 2));
```

ការពន្យល់៖ forEach ឆ្លងកាត់ធាតុនីមួយៗ និងអនុវត្តអនុគមន៍ដែលគុណធាតុនីមួយៗនឹង 2។ វាមិនប្រគល់តម្លៃទេ គ្រាន់តែប្រតិបត្តិសកម្មភាព។

លទ្ធផល៖

```
2
4
6
```

5.2.5 map() - បង្កើតអារេថ្មី

map() បង្កើតអារេថ្មីដោយអនុវត្តអនុគមន៍ទៅលើធាតុនីមួយៗនៃអារេដើម ដោយមិនកែប្រែអារេដើម។

ឧទាហរណ៍៖

```
const numbers = [1, 2, 3];
const doubled = numbers.map(num => num * 2);
console.log(doubled);
console.log(numbers);
```

ការពន្យល់៖ map បង្កើតអារេថ្មីដែលមានធាតុនីមួយៗគុណនឹង 2។ អារេដើម numbers មិនត្រូវបានកែប្រែទេ។

លទ្ធផល៖

```
[ 2, 4, 6 ]
[ 1, 2, 3 ]
```

5.2.6 filter() - ច្រោះធាតុ

filter() បង្កើតអារេថ្មីដែលមានតែធាតុដែលឆ្លើយតបនឹងលក្ខខណ្ឌជាក់លាក់។

ឧទាហរណ៍៖

```
const numbers = [1, 2, 3, 4, 5];
const even = numbers.filter(num => num % 2 === 0);
console.log(even);
console.log(numbers);
```

ការពន្យល់៖ filter ជ្រើសរើសតែលេខគូបានពី numbers។ អារេថ្មី even មានតែធាតុដែលបំពេញលក្ខខណ្ឌ។ អារេដើមនៅដដែល។

លទ្ធផល៖

```
[ 2, 4 ]
[ 1, 2, 3, 4, 5 ]
```

5.2.7 reduce() - បង្រួមអារេ

reduce() បង្រួមអារេទៅជាតម្លៃតែមួយដោយអនុវត្តអនុគមន៍លើធាតុនីមួយៗ។

ឧទាហរណ៍៖

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((total, num) => total + num, 0);
console.log(sum);
```

ការពន្យល់៖ reduce បូកធាតុទាំងអស់នៃ numbers ដោយចាប់ផ្តើមពី 0។ total ជាតម្លៃបច្ចុប្បន្ន និង num ជាធាតុនីមួយៗ។ លទ្ធផលគឺជាតម្លៃតែមួយ។

លទ្ធផល៖

```
10
```

5.3 Array Destructuring - ការបំបែកអារេ

Array destructuring អនុញ្ញាតឱ្យអ្នកទាញធាតុចេញពីអារេទៅជាអថេរដោយផ្ទាល់ ធ្វើឱ្យកូដកាន់តែសង្ខេប។

ឧទាហរណ៍៖

```
const colors = ["ក្រហម", "បៃតង", "ខៀវ"];
const [first, second] = colors;
console.log(first);
console.log(second);

// រំលងធាតុ
const [, , third] = colors;
console.log(third);
```

ការពន្យល់៖ [first, second] ទាញធាតុទីមួយ និងទីពីរទៅជាអថេរ។ [, , third] រំលងធាតុទីមួយ និងទីពីរ ដើម្បីទាញធាតុទីបី។

លទ្ធផល៖

```
ក្រហម
បៃតង
ខៀវ
```

5.4 Spread Operator (`...`) - ប្រតិបត្តិករពន្លា

Spread operator (`...`) ពន្លាធាតុអារេទៅជាធាតុនីមួយៗ ឬចម្លងអារេទៅអារេថ្មី។

ឧទាហរណ៍៖

```
const fruits = ["ផ្លែប៉ោម", "ចេក"];
const moreFruits = [...fruits, "ស្វាយ"];
console.log(moreFruits);

const numbers = [1, 2];
const combined = [...numbers, ...fruits];
console.log(combined);
```

ការពន្យល់៖ `...fruits` ពន្លាធាតុរបស់ `fruits` ទៅជាអារេថ្មី។ spread operator អនុញ្ញាតឱ្យបញ្ចូលអារេច្រើនជាមួយគ្នា ឬបន្ថែមធាតុថ្មី។

លទ្ធផល៖

```
[ 'ផ្លែប៉ោម', 'ចេក', 'ស្វាយ' ]
[ 1, 2, 'ផ្លែប៉ោម', 'ចេក' ]
```

5.5 Rest Parameter in Functions - ប៉ារ៉ាម៉ែត្រសល់នៅក្នុងអនុគមន៍

Rest parameter (`...`) ប្រមូលប៉ារ៉ាម៉ែត្រទាំងអស់នៅក្នុងអនុគមន៍ទៅជាអារេ ដែលអនុញ្ញាតឱ្យគ្រប់គ្រងចំនួនប៉ារ៉ាម៉ែត្រមិនកំណត់។

ឧទាហរណ៍៖

```
function sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
}
console.log(sum(1, 2, 3));
console.log(sum(10, 20, 30, 40));
```

ការពន្យល់៖ ...numbers ប្រមូលប៉ារ៉ាម៉ែត្រទាំងអស់ទៅជាអារេ។ reduce បូកធាតុទាំងអស់។ rest parameter អនុញ្ញាតឱ្យអនុគមន៍ទទួលចំនួនប៉ារ៉ាម៉ែត្រផ្សេងៗគ្នា។

លទ្ធផល៖

```
6
100
```

5.6 Finding and Searching in Arrays - ការស្វែងរកនៅក្នុងអារេ

JavaScript ផ្តល់វិធីសាស្ត្រដូចជា `indexOf()`, `find()`, `findIndex()`, និង `includes()` សម្រាប់ស្វែងរកធាតុនៅក្នុងអារេ។

ឧទាហរណ៍៖

```
const fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];
console.log(fruits.indexOf("ចេក"));
console.log(fruits.includes("ស្វាយ"));

const numbers = [10, 20, 30, 40];
const found = numbers.find(num => num > 25);
console.log(found);

const index = numbers.findIndex(num => num > 25);
console.log(index);
```

ការពន្យល់៖ `indexOf()` ប្រគល់លេខសន្ទស្សន៍នៃធាតុ ឬ -1 បើរកមិនឃើញ។ `includes()` ពិនិត្យថាតើធាតុមាននៅក្នុងអារេ។ `find()` ប្រគល់ធាតុដំបូងដែលបំពេញលក្ខខណ្ឌ។ `findIndex()` ប្រគល់លេខសន្ទស្សន៍នៃធាតុនោះ។

លទ្ធផល៖

```
1
true
30
2
```

5.7 Sorting and Reversing - ការតម្រៀប និង បញ្ច្រាស

`sort()` តម្រៀបធាតុនៅក្នុងអារេ និង `reverse()` បញ្ច្រាសលំដាប់ធាតុ។ ទាំងពីរកែប្រែអារេដើម។

ឧទាហរណ៍៖

```
const numbers = [30, 10, 20];
numbers.sort((a, b) => a - b);
console.log(numbers);

const fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];
fruits.reverse();
console.log(fruits);
```

ការពន្យល់៖ `sort((a, b) => a - b)` តម្រៀបលេខតាមលំដាប់ឡើង។ `reverse()` បញ្ច្រាសលំដាប់ធាតុទាំងមូល។ ទាំងពីរកែប្រែ `numbers` និង `fruits` ផ្ទាល់។

លទ្ធផល៖

```
[ 10, 20, 30 ]
[ 'ស្វាយ', 'ចេក', 'ផ្លែប៉ោម' ]
```

5.8 Checking if a Value is an Array - ការពិនិត្យថាតើជាអារេ

`Array.isArray()` ពិនិត្យថាតើតម្លៃជាអារេឬអត់ ដែលជួយការពារកំហុសនៅពេលធ្វើការជាមួយទិន្នន័យ។

ឧទាហរណ៍៖

```
const arr = [1, 2, 3];
const obj = { a: 1 };
console.log(Array.isArray(arr));
console.log(Array.isArray(obj));
```

ការពន្យល់៖ `Array.isArray(arr)` ប្រគល់ `true` ព្រោះ `arr` ជាអារេ។ `Array.isArray(obj)` ប្រគល់ `false` ព្រោះ `obj` ជាវត្ថុ។

លទ្ធផល៖

```
true
false
```

5.9 Concatenating Arrays - ការបញ្ចូលអារេ

`concat()` បញ្ចូលអារេច្រើនទៅជាអារេថ្មី ដោយមិនកែប្រែអារេដើម។

ឧទាហរណ៍៖

```
const fruits = ["ផ្លែប៉ោម", "ចេក"];
const moreFruits = ["ស្វាយ", "ទំពាំង"];
const allFruits = fruits.concat(moreFruits);
console.log(allFruits);
console.log(fruits);
```

ការពន្យល់៖ concat() បញ្ចូល fruits និង moreFruits ទៅជាអារេថ្មី។ អារេដើមនៅដដែល។

លទ្ធផល៖

```
[ 'ផ្លែប៉ោម', 'ចេក', 'ស្វាយ', 'ទំពាំង' ]
[ 'ផ្លែប៉ោម', 'ចេក' ]
```

5.10 Converting Arrays to Strings - ការបំប្លែងអារេទៅជាអក្សរ

join() និង toString() បំប្លែងអារេទៅជាអក្សរដោយបញ្ចាក់អក្សរភ្ជាប់ (ឬក្បៀសលំនាំដើមសម្រាប់ toString)។

ឧទាហរណ៍៖

```
const fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];
console.log(fruits.join(" - "));
console.log(fruits.toString());
```

ការពន្យល់៖ join(" - ") បញ្ចូលធាតុជាមួយ " - " ជាអក្សរភ្ជាប់។ toString() បញ្ចូលធាតុជាមួយក្បៀសជាលំនាំដើម។ អារេដើមមិនត្រូវបានកែប្រែទេ។

លទ្ធផល៖

```
ផ្លែប៉ោម - ចេក - ស្វាយ
ផ្លែប៉ោម,ចេក,ស្វាយ
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃអារេនៅក្នុង JavaScript៖

- Creating and Accessing Arrays: បង្កើតអារេដោយប្រើ [] និងចូលប្រើធាតុជាមួយលេខសន្ទស្សន៍។
- Array Methods: push/pop, shift/unshift, slice/splice, forEach, map, filter, និង reduce សម្រាប់ការគ្រប់គ្រងអារេ។
- Array Destructuring: ទាញធាតុទៅជាអថេរដោយផ្ទាល់។

- `Spread Operator`: ពន្លាអារេសម្រាប់ចម្លង ឬបញ្ចូលអារេ។
- `Rest Parameter`: ប្រមូលប៉ារ៉ាម៉ែត្រទៅជាអារេនៅក្នុងអនុគមន៍។
- `Finding and Searching`: `indexOf`, `find`, `findIndex`, និង `includes` សម្រាប់ស្វែងរកធាតុ។
- `Sorting and Reversing`: `sort` និង `reverse` សម្រាប់រៀបចំលំដាប់អារេ។
- `Checking Arrays`: `Array.isArray` ពិនិត្យថាតើតម្លៃជាអារេ។
- `Concatenating Arrays`: `concat` បញ្ចូលអារេច្រើន។
- `Converting to Strings`: `join` និង `toString` បំប្លែងអារេទៅជាអក្សរ។

ការយល់ដឹងអំពីអារេ និងវិធីសាស្ត្រទាំងនេះគឺជាមូលដ្ឋានគ្រឹះសម្រាប់ការគ្រប់គ្រងទិន្នន័យប្រកបដោយប្រសិទ្ធភាពនៅក្នុង JavaScript។

6. ES6 Features - លក្ខណៈពិសេស ES6

ES6 (ECMAScript 2015) បានណែនាំលក្ខណៈពិសេសថ្មីៗជាច្រើនដែលធ្វើឱ្យការសរសេរកូដ JavaScript កាន់តែសង្ខេប អានបានងាយស្រួល និងមានអនុភាព។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី `let` និង `const`, `template literals`, `destructuring`, `default parameters`, `spread operator`, `rest parameters`, `modules`, និង `classes` ដែលជួយធ្វើឱ្យកូដមានរចនាសម្ព័ន្ធ និងទំនើប។

6.1 Let and Const (Block Scope) - អថេរ Let និង Const

`let` និង `const` គឺជាវិធីថ្មីសម្រាប់ប្រកាសអថេរនៅក្នុង ES6 ដែលប្រើ `block scope` (សំដៅលើប្លុកកូដដែលនៅក្នុង) ផ្ទុយពី `var` ដែលប្រើ `function scope`។ `let` អនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃ ខណៈ `const` មិនអនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃបន្ទាប់ពីកំណត់។

ឧទាហរណ៍៖

```
// let
let score = 10;
if (true) {
    let score = 20; // block scope
    console.log(score); // អថេរក្នុងប្លុក
}
console.log(score); // អថេរខាងក្រៅ

// const
const name = "សុខា";
```

```
console.log(name);
// name = "វណ្ណៈ"; // នឹងបណ្តាលឱ្យមានកំហុស
```

ការពន្យល់៖ let score ក្នុងប្លុក if មិនប៉ះពាល់ដល់ score ខាងក្រៅ ដោយសារតែ block scope។ const name មិនអនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃបន្ទាប់ពីកំណត់ ប៉ុន្តែអនុញ្ញាតឱ្យកែប្រែលក្ខណៈសម្បត្តិនៃវត្ថុបើជា const ជាវត្ថុ។

លទ្ធផល៖

```
20
10
សុខា
```

6.2 Template Literals (String Interpolation) - អក្សរគំរូ

Template literals ប្រើ ` (backticks) ដើម្បីបង្កើតអក្សរដែលអនុញ្ញាតឱ្យបញ្ចូលអថេរ ឬកន្សោមដោយប្រើ $ និងគាំទ្រអក្សរច្រើនជួរ។

ឧទាហរណ៍៖

```
const name = "វណ្ណៈ";
const age = 25;

const greeting = `សួស្តី ${name}! អ្នកមានអាយុ ${age} ឆ្នាំ។`;
console.log(greeting);

const multiLine = `នេះជាអក្សរ
ច្រើនជួរ
នៅក្នុង ES6`;
console.log(multiLine);
```

ការពន្យល់៖ $ និង $ បញ្ចូលតម្លៃអថេរទៅក្នុងអក្សរ។ Template literals អនុញ្ញាតឱ្យសរសេរអក្សរច្រើនជួរដោយមិនចាំបាច់ប្រើ \n ដែលធ្វើឱ្យកូដអានបានងាយស្រួល។

លទ្ធផល៖

```
សួស្តី វណ្ណៈ! អ្នកមានអាយុ 25 ឆ្នាំ។
នេះជាអក្សរ
ច្រើនជួរ
នៅក្នុង ES6
```

6.3 Destructuring (Arrays and Objects) - ការបំបែក

Destructuring អនុញ្ញាតឱ្យអ្នកទាញតម្លៃចេញពីអារេ ឬវត្ថុទៅជាអថេរដោយផ្ទាល់ ធ្វើឱ្យកូដសង្ខេប និងអានបានងាយស្រួល។

ឧទាហរណ៍៖

```
// Array Destructuring
const colors = ["ក្រហម", "បៃតង", "ខៀវ"];
const [first, second] = colors;
console.log(first, second);

// Object Destructuring
const person = { name: "សុខា", age: 25 };
const { name, age } = person;
console.log(name, age);

// ប្តូរឈ្មោះអថេរ
const { name: personName } = person;
console.log(personName);
```

ការពន្យល់៖ [first, second] ទាញធាតុទីមួយ និងទីពីរពីអារេ។ ទាញលក្ខណៈសម្បត្តិពីវត្ថុ។ name: personName ប្តូរឈ្មោះអថេរសម្រាប់ភាពច្បាស់លាស់។ Destructuring កាត់បន្ថយការសរសេរកូដច្រើនដង។

លទ្ធផល៖

```
ក្រហម បៃតង
សុខា 25
សុខា
```

6.4 Default Parameters - ប៉ារ៉ាម៉ែត្រលំនាំដើម

Default parameters អនុញ្ញាតឱ្យកំណត់តម្លៃលំនាំដើមសម្រាប់ប៉ារ៉ាម៉ែត្រអនុគមន៍ ប្រសិនបើគ្មានតម្លៃត្រូវបានបញ្ជូន។

ឧទាហរណ៍៖

```
function greet(name = "ភ្ញៀវ") {
    console.log(`សួស្តី ${name}`);
}
```

```
greet();
greet("វណ្ណៈ");
```

ការពន្យល់៖ name = "ភ្ញៀវ" កំណត់តម្លៃលំនាំដើម។ បើមិនបញ្ជូនប៉ារ៉ាម៉ែត្រ អនុគមន៍ប្រើ "ភ្ញៀវ"។ បើបញ្ជូនតម្លៃ ដូចជា "វណ្ណៈ" វានឹងប្រើតម្លៃនោះជំនួស។

លទ្ធផល៖

```
សួស្តី ភ្ញៀវ
សួស្តី វណ្ណៈ
```

6.5 Spread Operator (...) - ប្រតិបត្តិករពន្លា

Spread operator (...) ពន្លាធាតុនៃអារេ ឬលក្ខណៈសម្បត្តិនៃវត្ថុទៅជាធាតុនីមួយៗ ឬចម្លងទៅទីតាំងថ្មី។

ឧទាហរណ៍៖

```
// អារេ
const numbers = [1, 2];
const newNumbers = [...numbers, 3, 4];
console.log(newNumbers);

// វត្ថុ
const person = { name: "សុខា" };
const updatedPerson = { ...person, age: 25 };
console.log(updatedPerson);
```

ការពន្យល់៖ ...numbers ពន្លាធាតុនៃ numbers ទៅជាអារេថ្មី។ ...person ចម្លងលក្ខណៈសម្បត្តិនៃ person ទៅវត្ថុថ្មី។ Spread operator ស័ក្តិសមសម្រាប់ការចម្លង ឬបញ្ចូលទិន្នន័យ។

លទ្ធផល៖

```
[ 1, 2, 3, 4 ]
{ name: 'សុខា', age: 25 }
```

6.6 Rest Parameters (...args) - ប៉ារ៉ាម៉ែត្រសល់

Rest parameters (...) ប្រមូលប៉ារ៉ាម៉ែត្រទាំងអស់នៅក្នុងអនុគមន៍ទៅជាអារេ ដែលអនុញ្ញាតឱ្យគ្រប់គ្រងចំនួនប៉ារ៉ាម៉ែត្រមិនកំណត់។

ឧទាហរណ៍៖

```
function sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
}
console.log(sum(1, 2, 3));
console.log(sum(10, 20, 30, 40));
```

ការពន្យល់៖ ...numbers ប្រមូលប៉ារ៉ាម៉ែត្រទាំងអស់ទៅជាអារេ។ reduce បូកធាតុទាំងអស់។ Rest parameters ធ្វើឱ្យអនុគមន៍អាចទទួលប៉ារ៉ាម៉ែត្របានច្រើន។

លទ្ធផល៖

```
6
100
```

6.7 Modules (import and export) - ម៉ូឌុល

ES6 បានណែនាំ modules ដែលអនុញ្ញាតឱ្យអ្នកបំបែកកូដទៅជាឯកសារផ្សេងៗគ្នា និងប្រើ export និង import ដើម្បីចែករំលែកអនុគមន៍ អថេរ ឬវត្ថុ។

ឧទាហរណ៍៖

ឯកសារ math.js:

```
// math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
```

ឯកសារ main.js:

```
// main.js
import { add, subtract } from './math.js';

console.log(add(5, 3));
console.log(subtract(5, 3));
```

ការពន្យល់៖ export នៅក្នុង math.js ធ្វើឱ្យ add និង subtract អាចប្រើបាន។ import នៅក្នុង main.js នាំយកអនុគមន៍ទាំងនេះសម្រាប់ប្រើប្រាស់។ Modules ជួយរៀបចំកូដ និងកាត់បន្ថយការប៉ះទង្គិច។

លទ្ធផល៖

```
8
```

```
2
```

6.8 Classes: Constructor, Methods, Inheritance, Static Methods - ថ្នាក់

Classes នៅក្នុង ES6 ផ្តល់វិធីសាស្ត្រទំនើបសម្រាប់បង្កើតវត្ថុ និងគ្រប់គ្រងមរតក។ ពួកវាមាន constructor, methods, inheritance និង static methods។

ឧទាហរណ៍៖

```javascript
// ថ្នាក់មូលដ្ឋាន
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    greet() {
        console.log(`សួស្តី ខ្ញុំឈ្មោះ ${this.name}`);
    }

    static info() {
        console.log("នេះជាថ្នាក់ Person");
    }
}

// មរតក
class Student extends Person {
    constructor(name, age, grade) {
        super(name, age);
        this.grade = grade;
    }

    study() {
        console.log(`${this.name} កំពុងសិក្សា និទ្ទេស ${this.grade}`);
    }
}

const student = new Student("សុខា", 25, "A");
student.greet();
```

```
student.study();
Person.info();
```

ការពន្យល់៖ constructor កំណត់លក្ខណៈសម្បត្តិនៃ Person។ greet ជាវិធីសាស្ត្រសម្រាប់វត្ថុ។ static info ជាវិធីសាស្ត្រដែលហៅតាមថ្នាក់។ Student ទទួលមរតកពី Person ដោយប្រើ extends និង super ដើម្បីចាប់ផ្តើមលក្ខណៈសម្បត្តិមេ។

លទ្ធផល៖

```
សួស្តី ខ្ញុំឈ្មោះ សុខា
សុខា កំពុងសិក្សា និទ្ទេស A
នេះជាថ្នាក់ Person
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើលក្ខណៈពិសេសសំខាន់ៗនៃ ES6៖

- Let and Const: ប្រកាសអថេរជាមួយ block scope ដែល let អនុញ្ញាតឱ្យផ្លាស់ប្តូរតម្លៃ និង const មិនអនុញ្ញាត។
- Template Literals: ប្រើ ` សម្រាប់អក្សរដែលគាំទ្រការបញ្ចូលអថេរ និងអក្សរច្រើនជួរ។
- Destructuring: ទាញតម្លៃពីអារេ ឬវត្ថុទៅជាអថេរដោយផ្ទាល់។
- Default Parameters: កំណត់តម្លៃលំនាំដើមសម្រាប់ប៉ារ៉ាម៉ែត្រអនុគមន៍។
- Spread Operator: ពន្លាអារេ ឬវត្ថុសម្រាប់ចម្លង ឬបញ្ចូលទិន្នន័យ។
- Rest Parameters: ប្រមូលប៉ារ៉ាម៉ែត្រទៅជាអារេនៅក្នុងអនុគមន៍។
- Modules: ប្រើ import និង export ដើម្បីរៀបចំកូដនៅក្នុងឯកសារផ្សេងៗ។
- Classes: បង្កើតវត្ថុជាមួយ constructor, methods, inheritance, និង static methods។

លក្ខណៈពិសេសទាំងនេះធ្វើឱ្យកូដ JavaScript មានភាពទំនើប ងាយស្រួលថែទាំ និងអាចពង្រីកបាន។

7. Asynchronous JavaScript - JavaScript អសមកាល

JavaScript ជាភាសាដែលដំណើរការជា single-threaded ប៉ុន្តែវាមានយន្តការសម្រាប់គ្រប់គ្រងប្រតិបត្តិការអសមកាល (asynchronous operations) ដូចជាការទាញទិន្នន័យពី API ឬការរង់ចាំព្រឹត្តិការណ៍។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី callbacks, promises, async/await, setTimeout/setInterval, និង event loop ដែលជួយធ្វើឱ្យកម្មវិធី JavaScript មានប្រសិទ្ធភាព និងឆ្លើយតបបានល្អ។

7.1 Callbacks - អនុគមន៍ហៅត្រឡប់

Callbacks គឺជាអនុគមន៍ដែលត្រូវបានបញ្ជូនជាប៉ារ៉ាម៉ែត្រទៅអនុគមន៍ផ្សេង និងត្រូវបានហៅបន្ទាប់ពីប្រតិបត្តិការអសមកាលបញ្ចប់។ វាជាវិធីសាមញ្ញបំផុតសម្រាប់គ្រប់គ្រងការងារអសមកាល។

ឧទាហរណ៍៖

```
function fetchData(callback) {
    setTimeout(() => {
        callback("ទិន្នន័យត្រូវបានទាញ");
    }, 1000);
}

fetchData(message => {
    console.log(message);
});
```

ការពន្យល់៖ fetchData ទទួល callback ជាប៉ារ៉ាម៉ែត្រ និងហៅវាបន្ទាប់ពីរង់ចាំ 1 វិនាទី។ អនុគមន៍ callback បោះពុម្ពសារទៅកុងសូល។ ទោះបីជា callbacks មានប្រយោជន៍ ក៏ប៉ុន្តែវាអាចបង្កើត callback hell ប្រសិនបើមានការហៅជាច្រើន។

លទ្ធផល៖

```
ទិន្នន័យត្រូវបានទាញ
```

7.2 Promises: then(), catch(), finally() - ការសន្យា

Promises គឺជាវត្ថុដែលតំណាងឱ្យលទ្ធផលនៃប្រតិបត្តិការអសមកាល។ វាមានបីស្ថានភាព៖ pending, fulfilled, និង rejected។ អ្នកប្រើ then() ដើម្បីគ្រប់គ្រងលទ្ធផលជោគជ័យ, catch() សម្រាប់កំហុស, និង finally() សម្រាប់សកម្មភាពបញ្ចប់។

ឧទាហរណ៍៖

```
const fetchData = new Promise((resolve, reject) => {
    setTimeout(() => {
        const success = true;
        if (success) {
            resolve("ទិន្នន័យជោគជ័យ");
        } else {
            reject("មានកំហុស");
        }
    }, 1000);
```

```
});

fetchData
    .then(data => console.log(data))
    .catch(error => console.log(error))
    .finally(() => console.log("បញ្ចប់ការទាញទិន្នន័យ"));
```

ការពន្យល់៖ Promise បង្កើតការសន្យាដែលដោះស្រាយ (resolve) ឬបដិសេធ (reject) បន្ទាប់ពី 1 វិនាទី។ then ដោះស្រាយទិន្នន័យជោគជ័យ, catch ចាប់កំហុស, និង finally ប្រតិបត្តិទោះបីជាមានលទ្ធផលណាក៏ដោយ។ Promises ធ្វើឱ្យការគ្រប់គ្រងអសមកាលកាន់តែច្បាស់ជាង callbacks។

លទ្ធផល៖

```
ទិន្នន័យជោគជ័យ
បញ្ចប់ការទាញទិន្នន័យ
```

7.3 Async/Await - អសមកាល/រង់ចាំ

Async/Await គឺជាវាក្យសម្ព័ន្ធដែលធ្វើឱ្យការគ្រប់គ្រង promises មើលទៅដូចកូដសមកាល។ async ប្រកាសអនុគមន៍ថាជាអសមកាល និង await រង់ចាំរហូតដល់ promise ដោះស្រាយ។

ឧទាហរណ៍៖

```
async function fetchData() {
    try {
        const promise = new Promise(resolve => {
            setTimeout(() => resolve("ទិន្នន័យជោគជ័យ"), 1000);
        });
        const data = await promise;
        console.log(data);
    } catch (error) {
        console.log(error);
    } finally {
        console.log("បញ្ចប់ការទាញទិន្នន័យ");
    }
}

fetchData();
```

ការពន្យល់៖ async ធ្វើឱ្យ fetchData ប្រគល់ promise។ await ផ្អាកការប្រតិបត្តិរហូតដល់ promise ដោះស្រាយ។ try/catch គ្រប់គ្រងកំហុស និង finally ប្រតិបត្តិនៅទីបញ្ចប់។ Async/await

ធ្វើឱ្យកូដអានបានងាយស្រួលជាង then/catch។

លទ្ធផល៖

```
ទិន្នន័យជោគជ័យ
បញ្ចប់ការទាញទិន្នន័យ
```

7.4 setTimeout() / setInterval() - កម្មវិធីកំណត់ពេល

setTimeout() ប្រតិបត្តិអនុគមន៍បន្ទាប់ពីកំណត់ពេលវេលា (គិតជាមិល្លីវិនាទី)។ setInterval() ប្រតិបត្តិអនុគមន៍ម្តងហើយម្តងទៀតនៅចន្លោះពេលកំណត់។

ឧទាហរណ៍៖

```
// setTimeout
setTimeout(() => {
    console.log("បន្ទាប់ពី 1 វិនាទី");
}, 1000);

// setInterval
let count = 0;
const intervalId = setInterval(() => {
    console.log(`ចំនួន: ${++count}`);
    if (count === 3) {
        clearInterval(intervalId);
        console.log("បញ្ឈប់ setInterval");
    }
}, 1000);
```

ការពន្យល់៖ setTimeout បោះពុម្ពសារបន្ទាប់ពី 1 វិនាទី។ setInterval បោះពុម្ព count រៀងរាល់ 1 វិនាទីរហូតដល់ count ឈានដល់ 3 ហើយ clearInterval បញ្ឈប់វា។ ឧបករណ៍ទាំងនេះជួយគ្រប់គ្រងការពន្យាពេល និងការធ្វើម្តងហើយម្តងទៀត។

លទ្ធផល៖

```
បន្ទាប់ពី 1 វិនាទី
ចំនួន: 1
ចំនួន: 2
ចំនួន: 3
បញ្ឈប់ setInterval
```

7.5 Event Loop and Call Stack - រង្វិលជុំព្រឹត្តិការណ៍ និងជង់ហៅ

Event Loop និង Call Stack គឺជាយន្តការស្នូលនៅក្នុង JavaScript សម្រាប់គ្រប់គ្រងប្រតិបត្តិការអសមកាល។ Call Stack ដំណើរការអនុគមន៍តាមលំដាប់ LIFO (Last In, First Out)។ Event Loop ផ្លាស់ប្តូរការងារអសមកាល (ដូចជា callbacks ពី setTimeout) ពី task queue ទៅ call stack នៅពេល call stack ទទេ។

ឧទាហរណ៍៖

```
console.log("ចាប់ផ្តើម");

setTimeout(() => {
    console.log("setTimeout callback");
}, 0);

console.log("បញ្ចប់");
```

ការពន្យល់៖ Call Stack ដំណើរការ console.log("ចាប់ផ្តើម") និង console.log("បញ្ចប់") មុន។ setTimeout ដាក់ callback ទៅក្នុង task queue។ Event Loop ផ្លាស់ប្តូរ callback ទៅ call stack បន្ទាប់ពី call stack ទទេ ទោះបីជាពេលវេលាកំណត់ជា 0 ក៏ដោយ។ នេះបង្ហាញពីអាទិភាពនៃកូដសមកាលលើអសមកាល។

លទ្ធផល៖

```
ចាប់ផ្តើម
បញ្ចប់
setTimeout callback
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃ JavaScript អសមកាល៖

- Callbacks: អនុគមន៍ដែលហៅបន្ទាប់ពីប្រតិបត្តិការអសមកាល ប៉ុន្តែអាចបង្កើត callback hell។
- Promises: គ្រប់គ្រងប្រតិបត្តិការអសមកាលជាមួយ then, catch, និង finally សម្រាប់ភាពច្បាស់លាស់។
- Async/Await: វាក្យសម្ព័ន្ធដែលធ្វើឱ្យ promises អានបានដូចកូដសមកាល ជាមួយ try/catch។
- setTimeout/setInterval: ឧបករណ៍សម្រាប់កំណត់ពេល និងប្រតិបត្តិការម្តងហើយម្តងទៀត។
- Event Loop and Call Stack: យន្តការសម្រាប់គ្រប់គ្រងកូដសមកាល និងអសមកាល ដោយផ្លាស់ប្តូរការងារពី task queue ទៅ call stack។

ការយល់ដឹងអំពីអសមកាលទាំងនេះគឺសំខាន់សម្រាប់ការសរសេរកម្មវិធី JavaScript ដែលឆ្លើយតបបានល្អ និងមានប្រសិទ្ធភាព។

8. DOM (Document Object Model) Manipulation - ការគ្រប់គ្រង DOM

`DOM (Document Object Model)` គឺជារចនាសម្ព័ន្ធដែលតំណាងឱ្យទំព័របណ្ដាញជាដើមឈើនៃវត្ថុ។ JavaScript អនុញ្ញាតឱ្យអ្នកធ្វើអន្តរកម្មជាមួយ DOM ដើម្បីចូលប្រើ កែប្រែ បង្កើត ឬលុបធាតុនៅលើទំព័របណ្ដាញ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការចូលប្រើធាតុ DOM, ការកែប្រែខ្លឹមសារ និងលក្ខណៈសម្បត្តិ, ការបង្កើត និងលុបធាតុ, ការគ្រប់គ្រងព្រឹត្តិការណ៍, ការកែប្រែស្ទីល, និងការធ្វើដំណើរក្នុង DOM។

8.1 Accessing DOM Elements: getElementById(), querySelector(), querySelectorAll() - ការចូលប្រើធាតុ DOM

JavaScript ផ្ដល់វិធីសាស្ត្រដើម្បីចូលប្រើធាតុ DOM ដូចជា `getElementById()`, `querySelector()`, និង `querySelectorAll()`។ វិធីសាស្ត្រទាំងនេះអនុញ្ញាតឱ្យអ្នកជ្រើសរើសធាតុដោយផ្អែកលើ ID, CSS selector ឬលក្ខណៈផ្សេងៗ។

ឧទាហរណ៍៖

```html
<!-- index.html -->
<div id="myDiv">សួស្ដី</div>
<p class="text">អក្សរ</p>
<p class="text">អក្សរទីពីរ</p>
```

```javascript
// getElementById
const div = document.getElementById("myDiv");
console.log(div.textContent);

// querySelector
const firstText = document.querySelector(".text");
console.log(firstText.textContent);

// querySelectorAll
```

```
const allTexts = document.querySelectorAll(".text");
allTexts.forEach(text => console.log(text.textContent));
```

ការពន្យល់៖ `getElementById` ជ្រើសរើសធាតុតាម ID។ `querySelector` ជ្រើសរើសធាតុដំបូងដែលផ្គូផ្គងជាមួយ CSS selector។ `querySelectorAll` ប្រគល់ `NodeList` នៃធាតុទាំងអស់ដែលផ្គូផ្គង ដែលអាចត្រូវបានឆ្លងកាត់ដោយ `forEach`។

លទ្ធផល៖

```
សួស្តី
អក្សរ
អក្សរ
អក្សរទីពីរ
```

8.2 Modifying Content and Attributes: innerHTML, textContent, setAttribute() - ការកែប្រែខ្លឹមសារ និងលក្ខណៈសម្បត្តិ

អ្នកអាចកែប្រែខ្លឹមសារនៃធាតុ DOM ដោយប្រើ `innerHTML` ឬ `textContent` និងកែប្រែលក្ខណៈសម្បត្តិ (attributes) ដោយប្រើ `setAttribute()`។

ឧទាហរណ៍៖

```
<!-- index.html -->
<div id="myDiv">សួស្តី</div>
<img id="myImage" src="old.jpg">
```

```
const div = document.getElementById("myDiv");

// innerHTML
div.innerHTML = "<strong>សួស្តីថ្មី</strong>";

// textContent
div.textContent = "អក្សរសាមញ្ញ";

// setAttribute
const img = document.getElementById("myImage");
img.setAttribute("src", "new.jpg");
console.log(img.getAttribute("src"));
```

ការពន្យល់៖ innerHTML កំណត់ខ្លឹមសារជាមួយ HTML tags។ `textContent` កំណត់អក្សរសុទ្ធ ដោយមិនបកស្រាយ HTML។ `setAttribute` កែប្រែលក្ខណៈសម្បត្តិ ដូចជា `src` នៃរូបភាព។ `getAttribute` ទាញតម្លៃលក្ខណៈសម្បត្តិ។

លទ្ធផល៖

```
new.jpg
```

8.3 Creating and Removing Elements: createElement(), removeChild() - ការបង្កើត និងលុបធាតុ

`createElement()` បង្កើតធាតុ DOM ថ្មី និង `removeChild()` លុបធាតុចេញពី DOM។ អ្នកអាចបន្ថែមធាតុដោយប្រើ `appendChild()` ឬ `append()`។

ឧទាហរណ៍៖

```html
<!-- index.html -->
<div id="container">
    <p id="oldText">អក្សរចាស់</p>
</div>
```

```javascript
const container = document.getElementById("container");

// បង្កើតធាតុថ្មី
const newText = document.createElement("p");
newText.textContent = "អក្សរថ្មី";
container.appendChild(newText);

// លុបធាតុ
const oldText = document.getElementById("oldText");
container.removeChild(oldText);

console.log(container.innerHTML);
```

ការពន្យល់៖ `createElement("p")` បង្កើតធាតុ p ថ្មី។ `appendChild` បន្ថែមវាទៅ `container`។ `removeChild` លុប `oldText` ចេញពី `container`។ លទ្ធផលបង្ហាញថា `container` មានតែអក្សរថ្មី។

លទ្ធផល៖

```
<p>អក្សរថ្មី</p>
```

8.4 Event Handling: addEventListener(), Event Delegation - ការគ្រប់គ្រងព្រឹត្តិការណ៍

addEventListener() ភ្ជាប់អនុគមន៍ទៅនឹងព្រឹត្តិការណ៍ (ដូចជា click) ទៅលើធាតុ DOM។ Event delegation គឺជាបច្ចេកទេសដែលប្រើធាតុមេដើម្បីគ្រប់គ្រងព្រឹត្តិការណ៍នៃធាតុកូន។

ឧទាហរណ៍៖

```
<!-- index.html -->
<button id="myButton">ចុចខ្ញុំ</button>
<ul id="list">
    <li>ធាតុ 1</li>
    <li>ធាតុ 2</li>
</ul>
```

```
// addEventListener
const button = document.getElementById("myButton");
button.addEventListener("click", () => {
    console.log("ប៊ូតុងត្រូវបានចុច");
});

// Event Delegation
const list = document.getElementById("list");
list.addEventListener("click", event => {
    if (event.target.tagName === "LI") {
        console.log(`ធាតុ: ${event.target.textContent}`);
    }
});
```

ការពន្យល់៖ addEventListener ភ្ជាប់ព្រឹត្តិការណ៍ click ទៅប៊ូតុង។ សម្រាប់ event delegation, ព្រឹត្តិការណ៍ click នៅលើ ul ចាប់ព្រឹត្តិការណ៍នៃធាតុ li ដោយពិនិត្យ event.target។ វិធីនេះមានប្រសិទ្ធភាពសម្រាប់ធាតុដែលបន្ថែមថ្មី។

លទ្ធផល៖ (នៅពេលចុចប៊ូតុង និងធាតុ li)

```
ប៊ូតុងត្រូវបានចុច
ធាតុ: ធាតុ 1
```

8.5 Styling with JavaScript: style Property - ការកែប្រែស្ទីលជាមួយ JavaScript

អ្នកអាចកែប្រែស្ទីល CSS នៃធាតុ DOM ដោយប្រើលក្ខណៈសម្បត្តិ `style` ដែលអនុញ្ញាតឱ្យកំណត់លក្ខណៈ CSS ដោយផ្ទាល់។

ឧទាហរណ៍៖

```
<!-- index.html -->
<div id="myDiv">សួស្តី</div>
```

```
const div = document.getElementById("myDiv");
div.style.backgroundColor = "lightblue";
div.style.fontSize = "20px";
div.style.padding = "10px";

console.log(div.style.backgroundColor);
```

ការពន្យល់៖ `style` កំណត់លក្ខណៈ CSS ដូចជា `backgroundColor` (ប្រើ camelCase ជំនួស `background-color`)។ លក្ខណៈសម្បត្តិ `style` ប៉ះពាល់ដល់ inline CSS នៃធាតុ។ ការទាញតម្លៃ `style` ប្រគល់តែតម្លៃ inline CSS ប៉ុណ្ណោះ។

លទ្ធផល៖

```
lightblue
```

8.6 DOM Traversal: Parent, Children, Siblings - ការធ្វើដំណើរក្នុង DOM

ការធ្វើដំណើរក្នុង DOM អនុញ្ញាតឱ្យអ្នករុករកទំនាក់ទំនងរវាងធាតុ ដូចជា `parentElement`, `children`, `firstChild`, `nextSibling`, និង `previousSibling`។

ឧទាហរណ៍៖

```
<!-- index.html -->
<div id="parent">
    <p>ធាតុ 1</p>
    <p>ធាតុ 2</p>
</div>
```

```
const parent = document.getElementById("parent");

// មេ
console.log(parent.tagName);

// កូន
const children = parent.children;
console.log(children[0].textContent);

// បងប្អូន
const firstChild = parent.firstElementChild;
const nextSibling = firstChild.nextElementSibling;
console.log(nextSibling.textContent);
```

ការពន្យល់៖ parent.tagName ប្រគល់ឈ្មោះធាតុមេ។ children ប្រគល់ HTMLCollection នៃធាតុកូន។ firstElementChild និង nextElementSibling អនុញ្ញាតឱ្យរុករកធាតុបងប្អូន។ លក្ខណៈសម្បត្តិទាំងនេះជួយរុករករចនាសម្ព័ន្ធ DOM។

លទ្ធផល៖

```
DIV
ធាតុ 1
ធាតុ 2
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃការគ្រប់គ្រង DOM នៅក្នុង JavaScript៖

- Accessing DOM Elements: ប្រើ getElementById, querySelector, និង querySelectorAll ដើម្បីជ្រើសរើសធាតុ។
- Modifying Content and Attributes: កែប្រែខ្លឹមសារជាមួយ innerHTML, textContent, និងលក្ខណៈសម្បត្តិជាមួយ setAttribute។
- Creating and Removing Elements: បង្កើតធាតុជាមួយ createElement និងលុបជាមួយ removeChild។
- Event Handling: គ្រប់គ្រងព្រឹត្តិការណ៍ជាមួយ addEventListener និងប្រើ event delegation សម្រាប់ប្រសិទ្ធភាព។
- Styling with JavaScript: កែប្រែស្ទីល CSS ដោយប្រើលក្ខណៈសម្បត្តិ style។
- DOM Traversal: រុករក DOM ដោយប្រើ parentElement, children, និង nextElementSibling។

ការយល់ដឹងអំពីការគ្រប់គ្រង DOM គឺសំខាន់សម្រាប់ការបង្កើតទំព័របណ្ដាញអន្តរកម្ម និងថាមវន្ត។

9. Event Handling - ការគ្រប់គ្រងព្រឹត្តិការណ៍

ការគ្រប់គ្រងព្រឹត្តិការណ៍នៅក្នុង JavaScript អនុញ្ញាតឱ្យអ្នកឆ្លើយតបទៅនឹងសកម្មភាពរបស់អ្នកប្រើប្រាស់ ដូចជាការចុចកណ្ដុរ ការវាយអក្សរ ឬការផ្ទុកទំព័របណ្ដាញ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីប្រភេទព្រឹត្តិការណ៍ DOM, ការប្រើ `addEventListener` និង `removeEventListener`, `event propagation` (bubbling និង capturing), និងការចូលប្រើលក្ខណៈសម្បត្តិនៃ `event object`។

9.1 DOM Events: Mouse, Keyboard, Form, Window Events - ព្រឹត្តិការណ៍ DOM

ព្រឹត្តិការណ៍ DOM គឺជាសកម្មភាពដែលកើតឡើងនៅលើទំព័របណ្ដាញ ដូចជាព្រឹត្តិការណ៍កណ្ដុរ (`click`, `mouseover`), ក្ដារចុច (`keydown`, `keyup`), ទម្រង់ (`submit`, `change`), និងបង្អួច (`load`, `resize`)។

ឧទាហរណ៍៖

```
<!-- index.html -->
<button id="btn">ចុចខ្ញុំ</button>
<input id="input" type="text" placeholder="វាយអក្សរ">
<form id="form">
    <input type="text">
    <button type="submit">ដាក់ស្នើ</button>
</form>
```

```
// Mouse Event
const btn = document.getElementById("btn");
btn.addEventListener("click", () => console.log("ប៊ូតុងត្រូវបានចុច"));

// Keyboard Event
const input = document.getElementById("input");
input.addEventListener("keydown", () => console.log("គ្រាប់ចុចត្រូវបានចុច")

// Form Event
const form = document.getElementById("form");
```

```
form.addEventListener("submit", e => {
    e.preventDefault();
    console.log("ទម្រង់ត្រូវបានដាក់ស្នើ");
});

// Window Event
window.addEventListener("load", () => console.log("ទំព័របានផ្ទុករួចរាល់"));
```

ការពន្យល់៖ `click` ចាប់ព្រឹត្តិការណ៍ចុចប៊ូតុង។ `keydown` ចាប់ការវាយអក្សរនៅក្នុង `input`។ `submit` ចាប់ការដាក់ស្នើទម្រង់ ដោយ `e.preventDefault()` ទប់ស្កាត់ការផ្ទុកទំព័រឡើងវិញ។ `load` ប្រតិបត្តិនៅពេលទំព័រផ្ទុករួចរាល់។

លទ្ធផល៖ (នៅពេលផ្ទុកទំព័រ ចុចប៊ូតុង វាយអក្សរ និងដាក់ស្នើទម្រង់)

```
ទំព័របានផ្ទុករួចរាល់
ប៊ូតុងត្រូវបានចុច
គ្រាប់ចុចត្រូវបានចុច
ទម្រង់ត្រូវបានដាក់ស្នើ
```

9.2 Event Listeners: addEventListener(), removeEventListener() - អ្នកស្តាប់ព្រឹត្តិការណ៍

`addEventListener()` ភ្ជាប់អនុគមន៍ទៅនឹងព្រឹត្តិការណ៍នៃធាតុ DOM។ `removeEventListener()` លុបអ្នកស្តាប់ព្រឹត្តិការណ៍ ដែលមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងធនធាន ឬការពារព្រឹត្តិការណ៍ដែលមិនចង់បាន។

ឧទាហរណ៍៖

```
<!-- index.html -->
<button id="btn">ចុចខ្ញុំ</button>
```

```
const btn = document.getElementById("btn");

const handleClick = () => {
    console.log("ប៊ូតុងត្រូវបានចុច");
};

// បន្ថែម Event Listener
btn.addEventListener("click", handleClick);
```

```
// លុប Event Listener បន្ទាប់ពី 3 វិនាទី
setTimeout(() => {
    btn.removeEventListener("click", handleClick);
    console.log("Event Listener ត្រូវបានលុប");
}, 3000);
```

ការពន្យល់៖ addEventListener ភ្ជាប់ handleClick ទៅព្រឹត្តិការណ៍ click។ បន្ទាប់ពី 3 វិនាទី, removeEventListener លុប handleClick ដូច្នេះការចុចប៊ូតុងបន្ទាប់មកមិនបង្កឱ្យមានសកម្មភាពទៀតទេ។ removeEventListener ទាមទារអនុគមន៍ដូចគ្នាដែលបានបន្ថែម។

លទ្ធផល៖ (នៅពេលចុចប៊ូតុងមុន 3 វិនាទី និងបន្ទាប់ពី 3 វិនាទី)

```
ប៊ូតុងត្រូវបានចុច
Event Listener ត្រូវបានលុប
```

9.3 Event Propagation: Bubbling and Capturing - ការផ្សព្វផ្សាយព្រឹត្តិការណ៍

Event propagation ពណ៌នាអំពីរបៀបដែលព្រឹត្តិការណ៍ផ្សព្វផ្សាយតាមរចនាសម្ព័ន្ធ DOM។ វាមានពីរដំណាក់កាល៖ capturing (ពីបង្អួចទៅធាតុគោលដៅ) និង bubbling (ពីធាតុគោលដៅទៅបង្អួច)។ អ្នកអាចគ្រប់គ្រងដំណាក់កាលទាំងនេះដោយប្រើប៉ារ៉ាម៉ែត្រ useCapture នៅក្នុង addEventListener ឬបញ្ឈប់ការផ្សព្វផ្សាយដោយ stopPropagation()។

ឧទាហរណ៍៖

```
<!-- index.html -->
<div id="parent">
    <button id="child">ចុចខ្ញុំ</button>
</div>
```

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");

// Bubbling
parent.addEventListener("click", () => console.log("ចុចមេ"));
child.addEventListener("click", () => console.log("ចុចកូន"));

// Capturing
parent.addEventListener("click", () => console.log("ចុចមេ (Capturing)")
```

```
// បញ្ឈប់ Propagation
child.addEventListener("click", e => {
    e.stopPropagation();
    console.log("ចុចកូន (បញ្ឈប់ Propagation)");
});
```

ការពន្យល់៖ នៅពេលចុច child, ព្រឹត្តិការណ៍ bubbling បណ្តាលឱ្យ child និង parent ប្រតិបត្តិតាមលំដាប់។ ប៉ារ៉ាម៉ែត្រ true នៅក្នុង addEventListener បើក capturing ដែលប្រតិបត្តិ parent មុន។ stopPropagation ទប់ស្កាត់ព្រឹត្តិការណ៍មិនឱ្យផ្សព្វផ្សាយទៅ parent។

លទ្ធផល៖ (នៅពេលចុចប៊ូតុង)

```
ចុចមេ (Capturing)
ចុចកូន (បញ្ឈប់ Propagation)
```

9.4 Event Object: Accessing Event Properties - វត្ថុព្រឹត្តិការណ៍

Event object គឺជាប៉ារ៉ាម៉ែត្រដែលផ្តល់ព័ត៌មានអំពីព្រឹត្តិការណ៍ ដូចជា target, type, clientX, ឬ key។ វាជួយអ្នកកំណត់សកម្មភាព និងបរិបទនៃព្រឹត្តិការណ៍។

ឧទាហរណ៍៖

```
<!-- index.html -->
<button id="btn">ចុចខ្ញុំ</button>
<input id="input" type="text" placeholder="វាយអក្សរ">
```

```
const btn = document.getElementById("btn");
const input = document.getElementById("input");

// Mouse Event Properties
btn.addEventListener("click", e => {
    console.log(`ប្រភេទ: ${e.type}, គោលដៅ: ${e.target.tagName}, ទីតាំង
});

// Keyboard Event Properties
input.addEventListener("keydown", e => {
```

```
    console.log(`គ្រាប់ចុច: ${e.key}, កូដ: ${e.code}`);
});
```

ការពន្យល់៖ `e.type` ប្រគល់ប្រភេទព្រឹត្តិការណ៍ (ឧ. `click`)។ `e.target` ចង្អុលទៅធាតុដែលបង្កព្រឹត្តិការណ៍។ `e.clientX` ផ្តល់ទីតាំងអ័ក្ស X នៃកណ្តុរ។ `e.key` និង `e.code` ផ្តល់ព័ត៌មានអំពីគ្រាប់ចុចដែលបានចុច។

លទ្ធផល៖ (នៅពេលចុចប៊ូតុង និងវាយអក្សរ "a")

```
ប្រភេទ: click, គោលដៅ: BUTTON, ទីតាំង X: 100
គ្រាប់ចុច: a, កូដ: KeyA
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃការគ្រប់គ្រងព្រឹត្តិការណ៍នៅក្នុង JavaScript៖

- `DOM Events`: ចាប់សកម្មភាពអ្នកប្រើប្រាស់ ដូចជាព្រឹត្តិការណ៍កណ្តុរ, ក្តារចុច, ទម្រង់, និងបង្អួច។
- `Event Listeners`: ប្រើ `addEventListener` ដើម្បីភ្ជាប់ព្រឹត្តិការណ៍ និង `removeEventListener` ដើម្បីលុបអ្នកស្តាប់។
- `Event Propagation`: យល់ពី `bubbling` និង `capturing` និងប្រើ `stopPropagation` ដើម្បីគ្រប់គ្រងការផ្សព្វផ្សាយ។
- `Event Object`: ចូលប្រើលក្ខណៈសម្បត្តិដូចជា `target`, `type`, `clientX`, និង `key` ដើម្បីទទួលព័ត៌មានព្រឹត្តិការណ៍។

ការគ្រប់គ្រងព្រឹត្តិការណ៍គឺជាមូលដ្ឋានគ្រឹះសម្រាប់ការបង្កើតទំព័របណ្តាញអន្តរកម្ម និងឆ្លើយតបបានល្អ។

10. Error Handling - ការគ្រប់គ្រងកំហុស

ការគ្រប់គ្រងកំហុសនៅក្នុង JavaScript អនុញ្ញាតឱ្យអ្នកចាប់ និងដោះស្រាយបញ្ហាដែលកើតឡើងអំឡុងពេលប្រតិបត្តិកម្មវិធី ដូចជាកំហុសក្នុងការបញ្ចូលទិន្នន័យ ឬការបរាជ័យនៃការហៅ API។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការប្រើ `try-catch`, `throw` ដើម្បីបង្កើតកំហុស, និងការបង្កើត `custom error objects` ដើម្បីគ្រប់គ្រងស្ថានភាពជាក់លាក់។

10.1 Try-Catch Block - ប្លុក Try-Catch

ប្លុក `try-catch` ត្រូវបានប្រើដើម្បីចាប់កំហុសដែលកើតឡើងនៅក្នុងកូដ។ កូដនៅក្នុង `try` ត្រូវបានប្រតិបត្តិ ហើយប្រសិនបើមានកំហុស កូដនៅក្នុង `catch` នឹងដោះស្រាយវា។ `finally` (ជាជម្រើស) ប្រតិបត្តិដោយមិនគិតពីលទ្ធផល។

ឧទាហរណ៍៖

```
try {
    const data = JSON.parse("ខ្សែអក្សរមិនត្រឹមត្រូវ");
    console.log(data);
} catch (error) {
    console.log("កំហុស:", error.message);
} finally {
    console.log("បញ្ចប់ការព្យាយាម");
}
```

ការពន្យល់៖ `JSON.parse` បរាជ័យដោយសារខ្សែអក្សរមិនមែនជា JSON ត្រឹមត្រូវ ដែលបណ្តាលឱ្យកំហុស។ `catch` ចាប់កំហុសនេះ និងបោះពុម្ពសារកំហុស។ `finally` ប្រតិបត្តិទោះបីជាមានកំហុសឬគ្មានកំហុស។ `try-catch` ការពារកម្មវិធីពីការគាំង។

លទ្ធផល៖

```
កំហុស: Unexpected token  in JSON at position 0
បញ្ចប់ការព្យាយាម
```

10.2 Throwing Errors - ការបង្កើតកំហុស

`throw` អនុញ្ញាតឱ្យអ្នកបង្កើតកំហុសផ្ទាល់ខ្លួននៅពេលមានលក្ខខណ្ឌមិនត្រឹមត្រូវ។ វាត្រូវបានប្រើរួមជាមួយ `try-catch` ដើម្បីគ្រប់គ្រងកំហុសដែលបានបង្កើត។

ឧទាហរណ៍៖

```
function divide(a, b) {
    if (b === 0) {
        throw new Error("មិនអាចចែកជាមួយសូន្យបានទេ");
    }
    return a / b;
}

try {
    const result = divide(10, 0);
    console.log(result);
} catch (error) {
    console.log("កំហុស:", error.message);
}
```

ការពន្យល់៖ អនុគមន៍ divide ពិនិត្យប្រសិនបើ b ជាសូន្យ ហើយប្រសិនបើបែបនោះ វា throw កំហុសដោយប្រើ Error។ try-catch ចាប់កំហុសនេះ និងបោះពុម្ពសារកំហុស។ throw មានប្រយោជន៍សម្រាប់ការពិនិត្យលក្ខខណ្ឌផ្ទាល់ខ្លួន។

លទ្ធផល៖

```
កំហុស: មិនអាចចែកជាមួយសូន្យបានទេ
```

10.3 Custom Error Objects - វត្ថុកំហុសផ្ទាល់ខ្លួន

Custom error objects អនុញ្ញាតឱ្យអ្នកបង្កើតប្រភេទកំហុសផ្ទាល់ខ្លួនដោយពង្រីកថ្នាក់ Error។ វាមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងកំហុសជាក់លាក់ក្នុងកម្មវិធីរបស់អ្នក។

ឧទាហរណ៍៖

```
// បង្កើត Custom Error
class ValidationError extends Error {
    constructor(message, field) {
        super(message);
        this.name = "ValidationError";
        this.field = field;
    }
}

function validateInput(input) {
    if (!input) {
        throw new ValidationError("បញ្ចូលទិន្នន័យមិនត្រឹមត្រូវ", "input");
    }
    return input;
}

try {
    validateInput("");
} catch (error) {
    console.log(`កំហុស: ${error.name}, សារ: ${error.message}, វាល: ${err
}
```

ការពន្យល់៖ ValidationError ពង្រីក Error ដើម្បីបង្កើតប្រភេទកំហុសផ្ទាល់ខ្លួនជាមួយលក្ខណៈសម្បត្តិបន្ថែម (field)។ validateInput throw កំហុសនេះប្រសិនបើ input ទទេ។ catch ចាប់ និងបង្ហាញព័ត៌មានលម្អិតអំពីកំហុស។ Custom errors ធ្វើឱ្យការគ្រប់គ្រងកំហុសមានភាពជាក់លាក់ជាង។

លទ្ធផល៖

```
កំហុស: ValidationError, សារ: បញ្ចូលទិន្នន័យមិនត្រឹមត្រូវ, វាល: input
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃការគ្រប់គ្រងកំហុសនៅក្នុង JavaScript៖

- `Try-Catch Block`: ចាប់ និងដោះស្រាយកំហុសដោយប្រើ `try`, `catch`, និង `finally` ដើម្បីការពារកម្មវិធីពីការគាំង។
- `Throwing Errors`: ប្រើ `throw` ដើម្បីបង្កើតកំហុសផ្ទាល់ខ្លួនសម្រាប់លក្ខខណ្ឌមិនត្រឹមត្រូវ។
- `Custom Error Objects`: បង្កើតប្រភេទកំហុសផ្ទាល់ខ្លួនដោយពង្រីក `Error` ដើម្បីគ្រប់គ្រងស្ថានភាពជាក់លាក់។

ការគ្រប់គ្រងកំហុសគឺសំខាន់សម្រាប់ការបង្កើតកម្មវិធីដែលអាចទុកចិត្តបាន និងងាយស្រួលក្នុងការថែទាំ។

11. JSON (JavaScript Object Notation) - JSON (សញ្ញាសម្គាល់វត្ថុ JavaScript)

`JSON (JavaScript Object Notation)` គឺជាទម្រង់ទិន្នន័យស្រាលសម្រាប់ផ្លាស់ប្តូរទិន្នន័យរវាងភាគីផ្សេងៗ ដូចជារវាងអតិថិជន និងម៉ាស៊ីនមេ។ វាមានរចនាសម្ព័ន្ធងាយស្រួលយល់ និងត្រូវបានគាំទ្រយ៉ាងទូលំទូលាយ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការបំប្លែងខ្សែអក្សរ JSON ទៅជាវត្ថុ JavaScript ដោយប្រើ `JSON.parse()` និងការបំប្លែងវត្ថុ JavaScript ទៅជាខ្សែអក្សរ JSON ដោយប្រើ `JSON.stringify()`។

11.1 Parsing JSON: JSON.parse() - ការបំប្លែង JSON

`JSON.parse()` បំប្លែងខ្សែអក្សរ JSON ដែលមានទម្រង់ត្រឹមត្រូវទៅជាវត្ថុ JavaScript ឬទិន្នន័យប្រភេទផ្សេង (ដូចជាអារេ)។ វ she ត្រូវបានប្រើជាទូទៅសម្រាប់ទាញទិន្នន័យពី API។ ប្រសិនបើខ្សែអក្សរ JSON មិនត្រឹមត្រូវ `JSON.parse()` នឹងបង្កឱ្យមានកំហុស `SyntaxError`។

ឧទាហរណ៍៖

```
try {
    const jsonString = '{"name": "សុខា", "age": 25, "city": "ភ្នំពេញ"}';
    const obj = JSON.parse(jsonString);
    console.log(obj.name);
    console.log(obj.age);
```

```
    console.log(obj.city);

    // ឧទាហរណ៍ JSON មិនត្រឹមត្រូវ
    const invalidJson = '{name: "សុខា"}';
    JSON.parse(invalidJson); // នឹងបណ្តាលឱ្យមានកំហុស
} catch (error) {
    console.log("កំហុស:", error.message);
}
```

ការពន្យល់៖ jsonString គឺជាខ្សែអក្សរ JSON ដែលមានទម្រង់ត្រឹមត្រូវ។ JSON.parse() បំប្លែងវាទៅជាវត្ថុ JavaScript ដែលអនុញ្ញាតឱ្យចូលប្រើលក្ខណៈសម្បត្តិដូចជា obj.name។ នៅពេលប្រើ invalidJson ដែលខ្វះសញ្ញាសម្រង់ វាបណ្តាលឱ្យមាន SyntaxError ដែលត្រូវបានចាប់ដោយ try-catch។

លទ្ធផល៖

```
សុខា
25
ភ្នំពេញ
កំហុស: Unexpected token n in JSON at position 1
```

11.2 Stringifying JSON: JSON.stringify() - ការបំប្លែងទៅជា JSON

JSON.stringify() បំប្លែងវត្ថុ JavaScript, អារេ, ឬទិន្នន័យប្រភេទផ្សេង (ដូចជាចំនួន ឬអក្សរ) ទៅជាខ្សែអក្សរ JSON។ វាត្រូវបានប្រើជាទូទៅសម្រាប់ផ្ញើទិន្នន័យទៅ API ឬរក្សាទុកទិន្នន័យ។ អ្នកអាចបញ្ជាក់ទម្រង់ដោយប្រើប៉ារ៉ាម៉ែត្របន្ថែម។

ឧទាហរណ៍៖

```
const obj = {
    name: "វណ្ណៈ",
    age: 30,
    city: "សៀមរាប",
    hobbies: ["អានសៀវភៅ", "ធ្វើដំណើរ"]
};

// ការបំប្លែងជា JSON
const jsonString = JSON.stringify(obj);
console.log(jsonString);
```

```
// ការបំប្លែងជាមួយទម្រង់ស្អាត
const prettyJson = JSON.stringify(obj, null, 2);
console.log(prettyJson);
```

ការពន្យល់៖ `JSON.stringify(obj)` បំប្លែង `obj` ទៅជាខ្សែអក្សរ JSON ស្តង់ដារ។ ដោយប្រើ `JSON.stringify(obj, null, 2)` បន្ថែមចន្លោះសម្រាប់ទម្រង់ស្អាត (pretty-printed) ដែលធ្វើឱ្យអានបានងាយស្រួល។ លក្ខណៈសម្បត្តិដូចជា `undefined` ឬ `function` នឹងត្រូវបានលុបចោលនៅពេលបំប្លែង។

លទ្ធផល៖

```
{"name":"វណ្ណៈ","age":30,"city":"សៀមរាប","hobbies":["អានសៀវភៅ","ធ្វើដំណើ
{
  "name": "វណ្ណៈ",
  "age": 30,
  "city": "សៀមរាប",
  "hobbies": [
    "អានសៀវភៅ",
    "ធ្វើដំណើរ"
  ]
}
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃ JSON នៅក្នុង JavaScript៖

- `Parsing JSON`: ប្រើ `JSON.parse()` ដើម្បីបំប្លែងខ្សែអក្សរ JSON ទៅជាវត្ថុ JavaScript ដោយប្រើ `try-catch` ដើម្បីចាប់កំហុស។
- `Stringifying JSON`: ប្រើ `JSON.stringify()` ដើម្បីបំប្លែងវត្ថុ JavaScript ទៅជាខ្សែអក្សរ JSON ជាមួយជម្រើសសម្រាប់ទម្រង់ស្អាត។

JSON គឺជាឧបករណ៍សំខាន់សម្រាប់ការផ្លាស់ប្តូរទិន្នន័យនៅក្នុងកម្មវិធីបណ្តាញ ដែលផ្តល់នូវវិធីសាមញ្ញ និងអាចទុកចិត្តបានក្នុងការតំណាងទិន្នន័យ។

12. Web APIs - API បណ្តាញ

`Web APIs` គឺជាចំណុចប្រទាក់ដែលផ្តល់ដោយកម្មវិធីរុករកដើម្បីអន្តរកម្មជាមួយមុខងារផ្សេងៗ ដូចជាការផ្ទុកទិន្នន័យ ការទាញទិន្នន័យពីម៉ាស៊ីនមេ ឬការគូរក្រាហ្វិក។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី

localStorage និង sessionStorage, cookies, Fetch API, Geolocation API, Canvas API, និង Audio/Video API។

12.1 Local Storage and Session Storage: localStorage.setItem(), localStorage.getItem(), sessionStorage - ការផ្ទុកទិន្នន័យក្នុងមូលដ្ឋាន និងសម័យ

localStorage និង sessionStorage អនុញ្ញាតឱ្យរក្សាទុកទិន្នន័យជាគូគន្លឹះ-តម្លៃនៅក្នុងកម្មវិធីរុករក។ localStorage រក្សាទុកទិន្នន័យជាអចិន្ត្រៃយ៍រហូតដល់ត្រូវបានលុប ចំណែក sessionStorage រក្សាទុកទិន្នន័យសម្រាប់សម័យរុករកបច្ចុប្បន្នប៉ុណ្ណោះ។

ឧទាហរណ៍៖

```
// localStorage
localStorage.setItem("username", "សុខា");
console.log(localStorage.getItem("username"));

// sessionStorage
sessionStorage.setItem("sessionId", "12345");
console.log(sessionStorage.getItem("sessionId"));

// លុបទិន្នន័យ
localStorage.removeItem("username");
console.log(localStorage.getItem("username"));
```

ការពន្យល់៖ setItem រក្សាទុកគន្លឹះ-តម្លៃនៅក្នុង localStorage ឬ sessionStorage។ getItem ទាញតម្លៃដោយផ្អែកលើគន្លឹះ។ removeItem លុបគន្លឹះ-តម្លៃ។ នៅក្នុងឧទាហរណ៍ ការទាញ username បន្ទាប់ពី removeItem ប្រគល់ null។

លទ្ធផល៖

```
សុខា
12345
null
```

12.2 Cookies: Setting and Reading Cookies - ខូឃី

Cookies គឺជាទិន្នន័យតូចៗដែលរក្សាទុកនៅក្នុងកម្មវិធីរុករក និងត្រូវបានផ្ញើជាមួយសំណើ HTTP។ ពួកវាត្រូវបានប្រើសម្រាប់រក្សាទុកព័ត៌មានអ្នកប្រើ ដូចជាចំណូលចិត្ត ឬសម័យចូល។

ឧទាហរណ៍៖

```
// កំណត់ Cookie
function setCookie(name, value, days) {
    const date = new Date();
    date.setTime(date.getTime() + (days * 24 * 60 * 60 * 1000));
    const expires = "expires=" + date.toUTCString();
    document.cookie = name + "=" + value + ";" + expires + ";path=/";
}

setCookie("user", "វណ្ណៈ", 7);

// អាន Cookie
function getCookie(name) {
    const cname = name + "=";
    const decodedCookie = decodeURIComponent(document.cookie);
    const ca = decodedCookie.split(";");
    for (let c of ca) {
        c = c.trim();
        if (c.indexOf(cname) === 0) {
            return c.substring(cname.length, c.length);
        }
    }
    return "";
}

console.log(getCookie("user"));
```

ការពន្យល់៖ setCookie បង្កើត cookie ដោយកំណត់ឈ្មោះ តម្លៃ និងថ្ងៃផុតកំណត់។ getCookie អាន cookie ដោយបំបែកខ្សែអក្សរ `document.cookie` និងស្វែងរកគន្លឹះ។ ឧទាហរណ៍រក្សាទុក cookie user=វណ្ណៈ សម្រាប់ 7 ថ្ងៃ និងទាញតម្លៃរបស់វា។

លទ្ធផល៖

```
វណ្ណៈ
```

12.3 Fetch API: Making HTTP Requests - Fetch API

`Fetch API` អនុញ្ញាតឱ្យធ្វើសំណើ HTTP (ដូចជា GET ឬ POST) ដើម្បីទាញ ឬផ្ញើទិន្នន័យទៅម៉ាស៊ីនមេ។ វាប្រើ `Promises` និងគាំទ្រ `async/await` សម្រាប់ការគ្រប់គ្រងអសមកាល។

ឧទាហរណ៍៖

```
async function fetchData() {
    try {
        const response = await fetch("https://jsonplaceholder.typicode.c
        const data = await response.json();
        console.log(data.name);
    } catch (error) {
        console.log("កំហុស:", error.message);
    }
}

fetchData();
```

ការពន្យល់៖ fetch ធ្វើសំណើ GET ទៅ API សាកល្បង។ `response.json()` បំប្លែងការឆ្លើយតបទៅជាវត្ថុ JavaScript។ `async/await` ធ្វើឱ្យកូដអានបានងាយស្រួល ហើយ `try-catch` ចាប់កំហុសបណ្តាញ។ ឧទាហរណ៍ទាញឈ្មោះអ្នកប្រើពី API។

លទ្ធផល៖

```
Leanne Graham
```

12.4 Geolocation API - API ទីតាំងភូមិសាស្ត្រ

Geolocation API អនុញ្ញាតឱ្យទទួលបានទីតាំងភូមិសាស្ត្ររបស់អ្នកប្រើ (រយៈទទឹង និងរយៈបណ្តោយ) ដោយតម្រូវឱ្យមានការអនុញ្ញាតពីអ្នកប្រើ។

ឧទាហរណ៍៖

```
if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(
        position => {
            console.log("រយៈទទឹង:", position.coords.latitude);
            console.log("រយៈបណ្តោយ:", position.coords.longitude);
        },
        error => {
            console.log("កំហុស:", error.message);
        }
    );
} else {
```

```
    console.log("Geolocation មិនត្រូវបានគាំទ្រទេ");
}
```

ការពន្យល់៖ navigator.geolocation.getCurrentPosition ទាញទីតាំងបច្ចុប្បន្ន។ ប្រសិនបើជោគជ័យ វាប្រគល់វត្ថុ position ដែលមាន coords.latitude និង coords.longitude។ ប្រសិនបើបរាជ័យ (ឧ. អ្នកប្រើបដិសេធ) អនុគមន៍កំហុសនឹងប្រតិបត្តិ។

លទ្ធផល៖ (អាស្រ័យលើទីតាំងអ្នកប្រើ)

```
រយៈទទឹង: 11.562108
រយៈបណ្តោយ: 104.916144
```

12.5 Canvas API - API ផ្ទាំងក្រណាត់

Canvas API អនុញ្ញាតឱ្យគូរក្រាហ្វិក 2D នៅលើទំព័របណ្តាញដោយប្រើធាតុ <canvas>។ វាត្រូវបានប្រើសម្រាប់ហ្គេម ការមើលឃើញទិន្នន័យ ឬគំនូរអន្តរកម្ម។

ឧទាហរណ៍៖

```
<!-- index.html -->
<canvas id="myCanvas" width="200" height="100"></canvas>
```

```
const canvas = document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");

ctx.fillStyle = "blue";
ctx.fillRect(50, 20, 100, 60);

ctx.strokeStyle = "red";
ctx.lineWidth = 2;
ctx.strokeRect(20, 10, 50, 30);
```

ការពន្យល់៖ getContext("2d") បង្កើតបរិបទ 2D សម្រាប់គូរ។ fillRect គូរចតុកោណកែងបំពេញពណ៌ខៀវ។ strokeRect គូរគ្រោងចតុកោណកែងពណ៌ក្រហម។ ប៉ារ៉ាម៉ែត្រនៃ fillRect និង strokeRect គឺ (x, y, ទទឹង, កម្ពស់)។

លទ្ធផល៖ (បង្ហាញចតុកោណកែងពណ៌ខៀវបំពេញ និងចតុកោណកែងពណ៌ក្រហមជាគ្រោងនៅលើផ្ទាំងក្រណាត់)

12.6 Audio/Video API - API អូឌីយ៉ូ/វីដេអូ

Audio/Video API អនុញ្ញាតឱ្យគ្រប់គ្រងការចាក់អូឌីយ៉ូ និងវីដេអូនៅលើទំព័របណ្ដាញដោយប្រើធាតុ <audio> និង <video>។ វាផ្តល់វិធីសាស្ត្រដូចជា play() និង pause()។

ឧទាហរណ៍៖

```
<!-- index.html -->
<audio id="myAudio" src="sample.mp3"></audio>
<button onclick="playAudio()">ចាក់</button>
<button onclick="pauseAudio()">ផ្អាក</button>
```

```
const audio = document.getElementById("myAudio");

function playAudio() {
    audio.play();
    console.log("កំពុងចាក់អូឌីយ៉ូ");
}

function pauseAudio() {
    audio.pause();
    console.log("អូឌីយ៉ូត្រូវបានផ្អាក");
}
```

ការពន្យល់៖ <audio> ផ្ទុកឯកសារអូឌីយ៉ូ។ play() ចាប់ផ្តើមចាក់ និង pause() ផ្អាកអូឌីយ៉ូ។ ប៊ូតុងហៅអនុគមន៍ទាំងនេះ និងបោះពុម្ពស្ថានភាពទៅកុងសូល។ ដំណើរការស្រដៀងគ្នាអាចត្រូវបានអនុវត្តសម្រាប់ <video>។

លទ្ធផល៖ (នៅពេលចុចប៊ូតុង)

```
កំពុងចាក់អូឌីយ៉ូ
អូឌីយ៉ូត្រូវបានផ្អាក
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្ដប់លើគោលគំនិតសំខាន់ៗនៃ Web APIs នៅក្នុង JavaScript៖

- Local Storage and Session Storage: រក្សាទុកទិន្នន័យជាមួយ localStorage (អចិន្ត្រៃយ៍) និង sessionStorage (សម័យ)។
- Cookies: កំណត់ និងអានទិន្នន័យអ្នកប្រើជាមួយ document.cookie។
- Fetch API: ធ្វើសំណើ HTTP ដើម្បីទាញទិន្នន័យជាមួយ fetch និង async/await។
- Geolocation API: ទទួលបានទីតាំងអ្នកប្រើជាមួយ navigator.geolocation។
- Canvas API: គូរក្រាហ្វិក 2D ជាមួយ <canvas>។

- `Audio/Video API`: គ្រប់គ្រងការចាក់អូឌីយ៉ូ និងវីដេអូជាមួយ `play()` និង `pause()`។

Web APIs ពង្រឹងសមត្ថភាពរបស់ JavaScript ដើម្បីបង្កើតកម្មវិធីបណ្ដាញអន្តរកម្ម និងមានមុខងារពេញលេញ។

13. Regular Expressions - កន្សោមនិយ័ត

`Regular Expressions (Regex)` គឺជាលំនាំដែលប្រើសម្រាប់ផ្គូផ្គង ស្វែងរក ឬជំនួសអក្សរនៅក្នុងខ្សែអក្សរ។ នៅក្នុង JavaScript កន្សោមនិយ័តត្រូវបានប្រើសម្រាប់ការងារដូចជាការផ្ទៀងផ្ទាត់ទម្រង់បញ្ចូល ឬការស្វែងរកអក្សរជាក់លាក់។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីវាក្យសម្ព័ន្ធមូលដ្ឋាន ការធ្វើតេស្តលំនាំ និងការផ្គូផ្គង/ជំនួសខ្សែអក្សរដោយប្រើ `test()`, `exec()`, `match()`, និង `replace()`។

13.1 Basic Syntax: Literals, Metacharacters, Quantifiers - វាក្យសម្ព័ន្ធមូលដ្ឋាន

កន្សោមនិយ័តមានអក្សរត្រង់ (literals) ដែលផ្គូផ្គងអក្សរជាក់លាក់ និងអក្សរពិសេស (metacharacters) ដូចជា `.`, \d, \w។ បរិមាណ (quantifiers) ដូចជា `*`, `+`, `{n}` កំណត់ចំនួនការផ្គូផ្គង។ លំនាំត្រូវបានរុំដោយ `/` និងអាចមានទង់ (flags) ដូចជា `i` (គ្មានករណីអក្សរធំតូច) ឬ `g` (សកល)។

ឧទាហរណ៍៖

```
// អក្សរត្រង់ និង Metacharacters
const literal = /សួស្តី/;
const digit = /d+/; // ផ្គូផ្គងលេខ 1 ឬច្រើន
const word = /w+/; // ផ្គូផ្គងអក្សរ លេខ ឬ _

// Quantifiers
const oneOrMore = /a+/; // a មួយឬច្រើន
const exactlyThree = /d{3}/; // លេខ 3 ខ្ទង់
const optional = /colou?r/; // u ជាជម្រើស

console.log(literal.test("សួស្តី ពិភពលោក")); // true
console.log(digit.test("ឆ្នាំ 2025")); // true
console.log(word.test("អក្សរ_123")); // true
console.log(oneOrMore.test("aaa")); // true
console.log(exactlyThree.test("123")); // true
```

```
console.log(optional.test("color")); // true
console.log(optional.test("colour")); // true
```

ការពន្យល់៖ /សួស្តី/ ផ្គូផ្គងខ្សែអក្សរ "សួស្តី"។ \d+ ផ្គូផ្គងលេខមួយឬច្រើន។ \w+ ផ្គូផ្គងអក្សរឬលេខ។ a+ ផ្គូផ្គង "a" មួយឬច្រើន។ \d3 ផ្គូផ្គងលេខ 3 ខ្ទង់។ u? ធ្វើឱ្យ "u" ជាជម្រើស។ `test()` ប្រគល់ `true` ប្រសិនបើលំនាំផ្គូផ្គង។

លទ្ធផល៖

```
true
true
true
true
true
true
true
```

13.2 Testing Patterns: test(), exec() - ការធ្វើតេស្តលំនាំ

`test()` ពិនិត្យថាតើលំនាំផ្គូផ្គងនឹងខ្សែអក្សរឬអត់ ដោយប្រគល់ `true` ឬ `false`។ `exec()` ស្វែងរកការផ្គូផ្គង និងប្រគល់អារេនៃព័ត៌មានលម្អិត (ឬ `null` ប្រសិនបើគ្មានការផ្គូផ្គង)។

ឧទាហរណ៍៖

```
const pattern = /d+/g; // ផ្គូផ្គងលេខទាំងអស់
const text = "ឆ្នាំ 2025 និង 123";

// test()
console.log(pattern.test(text)); // true

// exec()
let result;
while ((result = pattern.exec(text)) !== null) {
    console.log(`ផ្គូផ្គង: ${result[0]}, ទីតាំង: ${result.index}`);
}
```

ការពន្យល់៖ `test()` បញ្ជាក់ថាមានលេខនៅក្នុង `text`។ `exec()` ជាមួយទង់ g ស្វែងរកការផ្គូផ្គងជាបន្តបន្ទាប់ ដោយប្រគល់អារេជាមួយនឹងខ្សែអក្សរដែលផ្គូផ្គង និងទីតាំងរបស់វា។ វដ្ត `while` បន្តរហូតដល់ `exec()` ប្រគល់ `null`។

លទ្ធផល៖

```
true
ផ្គូផ្គង: 2025, ទីតាំង: 5
ផ្គូផ្គង: 123, ទីតាំង: 13
```

13.3 Matching and Replacing: match(), replace() - ការផ្គូផ្គង និងជំនួស

match() ស្វែងរកការផ្គូផ្គងនៅក្នុងខ្សែអក្សរ និងប្រគល់អារេនៃការផ្គូផ្គងទាំងអស់។ replace() ជំនួសខ្សែអក្សរដែលផ្គូផ្គងជាមួយខ្សែអក្សរថ្មី។ ទាំងពីរនេះត្រូវបានប្រើជាទូទៅជាមួយទង់ g សម្រាប់ការផ្គូផ្គងសកល។

ឧទាហរណ៍៖

```
const text = "សួស្តី 2025! ជួបគ្នាឆ្នាំ 2026";

// match()
const numbers = text.match(/d+/g);
console.log(numbers);

// replace()
const replaced = text.replace(/d+/g, "ឆ្នាំថ្មី");
console.log(replaced);
```

ការពន្យល់៖ match(/\d+/g) ប្រគល់អារេនៃលេខទាំងអស់នៅក្នុង text។ replace(/\d+/g, "ឆ្នាំថ្មី") ជំនួសលេខទាំងអស់ជាមួយ "ឆ្នាំថ្មី"។ ទង់ g ធានាថាការផ្គូផ្គងទាំងអស់ត្រូវបានរកឃើញ មិនមែនតែដំបូងប៉ុណ្ណោះទេ។

លទ្ធផល៖

```
["2025", "2026"]
សួស្តី ឆ្នាំថ្មី! ជួបគ្នាឆ្នាំ ឆ្នាំថ្មី
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃកន្សោមនិយ័តនៅក្នុង JavaScript៖

- Basic Syntax: ប្រើអក្សរត្រង់, អក្សរពិសេស (ដូចជា \d, \w), និងបរិមាណ (ដូចជា +, {n}) ដើម្បីបង្កើតលំនាំ។
- Testing Patterns: ប្រើ test() ដើម្បីពិនិត្យការផ្គូផ្គង និង exec() ដើម្បីទទួលបានព័ត៌មានលម្អិតអំពីការផ្គូផ្គង។

- `Matching and Replacing`: ប្រើ `match()` ដើម្បីស្វែងរកការផ្គូផ្គងទាំងអស់ និង `replace()` ដើម្បីជំនួសខ្សែអក្សរដែលផ្គូផ្គង។

កន្សោមនិយ័តគឺជាឧបករណ៍ដ៏មានអានុភាពសម្រាប់ការដោះស្រាយខ្សែអក្សរ និងការផ្ទៀងផ្ទាត់ទិន្នន័យនៅក្នុងកម្មវិធី។

14. Modules in JavaScript - ម៉ូឌុលនៅក្នុង JavaScript

`Modules` នៅក្នុង JavaScript អនុញ្ញាតឱ្យអ្នកបំបែកកូដទៅជាឯកសារដាច់ដោយឡែក ដែលធ្វើឱ្យកូដមានរបៀបរៀបរយ និងងាយស្រួលថែទាំ។ ម៉ូឌុលជួយគ្រប់គ្រងភាពអាស្រ័យ និងអនុញ្ញាតឱ្យនាំចូល/នាំចេញអនុគមន៍ វត្ថុ ឬតម្លៃ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការនាំចូលនិងនាំចេញម៉ូឌុល, `dynamic imports`, ភាពខុសគ្នារវាង `export default` និង `named exports`, និងរបៀបរៀបចំរចនាសម្ព័ន្ធកូដម៉ូឌុល។

14.1 Importing and Exporting Modules - ការនាំចូលនិងនាំចេញម៉ូឌុល

JavaScript ប្រើ `import` ដើម្បីនាំចូលអនុគមន៍ វត្ថុ ឬតម្លៃពីម៉ូឌុលផ្សេង និង `export` ដើម្បីធ្វើឱ្យធាតុទាំងនោះអាចប្រើបាននៅក្នុងម៉ូឌុលផ្សេងទៀត។ ម៉ូឌុលត្រូវបានប្រើជាមួយ ES Modules (ESM) ដែលតម្រូវឱ្យប្រើ `type="module"` នៅក្នុង HTML។

ឧទាហរណ៍៖

```
// math.js
export function add(a, b) {
    return a + b;
}

export const PI = 3.14159;

// main.js
import { add, PI } from './math.js';

console.log(add(2, 3));
console.log(PI);
```

```html
<!-- index.html -->
<script type="module" src="main.js"></script>
```

ការពន្យល់៖ នៅក្នុង `math.js`, `export` ធ្វើឱ្យ `add` និង `PI` អាចប្រើបាន។ នៅក្នុង `main.js`, `import` នាំចូល `add` និង `PI` ដោយបញ្ជាក់ផ្លូវឯកសារ `./math.js`។ ឯកសារ HTML ប្រើ `type="module"` ដើម្បីបើក ESM។ ឧទាហរណ៍នេះបង្ហាញការបូក និងតម្លៃ `PI`។

លទ្ធផល៖

```
5
3.14159
```

14.2 Dynamic Imports: import() - ការនាំចូលថាមវន្ត

`import()` អនុញ្ញាតឱ្យនាំចូលម៉ូឌុលដោយថាមវន្តនៅពេលកម្មវិធីកំពុងដំណើរការ ដែលមានប្រយោជន៍សម្រាប់ការផ្ទុកម៉ូឌុលតាមតម្រូវការ (lazy loading) ឬអាស្រ័យលើលក្ខខណ្ឌ។ វាប្រគល់ `Promise` ដែលអនុញ្ញាតឱ្យប្រើ `async/await`។

ឧទាហរណ៍៖

```javascript
// utils.js
export function greet(name) {
    return `សួស្តី ${name}`;
}

// main.js
async function loadModule() {
    try {
        const module = await import('./utils.js');
        console.log(module.greet("សុខា"));
    } catch (error) {
        console.log("កំហុស:", error.message);
    }
}

loadModule();
```

```html
<!-- index.html -->
<script type="module" src="main.js"></script>
```

ការពន្យល់៖ import('./utils.js') ផ្ទុក utils.js ដោយថាមវន្ត និងប្រគល់ Promise។ await ទាញម៉ូឌុល ហើយ module.greet ត្រូវបានហៅ។ try-catch ចាប់កំហុសប្រសិនបើការផ្ទុកបរាជ័យ។ ឧទាហរណ៍នេះបង្ហាញសារស្វាគមន៍។

លទ្ធផល៖

```
សួស្តី សុខា
```

14.3 Export Default vs Named Exports - ការនាំចេញលំនាំដើម ទល់នឹងការនាំចេញដែលមានឈ្មោះ

export default នាំចេញតម្លៃលំនាំដើមតែមួយពីម៉ូឌុល ដែលអនុញ្ញាតឱ្យនាំចូលដោយគ្មានឈ្មោះជាក់លាក់។ named exports នាំចេញតម្លៃច្រើនដោយឈ្មោះជាក់លាក់ ដែលតម្រូវឱ្យនាំចូលដោយឈ្មោះដូចគ្នា។

ឧទាហរណ៍៖

```javascript
// calculations.js
export default function multiply(a, b) {
    return a * b;
}

export const divide = (a, b) => a / b;

// main.js
import multiply, { divide } from './calculations.js';

console.log(multiply(4, 5));
console.log(divide(10, 2));
```

```html
<!-- index.html -->
<script type="module" src="main.js"></script>
```

ការពន្យល់៖ export default នាំចេញ multiply ដែលនាំចូលដោយគ្មានវង់ក្រចក។ export const divide គឺជា named export ដែលនាំចូលដោយប្រើវង់ក្រចក និងឈ្មោះ divide។ ឧទាហរណ៍នេះបង្ហាញលទ្ធផលនៃការគុណ និងចែក។

លទ្ធផល៖

```
20
5
```

14.4 Modular Code Structure - រចនាសម្ព័ន្ធកូដម៉ូឌុល

ការរៀបចំកូដជាម៉ូឌុលជួយរក្សាកូដឱ្យមានរបៀបរៀបរយ និងអាចធ្វើមាត្រដ្ឋានបាន។ ម៉ូឌុលនីមួយៗគួរមានទំនួលខុសត្រូវតែមួយ ដូចជាការគណនា ការគ្រប់គ្រង DOM ឬការទាញទិន្នន័យ។ ប្រើថតដើម្បីរៀបចំម៉ូឌុល និងឯកសារ index.js ដើម្បីប្រមូលនាំចេញ។

ឧទាហរណ៍៖

```
project/
├── index.html
├── main.js
└── utils/
    ├── math.js
    └── dom.js
```

```javascript
// utils/math.js
export const sum = (a, b) => a + b;

// utils/dom.js
export function updateText(id, text) {
    document.getElementById(id).textContent = text;
}

// main.js
import { sum } from './utils/math.js';
import { updateText } from './utils/dom.js';

console.log(sum(10, 20));
updateText('myDiv', 'សួស្តី ពិភពលោក');
```

```html
<!-- index.html -->
<div id="myDiv"></div>
<script type="module" src="main.js"></script>
```

ការពន្យល់៖ ថត utils រៀបចំម៉ូឌុល math.js (សម្រាប់ការគណនា) និង dom.js (សម្រាប់ DOM)។ main.js នាំចូលម៉ូឌុលទាំងនេះដើម្បីប្រើ sum និង updateText។ ឧទាហរណ៍នេះបង្ហាញការបូក និងធ្វើបច្ចុប្បន្នភាពអក្សរនៅលើទំព័រ។

លទ្ធផល៖

```
30
(អក្សរនៅលើទំព័រ: សួស្តី ពិភពលោក)
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃម៉ូឌុលនៅក្នុង JavaScript៖

- `Importing and Exporting Modules`: ប្រើ `import` និង `export` ដើម្បីចែករំលែកកូដរវាងឯកសារ។
- `Dynamic Imports`: ប្រើ `import()` ដើម្បីផ្ទុកម៉ូឌុលដោយថាមវន្តជាមួយ `async/await`។
- `Export Default vs Named Exports`: បែងចែករវាង `export default` (លំនាំដើមតែមួយ) និង `named exports` (ច្រើនដោយឈ្មោះ)។
- `Modular Code Structure`: រៀបចំកូដជាម៉ូឌុលនៅក្នុងថតដើម្បីធ្វើឱ្យកម្មវិធីមានរបៀបរៀបរយ និងអាចធ្វើមាត្រដ្ឋានបាន�l

ម៉ូឌុលជួយបង្កើនភាពអាចថែទាំបាន និងអនុញ្ញាតឱ្យបង្កើតកម្មវិធី JavaScript ដ៏ស្មុគស្មាញបានយ៉ាងមានប្រសិទ្ធភាព។

15. Object-Oriented JavaScript - JavaScript តម្រង់ទិសវត្ថុ

ការសរសេរកម្មវិធីតម្រង់ទិសវត្ថុ (OOP) នៅក្នុង JavaScript អនុញ្ញាតឱ្យអ្នករៀបចំកូដជាវត្ថុដែលមានលក្ខណៈសម្បត្តិ និងវិធីសាស្រ្ត ដែលធ្វើឱ្យកូដមានរបៀបរៀបរយ និងអាចប្រើឡើងវិញបាន។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី `classes` និងវត្ថុ, `constructor functions`, `inheritance`, `getters` និង `setters`, និង `static methods`។

15.1 Classes and Objects - ថ្នាក់ និងវត្ថុ

`Classes` គឺជាប្លង់សម្រាប់បង្កើតវត្ថុ ដែលមានលក្ខណៈសម្បត្តិ និងវិធីសាស្រ្ត។ វត្ថុគឺជាករណីជាក់ស្តែងនៃថ្នាក់។ JavaScript ប្រើ `class` ដើម្បីកំណត់ថ្នាក់ និង `new` ដើម្បីបង្កើតវត្ថុ។

ឧទាហរណ៍៖

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
```

```
    }

    greet() {
        return `សួស្តី ខ្ញុំឈ្មោះ ${this.name}`;
    }
}

const person = new Person("សុខា", 25);
console.log(person.name);
console.log(person.age);
console.log(person.greet());
```

ការពន្យល់៖ `class Person` កំណត់ថ្នាក់ជាមួយ `constructor` ដើម្បីផ្តល់តម្លៃដល់ name និង age។ វិធីសាស្ត្រ greet ប្រគល់សារស្វាគមន៍។ `new Person("សុខា", 25)` បង្កើតវត្ថុ person ដែលអាចចូលប្រើលក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រ។

លទ្ធផល៖

```
សុខា
25
សួស្តី ខ្ញុំឈ្មោះ សុខា
```

15.2 Constructor Functions - អនុគមន៍សាងសង់

`Constructor functions` គឺជាវិធីបែបបុរាណនៅក្នុង JavaScript ដើម្បីបង្កើតវត្ថុ មុនពេលមាន `class`។ ពួកវាជាអនុគមន៍ធម្មតា ប៉ុន្តែប្រើជាមួយ `new` ដើម្បីបង្កើតវត្ថុ។

ឧទាហរណ៍៖

```
function Car(brand, model) {
    this.brand = brand;
    this.model = model;

    this.drive = function() {
        return `${this.brand} ${this.model} កំពុងបើកបរ`;
    };
}

const car = new Car("Toyota", "Camry");
console.log(car.brand);
```

```
console.log(car.model);
console.log(car.drive());
```

ការពន្យល់៖ Car គឺជាអនុគមន៍សាងសង់ដែលកំណត់ brand និង model ដោយប្រើ this។ វិធីសាស្ត្រ drive ត្រូវបានបន្ថែមទៅវត្ថុ។ new Car("Toyota", "Camry") បង្កើតវត្ថុ car ដែលអាចចូលប្រើលក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រ។

លទ្ធផល៖

```
Toyota
Camry
Toyota Camry កំពុងបើកបរ
```

15.3 Inheritance - ការទទួលមរតក

Inheritance អនុញ្ញាតឱ្យថ្នាក់មួយ (ថ្នាក់កូន) ទទួលលក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រពីថ្នាក់មួយទៀត (ថ្នាក់មេ) ដោយប្រើ extends និង super។ វាជួយឱ្យកូដអាចប្រើឡើងវិញបាន។

ឧទាហរណ៍៖

```
class Animal {
    constructor(name) {
        this.name = name;
    }

    speak() {
        return `${this.name} បញ្ចេញសំឡេង`;
    }
}

class Dog extends Animal {
    constructor(name, breed) {
        super(name);
        this.breed = breed;
    }

    speak() {
        return `${this.name} ព្រុស: វ៉ូហ្វ!`;
    }
}
```

```
const dog = new Dog("ប៊ុនធឿន", "Golden Retriever");
console.log(dog.name);
console.log(dog.breed);
console.log(dog.speak());
```

ការពន្យល់៖ Animal គឺជាថ្នាក់មេជាមួយ name និង speak។ Dog ទទួលមរតកពី Animal ដោយប្រើ extends។ super(name) ហៅ constructor របស់ Animal។ Dog បដិសេធន៍ (override) speak ដើម្បីផ្តល់សំឡេងជាក់លាក់។

លទ្ធផល៖

```
ប៊ុនធឿន
Golden Retriever
ប៊ុនធឿន ព្រុស: វ៉ូហ្វ!
```

15.4 Getters and Setters - អ្នកទទួល និងអ្នកកំណត់

Getters និង setters អនុញ្ញាតឱ្យគ្រប់គ្រងការចូលប្រើលក្ខណៈសម្បត្តិដោយប្រើ get និង set។ ពួកវាផ្តល់នូវវិធីសាស្ត្រដើម្បីទាញ ឬកំណត់តម្លៃជាមួយនឹងតក្កវិជ្ជាបន្ថែម។

ឧទាហរណ៍៖

```
class User {
    constructor(name) {
        this._name = name;
    }

    get name() {
        return this._name.toUpperCase();
    }

    set name(value) {
        if (value.length > 0) {
            this._name = value;
        } else {
            console.log("ឈ្មោះមិនអាចទទេបានទេ");
        }
    }
}

const user = new User("វណ្ណៈ");
```

```
console.log(user.name);
user.name = "សុភា";
console.log(user.name);
user.name = "";
```

ការពន្យល់៖ get name ប្រគល់ _name ជាអក្សរធំ។ set name ពិនិត្យថាតើ value មានប្រវែងលើសពី 0 មុននឹងកំណត់ _name។ ការព្យាយាមកំណត់ឈ្មោះទទេបណ្តាលឱ្យមានសារព្រមាន។ _name ប្រើអនុសញ្ញាដើម្បីបង្ហាញថាវាជាលក្ខណៈសម្បត្តិឯកជន។

លទ្ធផល៖

```
វណ្ណៈ
សុភា
ឈ្មោះមិនអាចទទេបានទេ
```

15.5 Static Methods - វិធីសាស្ត្រឋិតិវន្ត

Static methods គឺជាវិធីសាស្ត្រដែលជាកម្មសិទ្ធិរបស់ថ្នាក់ផ្ទាល់ មិនមែនជាវត្ថុនៃថ្នាក់នោះទេ។ ពួកវាត្រូវបានហៅដោយផ្ទាល់លើថ្នាក់ដោយប្រើ static។

ឧទាហរណ៍៖

```
class MathUtils {
    static square(number) {
        return number * number;
    }

    static isEven(number) {
        return number % 2 === 0;
    }
}

console.log(MathUtils.square(5));
console.log(MathUtils.isEven(4));
console.log(MathUtils.isEven(7));
```

ការពន្យល់៖ static square និង static isEven ជាវិធីសាស្ត្រឋិតិវន្តនៃ MathUtils។ ពួកវាត្រូវបានហៅដោយផ្ទាល់លើថ្នាក់ (MathUtils.square) ដោយមិនចាំបាច់បង្កើតវត្ថុ។ ឧទាហរណ៍នេះគណនាការ៉េ និងពិនិត្យថាតើលេខជាចំនួនគូឬអត់។

លទ្ធផល៖

```
25
true
false
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃការសរសេរកម្មវិធីតម្រង់ទិសវត្ថុនៅក្នុង JavaScript៖

- `Classes and Objects`: ប្រើ `class` និង `new` ដើម្បីបង្កើតវត្ថុជាមួយលក្ខណៈសម្បត្តិ និងវិធីសាស្ត្រ។
- `Constructor Functions`: បង្កើតវត្ថុដោយប្រើអនុគមន៍សាងសង់ និង `new` ជាវិធីបែបបុរាណ។
- `Inheritance`: ប្រើ `extends` និង `super` ដើម្បីឱ្យថ្នាក់កូនទទួលមរតកពីថ្នាក់មេ។
- `Getters and Setters`: ប្រើ `get` និង `set` ដើម្បីគ្រប់គ្រងការចូលប្រើលក្ខណៈសម្បត្តិ។
- `Static Methods`: ប្រើ `static` ដើម្បីបង្កើតវិធីសាស្ត្រដែលហៅលើថ្នាក់ផ្ទាល់។

OOP នៅក្នុង JavaScript ជួយបង្កើតកូដដែលមានរបៀបរៀបរយ អាចប្រើឡើងវិញបាន និងងាយស្រួលថែទាំ។

16. Error Handling & Debugging - ការគ្រប់គ្រងកំហុស និងការកែកូដ

ការគ្រប់គ្រងកំហុស និងការកែកូដនៅក្នុង JavaScript គឺជាជំនាញសំខាន់សម្រាប់កំណត់ និងដោះស្រាយបញ្ហានៅក្នុងកម្មវិធី។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពីការប្រើ `Console API` ដើម្បីតាមដានទិន្នន័យ, បច្ចេកទេសកែកូដដូចជា `breakpoints`, ការបង្កើតកំហុសផ្ទាល់ខ្លួន, និងការយល់ដឹងអំពី `error stacks` ដើម្បីតាមដានប្រភពកំហុស។

16.1 Console API: console.log(), console.error(), console.table() - API កុងសូល

`Console API` ផ្តល់វិធីសាស្ត្រដូចជា `console.log()`, `console.error()`, និង `console.table()` ដើម្បីបោះពុម្ពទិន្នន័យទៅកុងសូលរបស់កម្មវិធីរុករក ដែលជួយតាមដានតម្លៃ និងកំហុសអំឡុងពេលកែកូដ។

ឧទាហរណ៍៖

```
const user = { name: "សុខា", age: 25, city: "ភ្នំពេញ" };

// console.log()
console.log("អ្នកប្រើ:", user);
```

```
// console.error()
console.error("កំហុស: ទិន្នន័យមិនត្រឹមត្រូវ");

// console.table()
console.table(user);
```

ការពន្យល់៖ console.log() បោះពុម្ពវត្ថុ user ទៅកុងសូល។ console.error() បង្ហាញសារកំហុសជាពណ៌ក្រហម។ console.table() បង្ហាញ user ជាតារាង ដែលងាយស្រួលសម្រាប់មើលលក្ខណៈសម្បត្តិ។

លទ្ធផល៖

```
អ្នកប្រើ: { name: "សុខា", age: 25, city: "ភ្នំពេញ" }
កំហុស: ទិន្នន័យមិនត្រឹមត្រូវ

┌─────────┬──────────┐
│ (index) │  Values  │
├─────────┼──────────┤
│  name   │  សុខា    │
│   age   │   25     │
│  city   │ ភ្នំពេញ   │
└─────────┴──────────┘
```

16.2 Debugging Techniques: Breakpoints, Stepping Through Code - បច្ចេកទេសកែកូដ

ការកែកូដប្រើឧបករណ៍ដូចជា DevTools នៅក្នុងកម្មវិធីរុករក (Chrome, Firefox) ដើម្បីកំណត់បញ្ហា។ Breakpoints ផ្អាកការប្រតិបត្តិកូដ និងអនុញ្ញាតឱ្យពិនិត្យតម្លៃអថេរ។ Stepping through code អនុញ្ញាតឱ្យអ្នកឆ្លោះទៅមុខបន្ទិចម្តងៗដើម្បីតាមដានលំហូរកម្មវិធី។

ឧទាហរណ៍៖

```
function calculateTotal(items) {
    let total = 0;
    debugger; // ផ្អាកនៅទីនេះ
    for (let item of items) {
        total += item.price;
    }
    return total;
}

const items = [
    { name: "សៀវភៅ", price: 10 },
```

```
    { name: "ប៊ិច", price: 2 }
];

console.log(calculateTotal(items));
```

ការពន្យល់៖ debugger; ផ្អាកការប្រតិបត្តិនៅក្នុង DevTools នៅចំណុចនេះ។ អ្នកអាចបើក DevTools (F12) និងប្រើផ្ទាំង "Sources" ដើម្បីកំណត់ breakpoints ដោយចុចលើលេខបន្ទាត់កូដ។ បន្ទាប់មក អ្នកអាចប្រើប៊ូតុង "Step Over" ឬ "Step Into" ដើម្បីឆ្លោះទៅមុខបន្តិចម្តងៗ និងពិនិត្យតម្លៃ total និង item។ ឧទាហរណ៍នេះគណនាតម្លៃសរុបនៃធាតុ។

លទ្ធផល៖

```
12
```

16.3 Throwing Custom Errors - ការបង្កើតកំហុសផ្ទាល់ខ្លួន

ការបង្កើតកំហុសផ្ទាល់ខ្លួនដោយប្រើ throw និងថ្នាក់ Error អនុញ្ញាតឱ្យអ្នកបង្កើតសារកំហុសជាក់លាក់សម្រាប់ស្ថានភាពផ្សេងៗ។ វាត្រូវបានប្រើរួមជាមួយ try-catch ដើម្បីគ្រប់គ្រងកំហុស។

ឧទាហរណ៍៖

```
class ValidationError extends Error {
    constructor(message, field) {
        super(message);
        this.name = "ValidationError";
        this.field = field;
    }
}

function validateInput(input) {
    if (!input) {
        throw new ValidationError("បញ្ចូលទិន្នន័យមិនត្រឹមត្រូវ", "input");
    }
    return input;
}

try {
    validateInput("");
} catch (error) {
```

```
    console.error(`កំហុស: ${error.name}, សារ: ${error.message}, វាល: ${e
}
```

ការពន្យល់៖ ValidationError ពង្រីក Error ដើម្បីបង្កើតកំហុសផ្ទាល់ខ្លួនជាមួយលក្ខណៈសម្បត្តិ field។ validateInput បោះកំហុសប្រសិនបើ input ទទេ។ try-catch ចាប់កំហុស និងបង្ហាញព័ត៌មានលម្អិត។

លទ្ធផល៖

```
កំហុស: ValidationError, សារ: បញ្ចូលទិន្នន័យមិនត្រឹមត្រូវ, វាល: input
```

16.4 Error Stacks - ជង់កំហុស

Error stacks ផ្តល់នូវដាននៃការហៅអនុគមន៍ដែលនាំទៅដល់កំហុស ដែលជួយកំណត់ទីតាំងបញ្ហានៅក្នុងកូដ។ លក្ខណៈសម្បត្តិ stack នៃវត្ថុ Error ផ្ទុកព័ត៌មាននេះ។

ឧទាហរណ៍៖

```
function divide(a, b) {
    if (b === 0) {
        throw new Error("មិនអាចចែកជាមួយសូន្យបានទេ");
    }
    return a / b;
}

function calculate(a, b) {
    return divide(a, b);
}

try {
    calculate(10, 0);
} catch (error) {
    console.error("កំហុស:", error.message);
    console.log("ជង់កំហុស:", error.stack);
}
```

ការពន្យល់៖ divide បោះកំហុសប្រសិនបើ b ជាសូន្យ�l calculate ហៅ divide។ នៅពេលកំហុសកើតឡើង try-catch ចាប់វា និងបោះពុម្ព message និង stack។ stack បង្ហាញថា divide និង calculate ត្រូវបានហៅនៅទីតាំងណា។

លទ្ធផល៖ (ជង់កំហុសអាចប្រែប្រួលអាស្រ័យលើបរិស្ថាន)

```
កំហុស: មិនអាចចែកជាមួយសូន្យបានទេ
ជង់កំហុស: Error: មិនអាចចែកជាមួយសូន្យបានទេ
    at divide (script.js:3:15)
    at calculate (script.js:8:12)
    at script.js:12:5
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគោលគំនិតសំខាន់ៗនៃការគ្រប់គ្រងកំហុស និងការកែកូដនៅក្នុង JavaScript៖

- `Console API`: ប្រើ `console.log()`, `console.error()`, និង `console.table()` ដើម្បីតាមដានទិន្នន័យ និងកំហុស។
- `Debugging Techniques`: ប្រើ `breakpoints` និង `stepping through code` នៅក្នុង DevTools ដើម្បីកំណត់បញ្ហា។
- `Throwing Custom Errors`: បង្កើតកំហុសផ្ទាល់ខ្លួនជាមួយ `throw` និង `Error` សម្រាប់ការគ្រប់គ្រងជាក់លាក់។
- `Error Stacks`: ប្រើ `error.stack` ដើម្បីតាមដានប្រភពកំហុសតាមរយៈការហៅអនុគមន៍។

ការគ្រប់គ្រងកំហុស និងការកែកូដជួយឱ្យអ្នកបង្កើតកម្មវិធីដែលអាចទុកចិត្តបាន និងងាយស្រួលក្នុងការថែទាំ។

17. JavaScript Design Patterns - គំរូរចនានៅក្នុង JavaScript

`Design Patterns` គឺជាដំណោះស្រាយដែលអាចប្រើឡើងវិញបានសម្រាប់បញ្ហាទូទៅនៅក្នុងការសរសេរកម្មវិធី។ នៅក្នុង JavaScript គំរូទាំងនេះជួយរៀបចំកូដឱ្យមានរបៀបរៀបរយ អាចថែទាំបាន និងអាចធ្វើមាត្រដ្ឋានបាន។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី `Module Pattern`, `Singleton Pattern`, `Observer Pattern`, `Factory Pattern`, និង `Decorator Pattern`។

17.1 Module Pattern - គំរូម៉ូឌុល

`Module Pattern` ប្រើ IIFE (Immediately Invoked Function Expression) ដើម្បីបង្កើតម៉ូឌុលឯកជន និងសាធារណៈ។ វាលាក់ទិន្នន័យឯកជន និងបង្ហាញតែអនុគមន៍ ឬលក្ខណៈសម្បត្តិដែលចង់ឱ្យខាងក្រៅចូលប្រើ។

ឧទាហរណ៍៖

```
const CounterModule = (function() {
    let count = 0; // ឯកជន

    return {
        increment: function() {
            count++;
            return count;
        },
        getCount: function() {
            return count;
        }
    };
})();

console.log(CounterModule.increment());
console.log(CounterModule.increment());
console.log(CounterModule.getCount());
console.log(CounterModule.count); // មិនអាចចូលប្រើបាន
```

ការពន្យល់៖ count គឺជាអថេរឯកជននៅក្នុង IIFE។ វត្ថុដែលប្រគល់មកមានវិធីសាស្ត្រ increment និង getCount ដែលជាសាធារណៈ។ ការព្យាយាមចូលប្រើ count ដោយផ្ទាល់ប្រគល់ undefined ដោយសារវាត្រូវបានលាក់។

លទ្ធផល៖

```
1
2
2
undefined
```

17.2 Singleton Pattern - គំរូតែមួយ

Singleton Pattern ធានាថាថ្នាក់ ឬវត្ថុមានករណីតែមួយគត់ និងផ្តល់ចំណុចចូលប្រើសកល។ វាមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងធនធានរួម ដូចជាការកំណត់រចនាសម្ព័ន្ធ។

ឧទាហរណ៍៖

```
const Singleton = (function() {
    let instance;
```

```javascript
    function createInstance() {
        return {
            config: { theme: "ភ្លឺ" },
            getConfig: function() {
                return this.config;
            }
        };
    }

    return {
        getInstance: function() {
            if (!instance) {
                instance = createInstance();
            }
            return instance;
        }
    };
})();

const config1 = Singleton.getInstance();
const config2 = Singleton.getInstance();

console.log(config1.getConfig());
console.log(config2.getConfig());
console.log(config1 === config2);
```

ការពន្យល់៖ `getInstance` ពិនិត្យថាតើ `instance` មានហើយឬនៅ។ ប្រសិនបើមិនទាន់មាន វាបង្កើតករណីថ្មី។ ការហៅ `getInstance` ច្រើនដងប្រគល់ករណីដូចគ្នា។ `config1 === config2` បញ្ជាក់ថាទាំងពីរជាករណីតែមួយ។

លទ្ធផល៖

```
{ theme: "ភ្លឺ" }
{ theme: "ភ្លឺ" }
true
```

17.3 Observer Pattern - គំរូអ្នកសង្កេត

`Observer Pattern` អនុញ្ញាតឱ្យវត្ថុមួយ (អ្នកសង្កេត) តាមដានការផ្លាស់ប្តូរនៅក្នុងវត្ថុមួយទៀត (ប្រធានបទ)។ វាមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងព្រឹត្តិការណ៍ ដូចជាការធ្វើបច្ចុប្បន្នភាព UI។

ឧទាហរណ៍៖

```
class Subject {
    constructor() {
        this.observers = [];
    }

    addObserver(observer) {
        this.observers.push(observer);
    }

    notify(data) {
        this.observers.forEach(observer => observer.update(data));
    }
}

class Observer {
    update(data) {
        console.log(`ទទួលបានទិន្នន័យ: ${data}`);
    }
}

const subject = new Subject();
const observer1 = new Observer();
const observer2 = new Observer();

subject.addObserver(observer1);
subject.addObserver(observer2);
subject.notify("សួស្តី ពិភពលោក");
```

ការពន្យល់៖ Subject រក្សាបញ្ជីនៃ observers។ addObserver បន្ថែមអ្នកសង្កេត។ notify ហៅ update លើអ្នកសង្កេតទាំងអស់ជាមួយទិន្នន័យ។ ឧទាហរណ៍នេះបង្ហាញអ្នកសង្កេតពីរនាក់ទទួលសារពី subject។

លទ្ធផល៖

```
ទទួលបានទិន្នន័យ: សួស្តី ពិភពលោក
ទទួលបានទិន្នន័យ: សួស្តី ពិភពលោក
```

17.4 Factory Pattern - គំរូរោងចក្រ

Factory Pattern បង្កើតវត្ថុដោយមិនបញ្ជាក់ថ្នាក់ជាក់លាក់។ វាប្រើអនុគមន៍រោងចក្រ ឬថ្នាក់ដើម្បីបង្កើតវត្ថុផ្សេងៗគ្នាដោយផ្អែកលើលក្ខខណ្ឌ។

ឧទាហរណ៍៖

```
class Car {
    constructor(model) {
        this.model = model;
    }

    drive() {
        return `${this.model} កំពុងបើកបរ`;
    }
}

class Truck {
    constructor(model) {
        this.model = model;
    }

    haul() {
        return `${this.model} កំពុងដឹកទំនិញ`;
    }
}

class VehicleFactory {
    static createVehicle(type, model) {
        switch (type) {
            case "car":
                return new Car(model);
            case "truck":
                return new Truck(model);
            default:
                throw new Error("ប្រភេទយានយន្តមិនស្គាល់");
        }
    }
}

const car = VehicleFactory.createVehicle("car", "Toyota");
const truck = VehicleFactory.createVehicle("truck", "Isuzu");
```

```
console.log(car.drive());
console.log(truck.haul());
```

ការពន្យល់៖ VehicleFactory បង្កើតវត្ថុ Car ឬ Truck ដោយផ្អែកលើ type។ createVehicle ប្រគល់វត្ថុសមស្រប ដែលអនុញ្ញាតឱ្យកូដអតិថិជនមិនចាំបាច់ដឹងពីថ្នាក់ជាក់លាក់។

លទ្ធផល៖

```
Toyota កំពុងបើកបរ
Isuzu កំពុងដឹកទំនិញ
```

17.5 Decorator Pattern - គំរូតុបតែង

Decorator Pattern អនុញ្ញាតឱ្យបន្ថែមមុខងារទៅវត្ថុដោយមិនកែប្រែកូដដើម។ នៅក្នុង JavaScript វាត្រូវបានអនុវត្តដោយប្រើអនុគមន៍ ឬវត្ថុដើម្បីរុំវត្ថុដើម។

ឧទាហរណ៍៖

```
class Coffee {
    cost() {
        return 5;
    }

    description() {
        return "កាហ្វេធម្មតា";
    }
}

function addMilk(coffee) {
    return {
        cost: function() {
            return coffee.cost() + 2;
        },
        description: function() {
            return `${coffee.description()}, បន្ថែមទឹកដោះគោ`;
        }
    };
}

function addSugar(coffee) {
    return {
```

```
        cost: function() {
            return coffee.cost() + 1;
        },
        description: function() {
            return `${coffee.description()}, បន្ថែមស្ករ`;
        }
    };
}

let coffee = new Coffee();
coffee = addMilk(coffee);
coffee = addSugar(coffee);

console.log(coffee.description());
console.log(coffee.cost());
```

ការពន្យល់៖ Coffee គឺជាវត្ថុមូលដ្ឋាន។ addMilk និង addSugar ជាអ្នកតុបតែងដែលបន្ថែមតម្លៃ និងការពិពណ៌នា។ ការអនុវត្តអ្នកតុបតែងជាបន្តបន្ទាប់បង្កើតកាហ្វេជាមួយទឹកដោះគោ និងស្ករ។

លទ្ធផល៖

```
កាហ្វេធម្មតា, បន្ថែមទឹកដោះគោ, បន្ថែមស្ករ
8
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើគំរូរចនាសំខាន់ៗនៅក្នុង JavaScript៖

- `Module Pattern`: ប្រើ IIFE ដើម្បីបង្កើតម៉ូឌុលជាមួយទិន្នន័យឯកជន និងសាធារណៈ។
- `Singleton Pattern`: ធានាករណីតែមួយនៃវត្ថុជាមួយចំណុចចូលប្រើសកល។
- `Observer Pattern`: អនុញ្ញាតឱ្យអ្នកសង្កេតតាមដានការផ្លាស់ប្តូរនៅក្នុងប្រធានបទ។
- `Factory Pattern`: បង្កើតវត្ថុផ្សេងៗគ្នាដោយប្រើរោងចក្រដោយផ្អែកលើលក្ខខណ្ឌ។
- `Decorator Pattern`: បន្ថែមមុខងារទៅវត្ថុដោយមិនកែប្រែកូដដើម។

គំរូរចនាទាំងនេះជួយបង្កើតកូដដែលមានរបៀបរៀបរយ អាចថែទាំបាន និងអាចបត់បែនបាន។

18. Advanced JavaScript Concepts - គោលគំនិតកម្រិតខ្ពស់នៅក្នុង JavaScript

គោលគំនិតកម្រិតខ្ពស់នៅក្នុង JavaScript ជួយអ្នកសរសេរកូដដែលមានប្រសិទ្ធភាព អាចបត់បែនបាន និងអាចគ្រប់គ្រងស្ថានភាពស្មុគស្មាញ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី closure, currying, higher-order functions, memoization, debouncing និង throttling, async iterators, និង generators។

18.1 Closure - ការបិទ

Closure គឺជាអនុគមន៍ដែលចងចាំវិសាលភាពរបស់វា ទោះបីជាវាត្រូវបានប្រតិបត្តិនៅកន្លែងផ្សេងក៏ដោយ។ វាអនុញ្ញាតឱ្យរក្សាទិន្នន័យឯកជន និងបង្កើតអនុគមន៍ដែលមានអាកប្បកិរិយាជាក់លាក់។

ឧទាហរណ៍៖

```
function createCounter() {
    let count = 0;
    return function() {
        count++;
        return count;
    };
}

const counter1 = createCounter();
const counter2 = createCounter();

console.log(counter1());
console.log(counter1());
console.log(counter2());
```

ការពន្យល់៖ createCounter បង្កើត closure ដែលរក្សា count។ អនុគមន៍ដែលប្រគល់មកនៅតែចូលប្រើ count បាន។ counter1 និង counter2 មាន count ឯករាជ្យរៀងៗខ្លួន។

លទ្ធផល៖

```
1
2
1
```

18.2 Currying - ការកាត់អនុគមន៍

Currying គឺជាបច្ចេកទេសបំបែកអនុគមន៍ដែលទទួលអាគុយម៉ង់ច្រើនទៅជាអនុគមន៍ជាបន្តបន្ទាប់ដែលទទួលអាគុយម៉ង់មួយៗ។ វាធ្វើឱ្យកូដអាចបត់បែន និងអាចប្រើឡើងវិញបាន។

ឧទាហរណ៍៖

```
function curryAdd(a) {
    return function(b) {
        return function(c) {
            return a + b + c;
        };
    };
}

const add5 = curryAdd(5);
const add5and3 = add5(3);

console.log(add5and3(2));
console.log(curryAdd(1)(2)(3));
```

ការពន្យល់៖ curryAdd បំបែកអនុគមន៍បូកទៅជាអនុគមន៍បីដែលទទួលអាគុយម៉ង់ a, b, និង c។ add5 ជា closure ដែលចង a=5។ ការហៅបន្តបន្ទាប់បន្ថែម b និង c។

លទ្ធផល៖

```
10
6
```

18.3 Higher-Order Functions - អនុគមន៍កម្រិតខ្ពស់

Higher-Order Functions គឺជាអនុគមន៍ដែលទទួលអនុគមន៍ជាអាគុយម៉ង់ ឬប្រគល់អនុគមន៍។ ពួកវាត្រូវបានប្រើសម្រាប់អនុគមន៍ដូចជា map, filter, និងការបង្កើតអនុគមន៍ថ្មី។

ឧទាហរណ៍៖

```
function withLogging(fn) {
    return function(...args) {
        console.log(`ហៅ ${fn.name} ជាមួយ ${args}`);
        return fn(...args);
    };
}

function multiply(a, b) {
    return a * b;
}
```

```
const loggedMultiply = withLogging(multiply);

console.log(loggedMultiply(3, 4));
console.log(multiply(3, 4));
```

ការពន្យល់៖ withLogging គឺជា higher-order function ដែលរុំ fn ដើម្បីបន្ថែមការកត់ត្រា។ loggedMultiply ប្រតិបត្តិ multiply ជាមួយការកត់ត្រាបន្ថែម។ multiply ដំណើរការដូចធម្មតាដោយគ្មានការកត់ត្រា។

លទ្ធផល៖

```
ហៅ multiply ជាមួយ 3,4
12
12
```

18.4 Memoization - ការចងចាំ

Memoization គឺជាបច្ចេកទេសរក្សាទុកលទ្ធផលនៃការគណនាដែលមានតម្លៃថ្លៃ ដើម្បីជៀសវាងការគណនាឡើងវិញ។ វាប្រើ closure ដើម្បីរក្សាឃ្លាំងសម្ងាត់។

ឧទាហរណ៍៖

```
function memoize(fn) {
    const cache = {};
    return function(...args) {
        const key = JSON.stringify(args);
        if (key in cache) {
            return cache[key];
        }
        cache[key] = fn(...args);
        return cache[key];
    };
}

function fibonacci(n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

const memoFibonacci = memoize(fibonacci);
```

```
console.log(memoFibonacci(10));
console.log(memoFibonacci(10)); // ប្រើឃ្លាំងសម្ងាត់
```

ការពន្យល់៖ memoize បង្កើត closure ជាមួយ cache។ វារក្សាទុកលទ្ធផលរបស់ fibonacci ដោយប្រើអាគុយម៉ង់ជា key�l ការហៅ memoFibonacci(10) លើកទីពីរប្រើឃ្លាំងសម្ងាត់ ដែលបង្កើនល្បឿន។

លទ្ធផល៖

```
55
55
```

18.5 Debouncing and Throttling - ការពន្យាពេល និងការកំណត់អត្រា

Debouncing ពន្យាពេលការប្រតិបត្តិអនុគមន៍រហូតដល់គ្មានការហៅបន្ថែមក្នុងរយៈពេលជាក់លាក់។ Throttling កំណត់អត្រាការប្រតិបត្តិអនុគមន៍ឱ្យមានចន្លោះពេលថេរ។ ទាំងពីរនេះមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងព្រឹត្តិការណ៍ UI។

ឧទាហរណ៍៖

```
function debounce(fn, delay) {
    let timeout;
    return function(...args) {
        clearTimeout(timeout);
        timeout = setTimeout(() => fn(...args), delay);
    };
}

function throttle(fn, limit) {
    let inThrottle;
    return function(...args) {
        if (!inThrottle) {
            fn(...args);
            inThrottle = true;
            setTimeout(() => inThrottle = false, limit);
        }
    };
}

const log = () => console.log("ហៅអនុគមន៍");
```

```
const debouncedLog = debounce(log, 1000);
const throttledLog = throttle(log, 1000);

// ការហៅច្រើនដង ប៉ុន្តែប្រតិបត្តិតែម្តងបន្ទាប់ពី 1 វិនាទី
debouncedLog();
debouncedLog();
debouncedLog();

// ការហៅច្រើនដង ប៉ុន្តែប្រតិបត្តិរៀងរាល់ 1 វិនាទី
throttledLog();
throttledLog();
throttledLog();
```

ការពន្យល់៖ debounce ធានាថា `log` ត្រូវបានប្រតិបត្តិតែម្តងបន្ទាប់ពីរង់ចាំ 1 វិនាទីដោយគ្មានការហៅបន្ថែម។ `throttle` អនុញ្ញាតឱ្យ `log` ប្រតិបត្តិរៀងរាល់ 1 វិនាទី ទោះបីមានការហៅច្រើនក៏ដោយ។

លទ្ធផល៖ (បន្ទាប់ពី 1 វិនាទី)

```
ហៅអនុគមន៍
ហៅអនុគមន៍
```

18.6 Async Iterators - ឧបករណ៍ធ្វើអន្តរកម្មអសមកាល

`Async Iterators` អនុញ្ញាតឱ្យធ្វើអន្តរកម្មលើទិន្នន័យអសមកាល (ដូចជា Promises) ដោយប្រើ `for await...of`។ ពួកវាមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងលំហូរទិន្នន័យអសមកាល។

ឧទាហរណ៍៖

```
async function* asyncGenerator() {
    const data = [1, 2, 3];
    for (const item of data) {
        await new Promise(resolve => setTimeout(resolve, 1000));
        yield item;
    }
}

(async () => {
    for await (const value of asyncGenerator()) {
        console.log(value);
```

```
    }
})();
```

ការពន្យល់៖ asyncGenerator គឺជា generator អសមកាលដែល yield តម្លៃបន្ទាប់ពីរង់ចាំ 1 វិនាទី។ for await...of ទទួលនិងបោះពុម្ពតម្លៃនីមួយៗ។

លទ្ធផល៖ (បោះពុម្ពរៀងរាល់ 1 វិនាទី)

```
1
2
3
```

18.7 Generators (function*, yield) - ម៉ាស៊ីនបង្កើត

Generators គឺជាអនុគមន៍ដែលអាចផ្អាក និងបន្តការប្រតិបត្តិ ដោយប្រើ function* និង yield។ ពួកវាមានប្រយោជន៍សម្រាប់ការគ្រប់គ្រងលំហូរទិន្នន័យបន្តបន្ទាប់។

ឧទាហរណ៍៖

```
function* rangeGenerator(start, end) {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const range = rangeGenerator(1, 5);

console.log(range.next().value);
console.log(range.next().value);
console.log([...range]);
```

ការពន្យល់៖ rangeGenerator ប្រើ yield ដើម្បីប្រគល់តម្លៃពី start ទៅ end។ next() ទាញតម្លៃបន្ទាប់។ [...range] ប្រមូលតម្លៃដែលនៅសល់ទាំងអស់ទៅជាអារេ។

លទ្ធផល៖

```
1
2
[3, 4, 5]
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្ដប់លើគោលគំនិតកម្រិតខ្ពស់នៅក្នុង JavaScript៖

- `Closure`: អនុគមន៍ដែលចងចាំវិសាលភាពរបស់វា សម្រាប់ទិន្នន័យឯកជន។
- `Currying`: បំបែកអនុគមន៍ច្រើនអាគុយម៉ង់ទៅជាអនុគមន៍ជាបន្តបន្ទាប់។
- `Higher-Order Functions`: អនុគមន៍ដែលទទួល ឬប្រគល់អនុគមន៍។
- `Memoization`: រក្សាទុកលទ្ធផលគណនាដើម្បីបង្កើនល្បឿន។
- `Debouncing and Throttling`: គ្រប់គ្រងអត្រាការប្រតិបត្តិអនុគមន៍សម្រាប់ UI។
- `Async Iterators`: ធ្វើអន្តរកម្មលើទិន្នន័យអសមកាលជាមួយ `for await...of`។
- `Generators`: អនុគមន៍ដែលផ្អាក និងបន្តជាមួយ `yield` សម្រាប់លំហូរទិន្នន័យ។

គោលគំនិតទាំងនេះផ្តល់ឧបករណ៍ដ៏មានអានុភាពសម្រាប់ការសរសេរកម្មវិធី JavaScript ដែលស្មុគស្មាញ និងមានប្រសិទ្ធភាព។

19. Testing in JavaScript - ការធ្វើតេស្តនៅក្នុង JavaScript

ការធ្វើតេស្តនៅក្នុង JavaScript ជួយធានាថាកូដដំណើរការត្រឹមត្រូវ និងការពារកំហុសនៅពេលធ្វើការផ្លាស់ប្តូរ។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី `unit testing` ជាមួយនឹងឧបករណ៍ដូចជា `Jest`, `Mocha`, និង `Chai`, `Test-Driven Development (TDD)`, និង `mocking` ដោយប្រើ `jest.mock` និង `Sinon.js`។

19.1 Unit Testing: Frameworks like Jest, Mocha, Chai - ការធ្វើតេស្តឯកតា

`Unit Testing` ពិនិត្យផ្នែកតូចៗនៃកូដ (ឯកតា) ដូចជាអនុគមន៍ ដើម្បីធានាថាពួកវាដំណើរការត្រឹមត្រូវ។ ឧបករណ៍ដូចជា `Jest`, `Mocha`, និង `Chai` ផ្តល់រចនាសម្ព័ន្ធ និងវិធីសាស្ត្រសម្រាប់សរសេរតេស្ត។ ឧទាហរណ៍នេះប្រើ `Jest`។

ឧទាហរណ៍៖

```
// math.js
export function add(a, b) {
    return a + b;
}

// math.test.js
```

```
import { add } from './math';

describe('Math functions', () => {
    test('add should return sum of two numbers', () => {
        expect(add(2, 3)).toBe(5);
        expect(add(-1, 1)).toBe(0);
        expect(add(0, 0)).toBe(0);
    });
});
```

ការពន្យល់៖ ឯកសារ math.js មានអនុគមន៍ add។ ឯកសារ math.test.js ប្រើ Jest ដើម្បីសរសេរតេស្ត។ describe ដាក់ឈ្មោះក្រុមតេស្ត។ test ពិនិត្យលទ្ធផលនៃ add ដោយប្រើ expect និង toBe។ ករណីតេស្តគ្របដណ្តប់ចំនួនវិជ្ជមាន អវិជ្ជមាន និងសូន្យ។

លទ្ធផល៖ (នៅពេលដំណើរការ jest)

```
PASS  ./math.test.js
Math functions
  ✓ add should return sum of two numbers (2 ms)
```

19.2 Test-Driven Development (TDD) - ការអភិវឌ្ឍដឹកនាំដោយតេស្ត

Test-Driven Development (TDD) គឺជាវិធីសាស្ត្រដែលសរសេរតេស្តមុននឹងសរសេរកូដផលិតកម្ម។ វដ្តនៃ TDD គឺ៖ សរសេរតេស្ត (បរាជ័យ), សរសេរកូដ (ឆ្លងកាត់), និងធ្វើឱ្យកូដប្រសើរឡើង។ ឧទាហរណ៍នេះប្រើ Jest។

ឧទាហរណ៍៖

```
// multiply.test.js
describe('Multiply function', () => {
    test('should multiply two numbers', () => {
        expect(multiply(2, 3)).toBe(6);
        expect(multiply(0, 5)).toBe(0);
        expect(multiply(-2, 4)).toBe(-8);
    });
});

// multiply.js (បន្ទាប់ពីតេស្តបរាជ័យ)
export function multiply(a, b) {
```

```
    return a * b;
}
```

ការពន្យល់៖ ដំបូង សរសេរ `multiply.test.js` ដើម្បីពិនិត្យអនុគមន៍ `multiply` ដែលមិនទាន់មាន។ តេស្តបរាជ័យដោយសារគ្មាន `multiply`។ បន្ទាប់មក សរសេរ `multiply.js` ដើម្បីឱ្យតេស្តឆ្លងកាត់។ ជាចុងក្រោយ អាចធ្វើឱ្យកូដប្រសើរបន្ថែមបាន។ ឧទាហរណ៍នេះបង្ហាញវដ្ត TDD។

លទ្ធផល៖ (បន្ទាប់ពីសរសេរកូដ)

```
PASS  ./multiply.test.js
Multiply function
  ✓ should multiply two numbers (3 ms)
```

19.3 Mocking: jest.mock(), Sinon.js - ការក្លែងបន្លំ

Mocking ជំនួសភាពអាស្រ័យពិតប្រាកដ (ដូចជា API ឬមុខងារខាងក្រៅ) ដោយក្លែងបន្លំ ដើម្បីធ្វើតេស្តឯកតាដោយឯករាជ្យ។ `jest.mock` និង `Sinon.js` គឺជាឧបករណ៍ទូទៅ។ ឧទាហរណ៍នេះប្រើ `Jest`។

ឧទាហរណ៍៖

```
// api.js
export async function fetchData() {
    const response = await fetch('https://api.example.com/data');
    return response.json();
}

// dataService.js
import { fetchData } from './api';

export async function getUserData() {
    const data = await fetchData();
    return `អ្នកប្រើ: ${data.name}`;
}

// dataService.test.js
jest.mock('./api');

import { fetchData } from './api';
import { getUserData } from './dataService';

describe('DataService', () => {
```

```
    test('getUserData returns formatted user name', async () => {
        fetchData.mockResolvedValue({ name: 'សុខា' });
        const result = await getUserData();
        expect(result).toBe('អ្នកប្រើ: សុខា');
        expect(fetchData).toHaveBeenCalled();
    });
});
```

ការពន្យល់៖ `jest.mock('./api')` ក្លែងបន្លំ `fetchData`។ `mockResolvedValue` កំណត់ឱ្យ `fetchData` ប្រគល់ `name: 'សុខា'` ។ តេស្តពិនិត្យថា `getUserData` បង្ហាញឈ្មោះត្រឹមត្រូវ និងថា `fetchData` ត្រូវបានហៅ។ ការក្លែងបន្លំជួយជៀសវាងការហៅ API ពិតប្រាកដ។

លទ្ធផល៖

```
PASS  ./dataService.test.js
DataService
  ✓ getUserData returns formatted user name (5 ms)
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្ដប់លើគោលគំនិតសំខាន់ៗនៃការធ្វើតេស្តនៅក្នុង JavaScript៖

- `Unit Testing`: ប្រើឧបករណ៍ដូចជា `Jest`, `Mocha`, និង `Chai` ដើម្បីពិនិត្យឯកតានៃកូដ។
- `Test-Driven Development (TDD)`: សរសេរតេស្តមុន បន្ទាប់មកសរសេរកូដ និងធ្វើឱ្យប្រសើរឡើង។
- `Mocking`: ប្រើ `jest.mock` ឬ `Sinon.js` ដើម្បីក្លែងបន្លំភាពអាស្រ័យសម្រាប់តេស្តឯករាជ្យ។

ការធ្វើតេស្តជួយបង្កើនគុណភាពកូដ និងធ្វើឱ្យកម្មវិធីអាចទុកចិត្តបាន។

20. Performance Optimization - ការបង្កើនប្រសិទ្ធភាពនៃការអនុវត្ត

ការបង្កើនប្រសិទ្ធភាពនៃការអនុវត្តនៅក្នុង JavaScript ជួយធ្វើឱ្យកម្មវិធីដំណើរការលឿនជាងមុន ប្រើធនធានតិច និងផ្តល់បទពិសោធន៍អ្នកប្រើប្រាស់ប្រសើរជាងមុន។ ផ្នែកនេះនឹងពន្យល់លម្អិតអំពី `Garbage Collection` សម្រាប់ការគ្រប់គ្រងអង្គចងចាំ, `Lazy Loading`, `Event Delegation`, `Debouncing` និង `Throttling`, និងការប្រើ `async` និង `defer` សម្រាប់ស្ក្រីប។

20.1 Memory Management: Garbage Collection - ការគ្រប់គ្រងអង្គចងចាំ

Garbage Collection គឺជាដំណើរការដែល JavaScript ដោះស្រាយអង្គចងចាំដែលមិនប្រើប្រាស់ទៀត។ វាដោយស្វ័យប្រវត្តិសម្អាតវត្ថុដែលគ្មានទំណាយោងទៅ។ ការយល់ដឹងអំពីការគ្រប់គ្រងអង្គចងចាំជួយជៀសវាងការលេចធ្លាយអង្គចងចាំ។

ឧទាហរណ៍៖

```
function createObjects() {
    let obj = { data: new Array(1000000).fill('ទិន្នន័យ') }; // បម្រុងទុកអ
    obj = null; // អនុញ្ញាតឱ្យ Garbage Collector សម្អាត
}

createObjects();
console.log('វត្ថុត្រូវបានដោះលែងសម្រាប់ Garbage Collection');
```

ការពន្យល់៖ ក្នុងឧទាហរណ៍នេះ obj បម្រុងទុកអង្គចងចាំច្រើនជាមួយអារេធំ។ ការកំណត់ obj = null ដកការយោង ដែលអនុញ្ញាតឱ្យ Garbage Collector សម្អាតអង្គចងចាំ។ ការជៀសវាងការរក្សាការយោងដែលមិនចាំបាច់ជួយកាត់បន្ថយការលេចធ្លាយអង្គចងចាំ។

លទ្ធផល៖

```
វត្ថុត្រូវបានដោះលែងសម្រាប់ Garbage Collection
```

20.2 Lazy Loading - ការផ្ទុកយឺត

Lazy Loading ពន្យាពេលការផ្ទុកធនធាន (ដូចជារូបភាព ឬម៉ូឌុល JavaScript) រហូតដល់ពេលដែលត្រូវការ។ វាកាត់បន្ថយពេលវេលាផ្ទុកដំបូង និងសន្សំធនធាន។

ឧទាហរណ៍៖

```
<!DOCTYPE html>
<html>
<head>
    <title>Lazy Loading Example</title>
</head>
<body>
    <img src="placeholder.jpg" data-src="large-image.jpg" alt="រូបភាព" l
    <button onclick="loadModule()">ផ្ទុកម៉ូឌុល</button>
```

```
    <script>
        // ការផ្ទុករូបភាពយឺត
        document.querySelectorAll('img[loading="lazy"]').forEach(img =>
            img.src = img.dataset.src;
        });

        // ការផ្ទុកម៉ូឌុល JavaScript យឺត
        async function loadModule() {
            const module = await import('./heavyModule.js');
            console.log(module.default());
        }
    </script>
</body>
</html>

<!-- heavyModule.js -->
export default function() {
    return 'ម៉ូឌុលធ្ងន់ត្រូវបានផ្ទុក';
}
```

ការពន្យល់៖ រូបភាពប្រើ loading="lazy" ដើម្បីផ្ទុកនៅពេលវាចូលក្នុង viewport។ import() ផ្ទុក heavyModule.js តែនៅពេលចុចប៊ូតុង។ វិធីទាំងពីរនេះកាត់បន្ថយការប្រើប្រាស់ធនធាននៅពេលចាប់ផ្តើម។

លទ្ធផល៖ (នៅពេលចុចប៊ូតុង)

```
ម៉ូឌុលធ្ងន់ត្រូវបានផ្ទុក
```

20.3 Event Delegation - ការប្រគល់សិទ្ធិព្រឹត្តិការណ៍

Event Delegation ប្រើអ្នកស្តាប់ព្រឹត្តិការណ៍តែមួយនៅលើធាតុមេ ដើម្បីគ្រប់គ្រងព្រឹត្តិការណ៍នៃធាតុកូន។ វា កាត់បន្ថយចំនួនអ្នកស្តាប់ និងធ្វើឱ្យការគ្រប់គ្រងធាតុថ្មីកាន់តែងាយស្រួល។

ឧទាហរណ៍៖

```
<!DOCTYPE html>
<html>
<head>
    <title>Event Delegation Example</title>
</head>
<body>
```

```
    <ul id="itemList">
        <li>ធាតុ 1</li>
        <li>ធាតុ 2</li>
    </ul>
    <button onclick="addItem()">បន្ថែមធាតុ</button>

    <script>
        document.getElementById('itemList').addEventListener('click', (e
            if (event.target.tagName === 'LI') {
                console.log(`ចុចលើ: ${event.target.textContent}`);
            }
        });

        function addItem() {
            const li = document.createElement('li');
            li.textContent = `ធាតុ ${document.getElementsByTagName('li')
            document.getElementById('itemList').appendChild(li);
        }
    </script>
</body>
</html>
```

ការពន្យល់៖ អ្នកស្តាប់ click តែមួយនៅលើ ul គ្រប់គ្រងព្រឹត្តិការណ៍សម្រាប់ធាតុ li ទាំងអស់។ នៅពេលបន្ថែម li ថ្មី អ្នកស្តាប់នៅតែដំណើរការដោយមិនចាំបាច់បន្ថែមអ្នកស្តាប់ថ្មី។ វាកាត់បន្ថយការប្រើអង្គចងចាំ។

លទ្ធផល៖ (នៅពេលចុចលើ ធាតុ 1)

```
ចុចលើ: ធាតុ 1
```

20.4 Debouncing/Throttling - ការពន្យាពេល/ការកំណត់អត្រា

Debouncing ពន្យាពេលការប្រតិបត្តិអនុគមន៍រហូតដល់គ្មានការហៅបន្ថែមក្នុងរយៈពេលជាក់លាក់។ Throttling កំណត់អត្រាការប្រតិបត្តិអនុគមន៍ឱ្យមានចន្លោះពេលថេរ។ ទាំងពីរនេះកាត់បន្ថយការប្រើ CPU សម្រាប់ព្រឹត្តិការណ៍ UI ញឹកញាប់។

ឧទាហរណ៍៖

```
function debounce(fn, delay) {
    let timeout;
    return function(...args) {
```

```
        clearTimeout(timeout);
        timeout = setTimeout(() => fn(...args), delay);
    };
}

function throttle(fn, limit) {
    let inThrottle;
    return function(...args) {
        if (!inThrottle) {
            fn(...args);
            inThrottle = true;
            setTimeout(() => inThrottle = false, limit);
        }
    };
}

const logResize = () => console.log('ទំហំបង្អួចត្រូវបានផ្លាស់ប្តូរ');

const debouncedResize = debounce(logResize, 1000);
const throttledResize = throttle(logResize, 1000);

window.addEventListener('resize', debouncedResize);
window.addEventListener('resize', throttledResize);
```

ការពន្យល់៖ debounce ធានាថា `logResize` ត្រូវបានប្រតិបត្តិតែម្តងបន្ទាប់ពីរង់ចាំ 1 វិនាទីដោយគ្មានការផ្លាស់ប្តូរទំហំ។ `throttle` អនុញ្ញាតឱ្យ `logResize` ប្រតិបត្តិរៀងរាល់ 1 វិនាទី ទោះបីមានការផ្លាស់ប្តូរទំហំជាបន្តបន្ទាប់។

លទ្ធផល៖ (នៅពេលផ្លាស់ប្តូរទំហំបង្អួច)

```
ទំហំបង្អួចត្រូវបានផ្លាស់ប្តូរ
(បន្ទាប់ពី 1 វិនាទី ឬនៅចន្លោះពេល 1 វិនាទីសម្រាប់ throttledResize)
```

20.5 Async and Defer Scripts - ស្ក្រីប Async និង Defer

គុណលក្ខណៈ: `async` និង `defer` នៅក្នុង `<script>` កែលម្អពេលវេលាផ្ទុកទំព័រដោយការគ្រប់គ្រងរបៀបផ្ទុកស្ក្រីប។ `async` ផ្ទុកស្ក្រីបដោយអសមកាល និងប្រតិបត្តិភ្លាម។ `defer` ផ្ទុកស្ក្រីបដោយអសមកាល ប៉ុន្តែប្រតិបត្តិបន្ទាប់ពី DOM ត្រូវបានផ្ទុករួច។

ឧទាហរណ៍៖

```html
<!DOCTYPE html>
<html>
<head>
    <title>Async and Defer Example</title>
    <!-- ស្រីបឯករាជ្យដែលមិនអាស្រ័យលើ DOM -->
    <script async src="analytics.js"></script>
    <!-- ស្រីបដែលត្រូវការ DOM -->
    <script defer src="domScript.js"></script>
</head>
<body>
    <div id="content">សួស្តី</div>
</body>
</html>

<!-- analytics.js -->
console.log('Analytics ត្រូវបានផ្ទុក');

<!-- domScript.js -->
document.getElementById('content').textContent = 'សួស្តី ពិភពលោក';
```

ការពន្យល់៖ analytics.js ប្រើ async ព្រោះវាមិនអាស្រ័យលើ DOM និងអាចប្រតិបត្តិភ្លាម។ domScript.js ប្រើ defer ដើម្បីរង់ចាំ DOM ត្រូវបានផ្ទុក មុននឹងធ្វើការកែប្រែ។ ទាំងពីរនេះជួយកាត់បន្ថយការស្ទះការផ្ទុកទំព័រ។

លទ្ធផល៖

```
Analytics ត្រូវបានផ្ទុក
(ទំព័របង្ហាញ: សួស្តី ពិភពលោក)
```

Summary - សង្ខេប

មេរៀននេះបានគ្របដណ្តប់លើបច្ចេកទេសសំខាន់ៗសម្រាប់ការបង្កើនប្រសិទ្ធភាពនៃការអនុវត្តនៅក្នុង JavaScript៖

- Memory Management: ប្រើ Garbage Collection ដើម្បីសម្អាតអង្គចងចាំដែលមិនប្រើ និងជៀសវាងការលេចធ្លាយ។
- Lazy Loading: ពន្យាពេលការផ្ទុកធនធានរហូតដល់ត្រូវការ ដើម្បីកាត់បន្ថយពេលវេលាផ្ទុក។
- Event Delegation: ប្រើអ្នកស្តាប់តែមួយនៅលើធាតុមេ ដើម្បីគ្រប់គ្រងព្រឹត្តិការណ៍កូន។
- Debouncing/Throttling: គ្រប់គ្រងអត្រាការប្រតិបត្តិអនុគមន៍សម្រាប់ព្រឹត្តិការណ៍ UI។
- Async and Defer Scripts: ប្រើ async និង defer ដើម្បីធ្វើឱ្យការផ្ទុកស្រីបមានប្រសិទ្ធភាព។

បច្ចេកទេសទាំងនេះជួយធ្វើឱ្យកម្មវិធី JavaScript ដំណើរការលឿន និងប្រើធនធានតិច។

អំពីយើង

CodeKhmerLearning ផ្តល់ឱ្យអ្នកនូវវគ្គសិក្សាពិសេសក្នុងការរៀនកូដភាសាខ្មែរ។ យើងមានក្រុមការងារព្រមមានជំនាញក្នុងការអភិវឌ្ឍកម្មវិធី និងរេបសាយ។

© 2025 CodeKhmerLearning. All Rights Reserved.

តំណភ្ជាប់រហ័ស

ផ្ទាំងដើម
វគ្គសិក្សា
អំពីយើង
ទំនាក់ទំនង

ទំនាក់ទំនង

សូមទំនាក់ទំនងមកកាន់យើងក្តី:

Email: thymuoyhak.biu@gmail.com

Phone: +855 888 052 812

© 2025 CodeKhmerLearning. All rights reserved.